Rīgas Metālapstrādes profesionālā vidusskola

Datorzinību un dokumentu pārvaldības metodiskā komisija

Kvalifikācijas eksāmena praktiskās daļas tehniskā dokumentācija

Tīmekļa vietne lokālo serveru pārdošanai

Darba autors: Sergejs Grinkevičs

9.grupa

_____(paraksts)

Rīga, 2026

Anotācija

Serveru pārdošanas un konfigurēšanas tīmekļa vietne ir paredzēta lokālo serveru izvēlei, konfigurēšanai un pasūtīšanai atbilstoši lietotāja vajadzībām. Tīmekļa vietne ļauj apmeklētājiem apskatīt un filtrēt serveru katalogu, izvēlēties servera komponentus un pievienot konfigurāciju iepirkumu grozam. Reģistrēti lietotāji pēc e-pasta adreses apstiprināšanas var noformēt un apmaksāt pasūtījumu, lejupielādēt rēķinu PDF formātā, apskatīt pasūtījumu vēsturi un pārvaldīt savu profilu. Administratoram ir iespēja pārvaldīt serverus un to komponentus, piesaistīt komponentus konkrētiem serveriem, apstrādāt pasūtījumus un norādīt rēķinos izmantojamos uzņēmuma rekvizītus.

Tīmekļa vietnes izstrādei tika izmantota PHP programmēšanas valoda, MySQL datubāzu pārvaldības sistēma, HTML, CSS ar Tailwind CSS ietvaru, JavaScript un KnockoutJS bibliotēka. KnockoutJS tika izmantota aprēķiniem klienta pusē un dinamiskai datu atjaunošanai bez lapas pārlādēšanas. Rēķinu ģenerēšanai izmantota Dompdf bibliotēka, e-pasta sūtīšanai – PHPMailer. Sistēma ir izstrādāta pēc MVC (Model–View–Controller) arhitektūras principiem, un izstrādes vide ir konteinerizēta ar Docker.

Dokumentācija satur ievadu, kurā pamatota sistēmas aktualitāte. Uzdevuma nostādnē ir definēti projekta mērķi un galvenās funkcijas. Prasību specifikācijā aprakstītas funkcionālās un nefunkcionālās prasības. Sniegts izmantoto tehnoloģiju un izstrādes rīku izvēles pamatojums. Sistēmas modelēšanas un projektēšanas nodaļā attēlota sistēmas arhitektūra, datubāzes ER diagramma un datu struktūras, bet funkcionālajā un dinamiskajā modelī – lietojumgadījumu, aktivitāšu un stāvokļu diagrammas. Lietotāja ceļvedī aprakstītas sistēmas prasības, uzstādīšana un lietošana. Testa piemēru nodaļā apkopoti galveno funkciju pārbaudes rezultāti. Darbs noslēdzas ar secinājumiem, informācijas avotu sarakstu un pielikumiem.

Kvalifikācijas darba tehniskā dokumentācija sastāv no 87 lappusēm, 30 attēliem, 37 tabulām un 2 pielikumiem. Pielikumos ir iekļauta projekta failu struktūra un pirmkoda fragmenti.

Annotation

The server sales and configuration website is designed for selecting, configuring and ordering local servers according to the user's needs. Visitors can browse and filter the server catalogue, choose server components and add a configuration to the shopping cart. After verifying their e-mail address, registered users can place and pay for orders, download invoices as PDF files, view their order history and manage their profile. The administrator can manage servers and components, assign components to specific servers, process orders and set the company details printed on invoices.

The website was developed using PHP, the MySQL database management system, HTML, CSS with the Tailwind CSS framework, JavaScript and the KnockoutJS library. KnockoutJS is used for client-side calculations and for updating data without reloading the page. Invoices are generated with the Dompdf library and e-mails are sent with PHPMailer. The system follows the MVC (Model–View–Controller) architecture, and the development environment is containerised with Docker.

The documentation contains an introduction that explains the relevance of the system. The task definition outlines the goals and main functions of the project. The requirements specification describes the functional and non-functional requirements. The choice of technologies and development tools is justified. The modelling and design chapter presents the system architecture, the database ER diagram and the data structures, while the functional and dynamic model contains use case, activity and state diagrams. The user guide describes the system requirements, installation and use. The test example chapter summarises the results of testing the main functions. The work ends with conclusions, a list of information sources and appendices.

The technical documentation of the qualification work consists of 87 pages, 30 figures, 37 tables and 2 appendices. The appendices include the project file structure and source code fragments.

SATURS

1. IEVADS 5
2. UZDEVUMA NOSTĀDNE 7
3. PRASĪBU SPECIFIKĀCIJA 9
3.1. FUNKCIONĀLĀS PRASĪBAS 9
3.2. NEFUNKCIONĀLĀS PRASĪBAS 21
4. UZDEVUMA RISINĀŠANAS LĪDZEKĻU APRAKSTS UN IZVĒLES PAMATOJUMS 24
5. SISTĒMAS MODELĒŠANA UN PROJEKTĒŠANA 26
6. FUNKCIONĀLAIS UN DINAMISKAIS MODELIS 33
7. LIETOTĀJA CEĻVEDIS 37
8. TESTA PIEMĒRS 55
9. SECINĀJUMI 60
INFORMĀCIJAS AVOTI 61
10. PIELIKUMS 64
10.1. FAILU STRUKTŪRA 64
10.2. PIRMKODS 66

1. IEVADS

Mūsdienās pastāv liels interneta pakalpojumu daudzums digitālo datu glabāšanai un izplatīšanai – mūzika, fotogrāfijas, video, dokumenti vai pat veselas lietotnes. Ar tik lielu izvēli ir viegli apjukt, un kļūst grūti izvēlēties konkrētu servisu. Tad rodas jautājums: kam var uzticēties? Kurš nenozags manus datus? Digitālā monetizācija un kapitālisms šo izvēli padara vēl sarežģītāku, jo tagad jādomā arī par finansiālo izdevīgumu.

Šādos brīžos rodas vēlme atrast vienu risinājumu visām šīm problēmām. Un risinājums pastāv – SSH self-hosting uz sava lokālā mājas servera. Iespējas ir ierobežotas tikai ar iztēli un paša servera datora tehniskajām īpašībām. Mājas serveris ļauj glabāt visus savus vērtīgos datus uz sava paša datora, un atbildība par datu drošību gulstas uz servera īpašnieka pleciem.

Lielākā daļa serveru servisu ir atvērtu pirmkodu projekti, kas rada lielāku uzticību nekā lielie komerciālie servisi, pat ja tie var nebūt populārākie vai plašāk izplatītie.

Tomēr rodas vēl viena problēma – šādus serverus elektronikas veikalos nepārdod, tie jāveido un jāprogrammē pašam. Tādēļ man radās ideja izveidot mājaslapu serveru pārdošanai, kur pircējs varētu izvēlēties datora parametrus, atmiņas apjomu un konkrētus servisus, kas atbilst viņa vajadzībām. Šādi serveri veicina digitālo neatkarību laikā, kad internets ar katru gadu tiek ierobežots arvien vairāk, un ir aizsardzības instruments pret savu personīgo datu, paroļu vai failu pārdošanu. Lielākā daļa servisu darbojas Docker konteineros uz Debian sistēmas, tādēļ šī vide ļauj elastīgi pārvaldīt lielāko daļu konfigurāciju un iestatījumu. Tas padara visu procesu pietiekami drošu un stabilu, jo Docker nodrošina izolāciju starp konteineriem, bet Debian piedāvā uzticamu, pārbaudītu un ātru pamatu servera darbībai.

Salīdzinot personīgā servera priekšrocības ar konkurējošiem komerciālajiem pakalpojumiem, konkrēti ar mākoņglabātuvēm, izmantojot Nextcloud, lietotājam paveras ļoti plašas iespējas. Nextcloud ir ārkārtīgi elastīgs lietotāja vajadzību ziņā – pieejams liels skaits spraudņu, uzraudzības rīku, izstrādātāju rīku, bagātīgas konfigurācijas iespējas un kopumā funkcionalitāte, kuru ierobežo tikai tehniskie, nevis komerciālie faktori.

Piemēram, Dropbox Standard abonements maksā 12 eiro mēnesī par vienu lietotāju. Lietotāji savā starpā dala 3 TB atmiņas, kā arī var atjaunot dzēstos failus 180 dienu laikā. Savukārt Nextcloud ir bez maksas, lietotāju skaits nav ierobežots, bet datu glabāšana ir atkarīga no pieejamās aparatūras. Piemēram, 5 TB ārējais HDD maksā aptuveni 120–180 eiro.

Google Drive piedāvā 2 TB par aptuveni 100 eiro gadā, un atmiņa tiek koplietota arī ar citiem Google pakalpojumiem. OneDrive par līdzīgu cenu gadā piedāvā 1 TB mākoņkrātuves un Microsoft Office komplektu. iCloud piedāvā 2 TB par aptuveni 10 eiro mēnesī vai 12 TB par aptuveni 60 eiro mēnesī, taču daļa no funkcijām ir paredzēta galvenokārt Apple ierīču lietotājiem.

Salīdzinot šos pakalpojumus ar Nextcloud, galvenā atšķirība ir tāda, ka dati tiek glabāti lietotāja pārvaldītā serverī, nevis trešās puses infrastruktūrā. Tādēļ lietotājs pats ir atbildīgs par sistēmas uzturēšanu, drošību un datu rezerves kopijām.

2. UZDEVUMA NOSTĀDNE

Šī kvalifikācijas projekta mērķis ir izveidot tīmekļa vietni, kurā lietotāji var salikt serveri, kas atbilst viņu prasībām, un veikt pasūtījumu. Katalogu, konfiguratoru un iepirkumu grozu var izmantot bez reģistrācijas, taču pasūtījumu var noformēt tikai reģistrēts lietotājs, kurš ir apstiprinājis savu e-pasta adresi. Tas ļauj saglabāt kontaktinformāciju turpmākiem pasūtījumiem, sasaistīt visus pasūtījumus ar konkrētu kontu un apskatīt to vēsturi. Saskarnei jābūt skaidrai un intuitīvai, un servera komponentu izvēlei jābūt loģiskai un konsekventai. Administratoram jāspēj pārvaldīt serverus, komponentus un lietotāju pasūtījumus.

Darba ietvaros ir realizētas šādas funkcijas:

- tīmekļa vietnes apmeklētājam (viesim) ir iespēja:
  - apskatīt pieejamo serveru katalogu un filtrēt to pēc cenas, operatīvās atmiņas, datu krātuves apjoma un procesora kodolu skaita;
  - konfigurēt serveri, katram komponenta tipam (CPU, RAM, SSD, HDD, GPU) izvēloties vienu no serverim pieejamajiem komponentiem, un redzēt kopējo cenu bez lapas pārlādes;
  - pievienot konfigurēto serveri iepirkumu grozam, dzēst preces no groza vai iztukšot to;
  - izveidot lietotāja kontu.

- reģistrētam lietotājam papildus ir iespēja:
  - pieteikties sistēmā un iziet no tās; pēc pieteikšanās viesa iepirkumu grozs saglabājas;
  - apstiprināt savu e-pasta adresi un pieprasīt apstiprinājuma e-pastu atkārtoti;
  - pēc e-pasta apstiprināšanas noformēt pasūtījumu ar automātiski aizpildītu kontaktinformāciju;
  - apmaksāt vai atcelt savu pasūtījumu un saņemt rēķinu e-pastā;
  - lejupielādēt pasūtījuma rēķinu PDF formātā;
  - apskatīt savu pasūtījumu vēsturi un katra pasūtījuma detaļas;
  - rediģēt profila datus (vārdu, e-pastu, mobilo tālruni) un mainīt paroli.

- sistēmas administratoram ir iespēja:
  - pievienot, rediģēt un dzēst serverus, norādīt to iebūvēto aparatūru un augšupielādēt attēlu;
  - paslēpt serveri no kataloga vai atkal padarīt to pieejamu;
  - pievienot, rediģēt un dzēst komponentus (procesorus, videokartes, operatīvo atmiņu, SSD un HDD diskus);

- piesaistīt pieejamos komponentus konkrētiem serveriem;
- apskatīt visus pasūtījumus, filtrēt un kārtot tos, kā arī mainīt pasūtījuma statusu;
- norādīt uzņēmuma rekvizītus, kas tiek izmantoti rēķinos.

Informācija par sistēmas lietotājiem un to mijiedarbību ar sistēmu tiek apkopota lietojumgadījumu diagrammā, kas ir pieejama 6. nodaļā (skat. 6.1. att.).

3. PRASĪBU SPECIFIKĀCIJA

Šajā nodaļā aprakstītas sistēmas funkcionālās un nefunkcionālās prasības. Funkcionālās prasības sagrupētas pēc lietotāju lomām, un katra funkcija aprakstīta tabulā ar ievaddatiem, apstrādi, izvaddatiem, kļūdu paziņojumiem un izsaucamajām prasībām. Lietotāja saskarne ir angļu valodā, tādēļ kļūdu paziņojumi norādīti tādā formā, kādā tos redz lietotājs.

3.1.FUNKCIONĀLĀS PRASĪBAS

Viesu funkcijas:

Viesu funkcijas ir pieejamas jebkuram tīmekļa vietnes apmeklētājam bez autorizācijas. Viesis var apskatīt katalogu, konfigurēt serverus un ielikt tos grozā, taču pasūtījuma noformēšanai ir jāizveido konts un jāapstiprina e-pasta adrese. Viesu funkcijas var izmantot arī reģistrēti lietotāji.

Serveru kataloga apskate un filtrēšana

3.1. tabula

Serveru kataloga apskates un filtrēšanas prasību apraksts

Identifikators	PS 3.1.1
Ievads	
Funkcija paredzēta sistēmā pieejamo serveru apskatei un atlasei pēc tehniskajiem parametriem. Katalogs ir pieejams visiem apmeklētājiem bez autorizācijas.	
Ievaddati	
1. Cena “no” un “līdz” – neobligāti lauki, skaitļi eiro, ne mazāki par 0. 2. RAM apjoms “no” un “līdz” – neobligāti lauki, skaitļi gigabaitos (var būt daļskaitļi, piem. 0.5). 3. Datu krātuves apjoms “no” un “līdz” – neobligāti lauki, veseli skaitļi gigabaitos. 4. Procesora kodolu skaits – neobligāta izvēle no saraksta (“Any”, 1–8, 12, 16, 32).	
Apstrāde	
1. Pēc jebkura filtra lauka izmaiņām KnockoutJS skatu modelis klienta pusē pārbauda, vai minimālā vērtība nepārsniedz maksimālo. 2. Sistēma nosūta pieprasījumu uz `api/servers/list.php` ar norādītajiem parametriem. 3. Serverī tiek pārbaudīts, vai diapazonu vērtības ir skaitliskas un vai minimums nepārsniedz maksimumu. 4. Sistēma izveido parametrizētu SQL vaicājumu, kas atlasa tikai pieejamos serverus (available = 1), kuri atbilst filtriem, un sakārto tos no jaunākā. 5. Atlasītie serveri tiek attēloti katalogā bez lapas pārlādes.	
Izvaddati	
1. Serveru kartītes ar attēlu, nosaukumu, procesora kodolu skaitu, RAM, GPU atmiņu, krātuves apjomu un sākuma cenu. 2. Poga “Configure” pārejai uz izvēlētā servera konfiguratoru.	
Kļūdas paziņojumi	
1. “Maximum price must be greater than or equal to minimum.” (analoģiski RAM un krātuves diapazonam). 2. “Price range must be numeric” (analoģiski RAM un krātuves diapazonam).	
Izsaucamās prasības	
1. Sistēmai ir pieejams savienojums ar datubāzi. 2. Lietotāja pārlūkprogrammā ir ieslēgts JavaScript.	

Servera konfigurēšana

3.2. tabula

Servera konfigurēšanas prasību apraksts

Identifikators	PS 3.1.2
Ievads	
Funkcija ļauj izvēlēties serverim pieejamos komponentus un uzreiz redzēt konfigurācijas kopējo cenu.	
Ievaddati	
1. Servera identifikators – tiek nodots saitē no kataloga (parametrs id). 2. Katram komponenta tipam (CPU, RAM, SSD, HDD, GPU) – viens serverim pieejamais komponents vai "None".	
Apstrāde	
1. Sistēma pārbauda, vai serveris eksistē un ir pieejams. Ja nav, lietotājs tiek novirzīts uz katalogu. 2. Sistēma iegūst serverim piesaistītos komponentus no tabulas server_components un sagrupē tos pēc tipa. 3. Komponenti tiek attēloti kā radio pogas, tādēļ katram tipam var izvēlēties tikai vienu. Nepieejamie komponenti ir attēloti neaktīvi. 4. KnockoutJS skatu modelis saskaita servera pamatcenu un izvēlēto komponentu cenas un atjauno kopējo cenu bez lapas pārlādes.	
Izvaddati	
1. Servera attēls, nosaukums un iebūvētā aparatūra. 2. Komponentu saraksts pa tipiem ar piemaksu par katru komponentu. 3. Konfigurācijas kopējā cena. 4. Poga "Add to Cart" (administratoram tās vietā paziņojums, ka administratori nevar veikt pasūtījumus).	
Kļūdas paziņojumi	
1. Nav paredzēti. Ja serveris neeksistē vai nav pieejams, lietotājs tiek novirzīts uz katalogu.	
Izsaucamās prasības	
1. Serveris eksistē un ir pieejams (available = 1). 2. Sistēmai ir pieejams savienojums ar datubāzi.	

Preces pievienošana iepirkumu grozam

3.3. tabula

Preces pievienošanas iepirkumu grozam prasību apraksts

Identifikators	PS 3.1.3
Ievads	
Funkcija paredzēta konfigurēta servera pievienošanai iepirkumu grozam turpmākai pasūtījuma noformēšanai. Grozu var izmantot arī viesis, taču pasūtījuma noformēšanai jāpiesakās sistēmā.	
Ievaddati	
1. Servera identifikators. 2. Izvēlēto komponentu identifikatoru saraksts. 3. CSRF aizsardzības marķieris pieprasījuma galvenē X-CSRF-Token. 4. Cena no pārlūkprogrammas netiek pieņemta – to aprēķina serveris.	

Apstrāde
1. Sistēma pārbauda pieprasījuma metodi (POST) un CSRF marķieri. 2. Sistēma pārbauda, vai lietotājs nav administrators. 3. Sistēma pārbauda, vai serveris ir pieejams. 4. Sistēma pārbauda, vai visi izvēlētie komponenti ir pieejami, piesaistīti šim serverim un vai katram tipam izvēlēts ne vairāk kā viens komponents. 5. Sistēma aprēķina cenu no datubāzē saglabātajām servera un komponentu cenām. 6. Sistēma atrod sesijai piesaistīto iepirkumu grozu vai izveido jaunu un saglabā preci (komponentu identifikatori tiek saglabāti JSON formātā). 7. Pārlūkprogrammā tiek atjaunots un atvērts groza sānu panelis.
Izvaddati
1. Iepirkumu grozam pievienota jauna prece. 2. Atjaunināts groza saturs un kopējā summa sānu panelī. 3. Viesim groza panelī tiek rādīta poga “Sign in to Checkout”, autorizētam lietotājam – “Checkout”.
Kļūdas paziņojumi
1. “Invalid CSRF token”. 2. “Administrator accounts cannot place orders”. 3. “Server is unavailable”. 4. “One or more components are unavailable for this server”. 5. “Only one component per type can be selected”. 6. “Invalid component selection”.
Izsaucamās prasības
1. Lietotājam ir aktīva sesija. 2. Lietotājs atrodas servera konfiguratora lapā.

Iepirkumu groza pārvaldība

3.4. tabula

Iepirkumu groza pārvaldības prasību apraksts

Identifikators	PS 3.1.4
Ievads	
Funkcija paredzēta iepirkumu groza satura apskatei un pārvaldībai. Lietotājs var apskatīt pievienotās preces un to konfigurācijas, dzēst atsevišķas preces vai iztukšot visu grozu.	
Ievaddati	
1. Sesijas identifikators – lietotāja aktīvās sesijas identifikators. 2. Dzēšamās preces identifikators. 3. CSRF aizsardzības marķieris.	
Apstrāde	
1. Pēc pogas “Cart” nospiešanas sistēma iegūst sesijai piesaistīto iepirkumu grozu vai izveido jaunu. 2. Sistēma iegūst groza preces ar servera nosaukumu, attēlu un izvēlētajiem komponentiem. 3. Sistēma aprēķina groza kopējo summu. 4. Dzēšot preci, sistēma pārbauda, vai tā pieder pašreizējās sesijas grozam. 5. Pēc pogas “Clear Cart” nospiešanas un apstiprinājuma dialoga sistēma dzēš visas groza preces. 6. Pēc autorizācijas sesijas grozs tiek piesaistīts lietotāja kontam, tāpēc viesa pievienotās preces saglabājas.	
Izvaddati	
1. Groza sānu panelis vai noformēšanas lapa ar precēm, komponentiem, cenām un kopējo summu. 2. Atjaunināts groza saturs pēc preces dzēšanas vai groza iztukšošanas.	
Kļūdas paziņojumi	
1. “Item not found in current cart”. 2. “No item specified”. 3. “Invalid CSRF token”.	

Izsaucamās prasības
1. Lietotājam ir aktīva sesija. 2. Lietotājs nav autorizējies kā administrators.

Lietotāja reģistrācija

3.5. tabula

Lietotāja reģistrācijas prasību apraksts

Identifikators	PS 3.1.5
Ievads	
Funkcija paredzēta jauna lietotāja konta izveidei. Konts ar apstiprinātu e-pasta adresi ir nepieciešams pasūtījuma noformēšanai, kā arī ļauj saglabāt kontaktinformāciju un apskatīt pasūtījumu vēsturi.	
Ievaddati	
1. Vārds – obligāts lauks. 2. E-pasta adrese – obligāts lauks, korektā formātā, unikāla sistēmā. 3. Parole – obligāts lauks. 4. Paroles apstiprinājums – obligāts lauks, vērtībai jāsakrīt ar paroli.	
Apstrāde	
1. Sistēma pārbauda CSRF marķieri. 2. Sistēma pārbauda, vai obligātie lauki nav tukši. 3. Sistēma pārbauda e-pasta formātu un to, vai adrese vēl nav reģistrēta. 4. Sistēma pārbauda, vai parole sakrīt ar apstiprinājumu. 5. Parole tiek jaukta ar funkciju password_hash (bcrypt), un datubāzē tiek saglabāta tikai jaucējvērtība. 6. Sistēma ģenerē nejaušu 64 rakstzīmju apstiprināšanas marķieri un izveido kontu ar lomu “user”. 7. Sistēma nosūta e-pasta vēstuli ar adreses apstiprināšanas saiti. 8. Lietotājs tiek novirzīts uz autorizācijas lapu. Ja reģistrācija tika sākta no noformēšanas lapas, parametrs redirect tiek saglabāts, lai pēc pieteikšanās lietotājs atgrieztos pie pasūtījuma.	
Izvaddati	
1. Jauns lietotāja konts datubāzē. 2. E-pasta vēstule ar apstiprināšanas saiti. 3. Novirzīšana uz autorizācijas lapu.	
Kļūdas paziņojumi	
1. “Name is required”. 2. “Email is required”. 3. “Invalid email format”. 4. “This email is already taken”. 5. “Password is required”. 6. “Confirmation password is required”. 7. “Passwords do not match”. 8. “Invalid CSRF token”.	
Izsaucamās prasības	
1. Sistēmai ir pieejams savienojums ar datubāzi. 2. Vēstules nosūtīšanai ir konfigurēts SMTP serveris.	

Reģistrētu lietotāju funkcijas:

Šīs funkcijas ir pieejamas lietotājiem, kuri ir izveidojuši kontu un pieteikušies sistēmā. Pasūtījuma noformēšanai papildus nepieciešama apstiprināta e-pasta adrese.

Lietotāja autorizācija un iziešana

3.6. tabula

Lietotāja autorizācijas un iziešanas prasību apraksts

Identifikators	PS 3.1.6
Ievads	
Funkcija paredzēta reģistrēta lietotāja vai administratora autentifikācijai un iziešanai no sistēmas.	
Ievaddati	
1. E-pasta adrese – obligāts lauks. 2. Parole – obligāts lauks. 3. Parametrs redirect – lapa, uz kuru atgriezties pēc pieteikšanās (neobligāts). 4. Iziešanai – CSRF aizsardzības marķieris.	
Apstrāde	
1. Sistēma pārbauda CSRF marķieri. 2. Sistēma meklē lietotāju pēc e-pasta adreses un pārbauda paroli ar funkciju password_verify. 3. Veiksmīgas autentifikācijas gadījumā sesijā tiek saglabāts lietotāja identifikators, vārds, loma un e-pasta apstiprināšanas statuss. 4. Sesijai piesaistītais iepirkumu grozs tiek piesaistīts lietotāja kontam. 5. Lietotājs tiek novirzīts uz lapu, no kuras tika atvērta autorizācija (parametrs redirect). Atļautas tikai lapas checkout.php un my-orders.php, citos gadījumos – serveru katalogs. Tas novērš novirzīšanu uz svešām vietnēm. Lietotājs ar neapstiprinātu e-pastu vienmēr tiek novirzīts uz katalogu. 6. Galvenē parādās lietotāja vārds un poga “Logout”, administratoram arī izvēlne “Manage”. 7. Iziešana notiek ar POST pieprasījumu un CSRF marķieri: sesija tiek iznīcināta un sesijas sīkdatne dzēsta.	
Izvaddati	
1. Aktīva lietotāja sesija. 2. Navigācija atbilstoši lietotāja lomai.	
Kļūdas paziņojumi	
1. “Invalid email or password” – vienāds paziņojums gan nepareizai parolei, gan neeksistējošam e-pastam, lai nevarētu noskaidrot, kuras adreses ir reģistrētas. 2. “Invalid CSRF token”.	
Izsaucamās prasības	
1. Lietotājam ir izveidots konts sistēmā. 2. Sistēmai ir pieejams savienojums ar datubāzi.	

E-pasta adreses apstiprināšana

3.7. tabula

E-pasta adreses apstiprināšanas prasību apraksts

Identifikators	PS 3.1.7
Ievads	
Funkcija apstiprina, ka e-pasta adrese pieder lietotājam. Kamēr adrese nav apstiprināta, lietotājs nevar noformēt pasūtījumu, un lapas augšā tiek rādīts paziņojums.	
Ievaddati	
1. Apstiprināšanas marķieris saitē (64 heksadecimālas rakstzīmes). 2. Atkārtotai nosūtīšanai – poga “Resend verification email” paziņojuma joslā.	

Apstrāde
1. Sistēma pārbauda marķiera formātu un meklē lietotāju ar šādu marķieri. 2. Sistēma saglabā apstiprināšanas laiku (email_verified_at) un dzēš marķieri, tāpēc saite ir izmantojama tikai vienu reizi. 3. Ja lietotājs ir autorizējies, sesijas dati tiek atjaunoti un paziņojuma josla pazūd. 4. Pēc pogas "Resend verification email" nospiešanas sistēma ģenerē jaunu marķieri un nosūta jaunu vēstuli. 5. Mainot e-pasta adresi profilā, jaunā adrese ir jāapstiprina atkārtoti. Administratoriem apstiprināšana nav nepieciešama.
Izvaddati
1. Lapa "Email verified" vai "Verification failed". 2. Paziņojums "Verification email sent".
Kļūdas paziņojumi
1. "This verification link is invalid." 2. "Email could not be sent. Check the local mail configuration." 3. "Authentication required".
Izsaucamās prasības
1. Lietotājam ir konts ar neapstiprinātu e-pasta adresi. 2. Vēstules nosūtīšanai ir konfigurēts SMTP serveris.

Pasūtījuma noformēšana

3.8. tabula

Pasūtījuma noformēšanas prasību apraksts

Identifikators	PS 3.1.8
Ievads	
Funkcija paredzēta pasūtījuma izveidei no iepirkumu grozā esošajām precēm. Tā ir pieejama tikai autorizētiem lietotājiem ar apstiprinātu e-pasta adresi.	
Ievaddati	
1. Vārds un uzvārds – obligāts lauks. 2. E-pasta adrese – obligāts lauks, korektā e-pasta formātā. 3. Tālruņa numurs – neobligāts lauks, līdz 15 cipariem, var sākties ar "+". 4. Komentārs – neobligāts lauks. 5. Vārds, e-pasts un tālrunis tiek aizpildīti automātiski no lietotāja profila, un tos var mainīt.	
Apstrāde	
1. Ja lietotājs nav autorizējies, sistēma novirza viņu uz autorizācijas lapu ar parametru `redirect=checkout.php` un pēc pieteikšanās atgriež uz noformēšanas lapu ar to pašu grozu. 2. Ja lietotāja e-pasta adrese nav apstiprināta, viņš tiek novirzīts uz katalogu, kur redzams paziņojums par apstiprināšanu. 3. Pasūtījuma izveides pieprasījumā sistēma atkārtoti pārbauda CSRF marķieri, autorizāciju, lietotāja lomu (administrators pasūtīt nevar) un e-pasta apstiprinājumu. 4. Sistēma pārbauda obligātos laukus un e-pasta formātu, kā arī normalizē tālruņa numuru (atstarpes, iekavas un defises tiek noņemtas). 5. Sistēma iegūst groza preces. Ja grozs ir tukšs, pasūtījums netiek izveidots. 6. Katrai precei sistēma atkārtoti pārbauda konfigurācijas pieejamību un pārrēķina cenu. 7. Vienā datubāzes transakcijā tiek izveidots ieraksts tabulā orders ar statusu "pending" un ieraksti tabulā order_items, saglabājot servera nosaukuma, apraksta, attēla, cenas un komponentu kopijas. 8. Kļūdas gadījumā transakcija tiek atcelta, un datubāzē netiek saglabāts nekas – ne pasūtījums, ne tā preces. 9. Lietotājs tiek novirzīts uz pasūtījuma lapu. Iepirkumu grozs šajā brīdī netiek iztukšots.	
Izvaddati	
1. Izveidots pasūtījums ar statusu "Awaiting Payment". 2. Paziņojums "Order successfully created". 3. Pasūtījuma lapa ar precēm, kontaktinformāciju un pogām "Pay Now" un "Cancel Order".	

Kļūdas paziņojumi
1. “Sign in to place an order”. 2. “Authentication required”. 3. “Name and email are required”. 4. “Invalid email format”. 5. “Phone must contain up to 15 digits and may start with +”. 6. “Cart is empty”. 7. “Cart contains an unavailable configuration. Please rebuild the item.” 8. “Verify your account to complete registration and be able to place an order.” 9. “Administrator accounts cannot place orders”.
Izsaucamās prasības
1. Lietotājs ir autorizējies sistēmā, un viņa e-pasta adrese ir apstiprināta. 2. Iepirkumu grozā ir vismaz viena prece. 3. Sistēmai ir pieejams savienojums ar datubāzi.

Pasūtījuma apmaksa

3.9. tabula

Pasūtījuma apmaksas prasību apraksts

Identifikators	PS 3.1.9
Ievads	
Funkcija imitē pasūtījuma apmaksu un nosūta klientam rēķinu. Reāla maksājumu sistēma kvalifikācijas darba ietvaros nav integrēta, apmaksu apstiprina administrators.	
Ievaddati	
1. Pasūtījuma identifikators. 2. CSRF aizsardzības marķieris.	
Apstrāde	
1. Sistēma pārbauda, vai pasūtījums eksistē. 2. Sistēma pārbauda, vai pasūtījums pieder autorizētajam lietotājam (pēc lietotāja identifikatora). 3. Sistēma pārbauda, vai pasūtījuma statuss ir “pending”. 4. Pasūtījuma statuss tiek mainīts uz “processing”. 5. Sistēma ģenerē PDF rēķinu ar pārdevēja rekvizītiem un nosūta to uz pasūtījumā norādīto e-pasta adresi. 6. Iepirkumu grozs tiek iztukšots. 7. Pēc paziņojuma lapa tiek pārlādēta un parāda jauno statusu.	
Izvaddati	
1. Pasūtījuma statuss “Processing”. 2. E-pasta vēstule ar rēķinu PDF pielikumā. 3. Paziņojums “Payment received successfully. The invoice was sent to your email.”	
Kļūdas paziņojumi	
1. “Payment received and order is processing, but the invoice email could not be sent.” (brīdinājums). 2. “You can only pay for your own orders”. 3. “Cannot pay for order with status: …”. 4. “Order not found”.	
Izsaucamās prasības	
1. Lietotājs ir autorizējies un ir pasūtījuma īpašnieks. 2. Pasūtījuma statuss ir “pending”.	

Pasūtījuma atcelšana

3.10. tabula

Pasūtījuma atcelšanas prasību apraksts

Identifikators	PS 3.1.10
Ievads	
Funkcija ļauj klientam atcelt vēl neapmaksātu pasūtījumu.	
Ievaddati	
1. Pasūtījuma identifikators. 2. Lietotāja apstiprinājums dialoglodziņā. 3. CSRF aizsardzības marķieris.	
Apstrāde	
1. Sistēma parāda apstiprinājuma dialogu “Cancel this order?”. 2. Sistēma pārbauda, vai pasūtījums pieder autorizētajam lietotājam. 3. Sistēma pārbauda, vai pasūtījuma statuss ir “pending”. 4. Pasūtījuma statuss tiek mainīts uz “cancelled”. 5. Iepirkumu grozs tiek saglabāts, lai lietotājs varētu to labot un noformēt jaunu pasūtījumu.	
Izvaddati	
1. Pasūtījuma statuss “Cancelled”. 2. Paziņojums “Order successfully cancelled”.	
Kļūdas paziņojumi	
1. “You can only cancel your own orders”. 2. “Cannot cancel order with status: …”.	
Izsaucamās prasības	
1. Lietotājs ir autorizējies un ir pasūtījuma īpašnieks. 2. Pasūtījuma statuss ir “pending”.	

Rēķina lejupielāde

3.11. tabula

Rēķina lejupielādes prasību apraksts

Identifikators	PS 3.1.11
Ievads	
Funkcija ļauj lejupielādēt pasūtījuma rēķinu PDF formātā no pasūtījuma lapas vai pasūtījuma detaļu lapas.	
Ievaddati	
1. Pasūtījuma identifikators.	
Apstrāde	
1. Sistēma pārbauda, vai identifikators ir pozitīvs vesels skaitlis. 2. Sistēma iegūst pasūtījumu ar precēm un komponentiem. 3. Sistēma pārbauda piekļuves tiesības: rēķinu var lejupielādēt tikai pasūtījuma īpašnieks vai administrators. 4. Sistēma iegūst pārdevēja rekvizītus no administratora profila. 5. Sistēma aprēķina cenā iekļauto PVN 21 % apmērā un kopsummu vārdiem. 6. Sistēma ģenerē PDF failu un nosūta to pārlūkprogrammai lejupielādei (invoice-order-N.pdf).	
Izvaddati	
1. PDF rēķins ar pārdevēja un pircēja datiem, precēm un to komponentiem, summu bez PVN, PVN summu, kopsummu un kopsummu vārdiem.	
Kļūdas paziņojumi	
1. “Invalid order ID” (HTTP 400). 2. “Order not found” (HTTP 404). 3. “Access denied” (HTTP 403).	

Izsaucamās prasības
1. Lietotājs ir autorizējies. 2. Pasūtījums eksistē, un lietotājam ir tiesības to apskatīt.

Pasūtījumu vēstures apskate

3.12. tabula

Pasūtījumu vēstures apskates prasību apraksts

Identifikators	PS 3.1.12
Ievads	
Funkcija ļauj autorizētam lietotājam apskatīt savus pasūtījumus, to statusus un detalizētu informāciju par katru pasūtījumu.	
Ievaddati	
1. Lietotāja identifikators no aktīvās sesijas. 2. Pasūtījuma identifikators – detaļu apskatei.	
Apstrāde	
1. Neautorizēts lietotājs tiek novirzīts uz autorizācijas lapu, administrators – uz pasūtījumu pārvaldības lapu. 2. Sistēma iegūst lietotāja pasūtījumus ar preču skaitu, sakārtotus no jaunākā. 3. Katram statusam tiek piešķirts krāsains apzīmējums. 4. Detaļu lapā preces tiek attēlotas no pasūtījuma brīdī saglabātajām kopijām, tāpēc vēsture nemainās, ja administrators vēlāk maina vai dzēš serveri. 5. Pasūtījumiem ar statusu “pending” tiek rādīta poga “Pay Now”.	
Izvaddati	
1. Pasūtījumu tabula: identifikators, datums, preču skaits, summa, statuss un poga “Details”. 2. Pasūtījuma detaļu lapa ar precēm, komponentiem, klienta informāciju un rēķina lejupielādes pogu.	
Kļūdas paziņojumi	
1. “Access denied” – mēģinot atvērt cita lietotāja pasūtījumu. 2. Ja pasūtījumu nav, tiek rādīts paziņojums “No orders yet”.	
Izsaucamās prasības	
1. Lietotājs ir autorizējies sistēmā.	

Profila rediģēšana un paroles maiņa

3.13. tabula

Profila rediģēšanas un paroles maiņas prasību apraksts

Identifikators	PS 3.1.13
Ievads	
Funkcija ļauj lietotājam mainīt savus profila datus un paroli.	
Ievaddati	
1. Vārds – obligāts lauks, līdz 255 rakstzīmēm. 2. E-pasta adrese – obligāts lauks, korektā formātā, unikāla sistēmā. 3. Mobilais tālrunis – neobligāts lauks, līdz 15 cipariem, var sākties ar “+”. 4. Paroles maiņai – pašreizējā parole, jaunā parole (vismaz 8 rakstzīmes) un tās apstiprinājums.	
Apstrāde	
1. Sistēma pārbauda CSRF marķieri un ievadītos laukus. 2. Tālruņa numurs tiek normalizēts. 3. Ja e-pasta adrese ir mainīta, sistēma pārbauda tās unikalitāti, bet parastam lietotājam arī ģenerē jaunu apstiprināšanas marķieri un nosūta vēstuli. 4. Dati tiek saglabāti, un sesijā tiek atjaunots lietotāja vārds. 5. Paroles maiņai sistēma pārbauda pašreizējo paroli, jaunās paroles garumu un sakritību ar apstiprinājumu, pēc tam saglabā jaunās paroles jaucējvērtību.	

Izvaddati
1. “Profile updated.” vai “Profile updated. Please verify your new email address.” 2. “Password updated.”
Kļūdas paziņojumi
1. “Name is required”. 2. “Invalid email format”. 3. “Email already taken”. 4. “Phone must contain up to 15 digits and may start with +”. 5. “Current password is incorrect”. 6. “New password must be at least 8 characters”. 7. “New passwords do not match”.
Izsaucamās prasības
1. Lietotājs ir autorizējies sistēmā.

Administrācijas funkcijas:

Administrācijas funkcijas ir pieejamas tikai lietotājiem ar lomu “admin”. Katrā administrācijas lapā starpprogrammatūra (Middleware) Admin pārbauda lomu un citus lietotājus novirza uz katalogu.

Serveru pārvaldība

3.14. tabula

Serveru pārvaldības prasību apraksts

Identifikators	PS 3.1.14
Ievads	
Funkcija paredzēta serveru kataloga pārvaldībai. Administrators var pievienot, rediģēt un dzēst serverus, kā arī paslēpt tos no kataloga.	
Ievaddati	
1. Nosaukums – obligāts lauks. 2. Apraksts – obligāts lauks. 3. Pamatcena – obligāts lauks, skaitlis, ne mazāks par 0. 4. Iebūvētā aparatūra (ja atzīmēts “Has built-in hardware”): RAM GB (pozitīvs skaitlis, līdz 3 zīmēm aiz komata), krātuve GB, CPU kodoli un GPU atmiņa GB (pozitīvi veseli skaitļi); jānorāda vismaz viena vērtība. 5. Attēls – neobligāts, JPEG, PNG, WebP vai SVG fails līdz 5 MB. 6. Pieejamības statuss – tiek pārslēgts ar pogu serveru sarakstā.	
Apstrāde	
1. Sistēma pārbauda administratora lomu un CSRF marķieri. 2. Sistēma pārbauda ievadītos laukus. 3. Attēlam sistēma nosaka faila tipu pēc satura, pārbauda izmēru un attēla dimensijas (līdz 25 megapikseļiem). SVG failam tiek atstāti tikai droši elementi un atribūti, bet skripti un notikumu apstrādātāji tiek izdzēsti. 4. Attēls tiek saglabāts ar nejauši ģenerētu faila nosaukumu. 5. Sistēma saglabā jaunu serveri vai atjauno esošā servera datus. 6. Dzēšana notiek pēc apstiprinājuma dialoga. Kopā ar serveri tiek dzēstas tā komponentu piesaistes un groza preces, bet pasūtījumu vēsture saglabājas. 7. Nepieejams serveris netiek rādīts katalogā un to nevar pasūtīt.	
Izvaddati	
1. Atjaunināts serveru saraksts ar identifikatoru, nosaukumu, cenu, statusu un darbību pogām.	

Kļūdas paziņojumi
1. “Name is required”. 2. “Description is required”. 3. “Price must be a non-negative number”. 4. “Enter at least one built-in hardware value”. 5. “RAM must be positive and use at most 3 decimal places”. 6. “CPU cores must be a positive whole number” (analoģiski krātuvei un GPU atmiņai). 7. “Only valid JPEG, PNG, WebP, and sanitized SVG images are allowed”. 8. “Image must be 5 MB or smaller”. 9. “SVG is invalid or contains unsupported active content”. 10. “Invalid CSRF token”.
Izsaucamās prasības
1. Lietotājs ir autorizējies kā administrators.

Komponentu piesaiste serverim

3.15. tabula

Komponentu piesaistes serverim prasību apraksts

Identifikators	PS 3.1.15
Ievads	
Funkcija nosaka, kuri komponenti ir pieejami konkrētā servera konfiguratorā. Tajā pašā lapā var izveidot arī jaunu komponentu.	
Ievaddati	
1. Atzīmēto komponentu saraksts (izvēles rūtiņas, sagrupētas pēc tipa). 2. Jaunam komponentam – nosaukums, tips, specifikācija atbilstoši tipam un cena.	
Apstrāde	
1. Sistēma pārbauda administratora lomu un CSRF marķieri. 2. Sistēma salīdzina atzīmētos komponentus ar esošajām piesaistēm. 3. Noņemtās piesaistes tiek dzēstas no tabulas server_components, jaunās – pievienotas. 4. Pēc saglabāšanas lapa tiek pārlādēta, un tiek parādīts paziņojums par veiksmīgu saglabāšanu. 5. Jauns komponents tiek izveidots, un administrators atgriežas tajā pašā lapā.	
Izvaddati	
1. Paziņojums “Server components saved successfully.” 2. Atjaunināts servera komponentu saraksts. 3. Konfiguratorā tiek rādīti tikai piesaistītie komponenti.	
Kļūdas paziņojumi	
1. “Price must be a non-negative number”. 2. “Invalid CSRF token”.	
Izsaucamās prasības	
1. Lietotājs ir autorizējies kā administrators. 2. Serveris eksistē.	

Komponentu pārvaldība

3.16. tabula

Komponentu pārvaldības prasību apraksts

Identifikators	PS 3.1.16
Ievads	
Funkcija paredzēta komponentu bibliotēkas pārvaldībai. Administrators var pievienot, rediģēt un dzēst komponentus.	

Ievaddati
1. Nosaukums – obligāts lauks. 2. Tips – izvēle no saraksta: CPU, GPU, RAM, SSD, HDD. 3. Specifikācija: CPU – kodolu skaits un frekvence GHz; GPU – video atmiņa GB; RAM, SSD, HDD – apjoms GB. Izveidojot komponentu, sistēma pati sastāda specifikācijas tekstu (piem. “12 cores, 3.6 GHz”), rediģējot to ievada kā tekstu. 4. Cena – obligāts lauks, skaitlis, ne mazāks par 0.
Apstrāde
1. Sistēma pārbauda administratora lomu un CSRF marķieri. 2. Sistēma pārbauda ievadītos laukus atbilstoši izvēlētajam tipam. 3. Sistēma sastāda specifikācijas tekstu un saglabā komponentu vai atjauno esošo. 4. Dzēšana notiek pēc apstiprinājuma dialoga. Kopā ar komponentu tiek dzēstas tā piesaistes serveriem.
Izvaddati
1. Komponentu saraksts ar tipu, nosaukumu, specifikāciju, cenu un darbību pogām.
Kļūdas paziņojumi
1. “Component name is required”. 2. “Component type is invalid”. 3. “Price must be a non-negative number”. 4. “CPU cores and frequency are required”. 5. “GPU VRAM is required”. 6. “RAM capacity is required”. 7. “Storage capacity is required”. 8. “Component specifications are required”.
Izsaucamās prasības
1. Lietotājs ir autorizējies kā administrators.

Pasūtījumu pārvaldība

3.17. tabula

Pasūtījumu pārvaldības prasību apraksts

Identifikators	PS 3.1.17
Ievads	
Funkcija paredzēta visu sistēmas pasūtījumu apskatei, filtrēšanai un statusa maiņai.	
Ievaddati	
1. Statusa filtrs – izvēle no saraksta. 2. Kārtošana – jaunākie, vecākie, cena augošā vai dilstošā secībā. 3. Darbība – “Mark as Paid” vai “Mark as Completed”.	
Apstrāde	
1. Sistēma pārbauda administratora lomu. 2. Sistēma iegūst visus pasūtījumus kopā ar pasūtītāja konta lietotājvārdu. 3. Filtrēšana notiek klienta pusē ar KnockoutJS, kārtošana – ar JavaScript, bez lapas pārlādes. 4. Mainot statusu, sistēma pārbauda CSRF marķieri un atļauj tikai pārejas “processing” → “paid” un “paid” → “completed”. 5. Statuss datubāzē tiek mainīts tikai tad, ja pašreizējais statuss sakrīt ar gaidīto. Tas novērš vienas darbības dubultu izpildi. 6. No saraksta var atvērt pasūtījuma detaļas un lejupielādēt rēķinu.	
Izvaddati	
1. Pasūtījumu tabula ar klientu, e-pastu, datumu, summu, statusu un darbībām. 2. Atjaunināts pasūtījuma statuss.	

Kļūdas paziņojumi
1. “Invalid order action”. 2. “Order not found”. 3. “This action is not available for the current order status”. 4. “Order status changed before this action could be applied”. 5. “Invalid CSRF token”.
Izsaucamās prasības
1. Lietotājs ir autorizējies kā administrators.

Rēķina rekvizītu pārvaldība

3.18. tabula

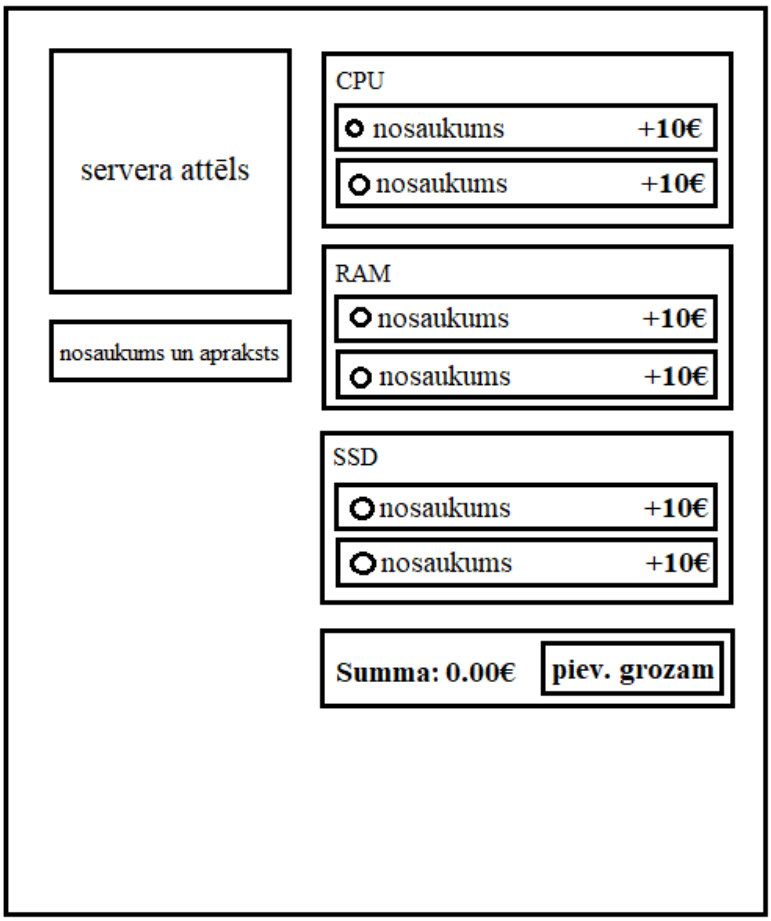
Rēķina rekvizītu pārvaldības prasību apraksts

Identifikators	PS 3.1.18
Ievads	
Funkcija ļauj administratoram norādīt pārdevēja rekvizītus, kas tiek drukāti visos rēķinos.	
Ievaddati	
1. Uzņēmuma nosaukums (līdz 255 rakstzīmēm), reģistrācijas numurs (100), PVN numurs (100), adrese (255), pasta indekss (30), pilsēta (100), valsts (100), tālrunis (50), e-pasts (255, korektā formātā) un bankas konts vai IBAN (100). Visi lauki ir neobligāti.	
Apstrāde	
1. Sistēma pārbauda administratora lomu un CSRF marķieri. 2. Sistēma pārbauda lauku garumu un e-pasta formātu. 3. Tukši lauki tiek saglabāti kā NULL un rēķinā netiek rādīti. 4. Rekvizīti tiek saglabāti administratora ierakstā un izmantoti visos turpmāk ģenerētajos rēķinos. Ja uzņēmuma nosaukums nav norādīts, rēķinā tiek rādīts “Server Catalog”.	
Izvaddati	
1. Paziņojums “Invoice details saved.” 2. Atjaunināti pārdevēja rekvizīti rēķinos.	
Kļūdas paziņojumi	
1. “Only administrators can update invoice details”. 2. “… must not exceed N characters”. 3. “Invoice email must be valid”.	
Izsaucamās prasības	
1. Lietotājs ir autorizējies kā administrators.	

3.2.NEFUNKCIONĀLĀS PRASĪBAS

1. Sistēmas saskarnes valodai jābūt angļu valodai.
2. Serveru kataloga, servera konfiguratora un iepirkumu groza lapām jāielādējas saprātīgā laikā atkarībā no lietotāja interneta savienojuma ātruma.
3. Sistēmai jānodrošina aizsardzība pret izplatītākajiem tīmekļa ievainojamību veidiem: SQL injekcijām (parametrizēti PDO vaicājumi), XSS uzbrukumiem (izvades kodēšana ar htmlspecialchars) un CSRF uzbrukumiem (marķieris katrā datus mainošā pieprasījumā).
4. Lietotāju paroles jāglabā datubāzē jauktā veidā (password_hash), nevis atklātā tekstā.
5. Administratīvajām funkcijām jābūt pieejamām tikai lietotājiem ar administratora lomu.

6. Pasūtījuma cenai jātiek aprēķinātai serverī no datubāzes datiem, neuzticoties pārlūkprogrammas nosūtītajām vērtībām.
7. Pasūtījuma vēsturei jāsaglabājas nemainīgai arī tad, ja administrators vēlāk maina vai dzēš serveri vai komponentu.
8. Pasūtījumu drīkst noformēt tikai autorizēts lietotājs ar apstiprinātu e-pasta adresi, un pasūtījumu, tā apmaksu un rēķinu var piekļūt tikai tā īpašnieks vai administrators.
9. Augšupielādētajiem attēliem jābūt pārbaudītiem pēc satura un izmēra, bet SVG failiem jābūt attīrītiem no aktīva satura.
10. Lietotāja saskarnei jābūt intuitīvai un viegli lietojamai bez papildu apmācības, navigācijai starp sadaļām jābūt skaidrai un loģiskai.
11. Sistēmai jāsniedz lietotājam saprotami paziņojumi par veiksmīgi izpildītām darbībām un kļūdām.
12. Servera konfigurācijas cenas aprēķinam un kataloga filtrēšanai jānotiek bez pilnīgas lapas pārlādes.
13. Sistēmas kodam jābūt strukturētam atbilstoši MVC arhitektūrai un sadalītam atsevišķos moduļos.
14. Sistēmai jādarbojas mūsdienīgās tīmekļa pārlūkprogrammās, piemēram, Google Chrome, Microsoft Edge un Mozilla Firefox.
15. Sistēmai jābūt palaižamai uz cita datora ar vienu komandu, izmantojot Docker Compose, bez atsevišķas PHP, MySQL vai Node.js instalēšanas.
16. Lietotāja saskarnē jābūt pieejamam gaišajam un tumšajam režīmam.
17. Lietotāja saskarnei jāpielāgojas dažādiem ekrāna izmēriem – no mobilajiem tālruņiem līdz platiem monitoriem.
18. Servera konfigurācijas formai jānodrošina ērta komponentu izvēle un kopējās cenas pārskatīšana (skat. 3.1. att.).



3.1. att. Servera konfigurācijas formas skice

4. UZDEVUMA RISINĀŠANAS LĪDZEKĻU APRAKSTS UN IZVĒLES PAMATOJUMS

Tīmekļa vietne ir izstrādāta kā serveru konfigurēšanas un pārdošanas sistēma, kas sastāv no lietotāja saskarnes, servera puses loģikas un datubāzes. Tehnoloģijas tika izvēlētas, ņemot vērā to funkcionalitāti, pieejamību, dokumentācijas kvalitāti un piemērotību tīmekļa lietojumprogrammu izstrādei, kā arī mācību laikā iegūtās zināšanas.

Priekšgalsistēmas izstrādē tika izmantotas šādas tehnoloģijas:

- **HTML5** – tīmekļa lapu struktūras, formu un tabulu izveidei.
- **CSS un Tailwind CSS (versija 4.3)** – lietotāja saskarnes noformējumam. Tailwind nodrošina gatavu utilītklašu kopumu, kas ļauj ātri veidot vienotu dizainu, nerakstot lielu daudzumu pielāgota CSS koda. Gala CSS fails tiek ģenerēts ar Tailwind CLI, un tajā nonāk tikai projektā izmantotās klases.
- **JavaScript** – dinamisku funkciju realizācijai klienta pusē: datu pieprasījumiem ar fetch, iepirkumu groza sānu panelim, paziņojumiem, apstiprinājuma dialogiem, gaišā un tumšā režīma pārslēgšanai.
- **KnockoutJS (versija 3.5.3)** – JavaScript bibliotēka datu sasaistei (MVVM). Tā izmantota servera konfigurācijas cenas pārrēķināšanai, kataloga filtrēšanai un pasūtījumu filtrēšanai administratora lapā bez lapas pārlādes. Bibliotēka ir neliela un nav atkarīga no citām bibliotēkām.

Aizmugursistēmas izstrādē tika izmantotas šādas tehnoloģijas:

- **PHP (versija 8.2)** – servera puses programmēšanas valoda lietotāju autentifikācijai, datu apstrādei, pasūtījumu pārvaldībai un saziņai ar datubāzi. PHP ir plaši atbalstīta tīmekļa hostinga pakalpojumos un nodrošina iebūvētas funkcijas drošai paroļu jaukšanai.
- **MySQL (versija 8.4)** – relāciju datubāzu pārvaldības sistēma lietotāju, serveru, komponentu, iepirkumu grozu un pasūtījumu glabāšanai. Lokāli izstrādes laikā tika izmantota arī XAMPP komplektā iekļautā saderīgā MariaDB sistēma.
- **PDO (PHP Data Objects)** – datubāzes piekļuves slānis, kas ļauj izmantot parametrizētus SQL vaicājumus un transakcijas.
- **Composer** – PHP atkarību pārvaldnieks, ar kuru tiek instalētas trešo pušu bibliotēkas.

- **Dompdf (versija 3.1)** – bibliotēka, kas pārveido HTML veidni PDF failā. Izmantota pasūtījuma rēķinu ģenerēšanai.
- **PHPMailer (versija 6.10)** – bibliotēka e-pasta sūtīšanai caur SMTP. Izmantota e-pasta adreses apstiprināšanas vēstulēm un rēķinu nosūtīšanai ar PDF pielikumu.
- **phpdotenv (versija 5.6)** – bibliotēka konfigurācijas nolasīšanai no .env faila, lai paroles un piekļuves dati netiktu glabāti pirmkodā.
- **PHPUnit (versija 11.5)** – automatizēto testu ietvars.

Sistēmas izstrādes procesā tika izmantoti šādi rīki:

- **Docker un Docker Compose** – konteinerizācijas rīki, ar kuriem viena komanda palaiž visu izstrādes vidi: Apache ar PHP, MySQL datubāzi, phpMyAdmin, Tailwind CSS ģeneratoru un pēc izvēles Mailpit. Tas ļauj palaist projektu uz jebkura datora bez atsevišķas programmatūras instalēšanas.
- **XAMPP** – lokālās izstrādes vide ar Apache tīmekļa serveri, PHP un MariaDB, kas tika izmantota projekta sākumposmā.
- **phpMyAdmin** – rīks datubāzes administrēšanai un datu pārvaldībai.
- **Mailpit** – lokāls SMTP serveris, kas pārtver izejošās vēstules un ļauj tās apskatīt pārlūkprogrammā, nesūtot tās īstiem adresātiem.
- **Visual Studio Code** – pirmkoda redaktors projekta izstrādei un atkļūdošanai.
- **Git un GitHub** – versiju kontroles sistēma un pirmkoda glabātuve.
- **PlantUML** – rīks diagrammu (ER, lietojumgadījumu, aktivitāšu, stāvokļu) zīmēšanai no teksta.

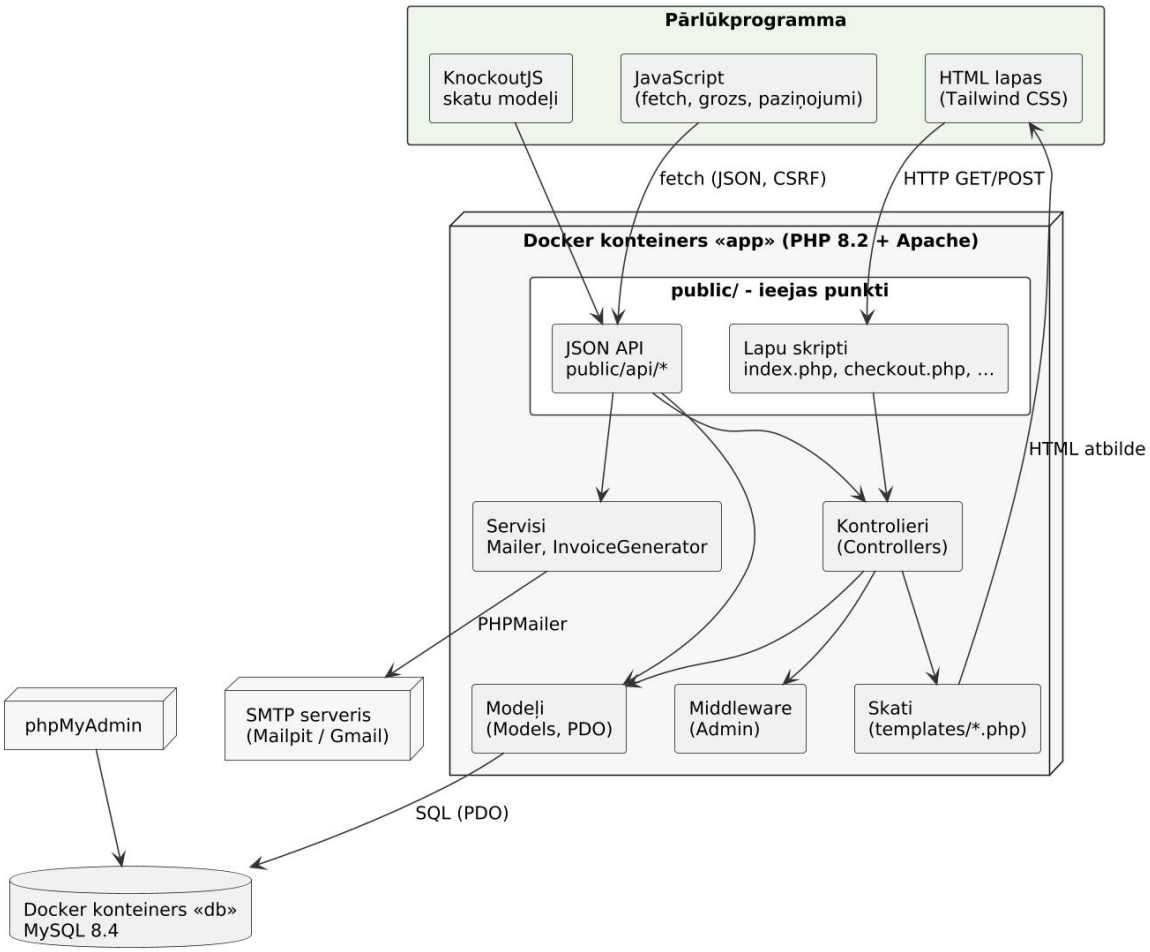
Izvēlēto tehnoloģiju kopums nodrošina tīmekļa vietnes funkcionalitāti, drošu datu glabāšanu, lietotāja saskarnes dinamisku darbību un ērtu sistēmas uzturēšanu un pārnešanu uz citu datoru.

5. SISTĒMAS MODELĒŠANA UN PROJEKTĒŠANA

Sistēmas arhitektūra

Sistēma ir veidota pēc MVC (Model–View–Controller) arhitektūras principiem. Datu apstrāde, biznesa loģika un lietotāja saskarne ir nodalīti atsevišķos moduļos (skat. 5.1. att.):

- **Modeļi** (`src/models`) – klases User, Server, Component, ServerComponent, Cart un Order, kas izpilda SQL vaicājumus caur PDO. Visi vaicājumi ir parametrizēti.
- **Kontrolieri** (`src/controllers`) – saņem pieprasījumu, pārbauda datus un tiesības, izsauc modeļus un izvēlas skatu. Administratora kontrolieri atrodas apakšmapē `admin`.
- **Skati** (`templates`) – PHP veidnes, kas ģenerē HTML. Kopīgās daļas (galvene, kājene, groza panelis, sīkdatņu paziņojums) ir atsevišķos failos.
- **Ieejas punkti** (`public`) – katrai lapai ir savs PHP fails, kas ielādē konfigurāciju un izsauc vajadzīgo kontroliera metodi. Mapē `public/api` atrodas JSON saskarnes punkti, kurus izsauc JavaScript.
- **Servisi** (`src/services`) – Mailer (e-pasta sūtīšana ar PHPMailer) un InvoiceGenerator (PDF rēķina ģenerēšana ar Dompdf). Fails CompatibilityChecker.php ir izveidots kā sagatave nākotnei – tajā paredzēts pārbaudīt, vai izvēlētie komponenti ir savietojami ar pārējo servera aparatūru, bet pašlaik tas netiek izmantots.
- **Palīgklases un starpprogrammatūra** (`src/helpers`, `src/middleware`) – JSON atbilžu formēšana, tālruņa numuru normalizēšana, sīkdatņu piekrišana un administratora tiesību pārbaude.
- **Kodols** (`src/Core`) – ceļu un URL konstantes, .env faila ielāde un CSRF marķieru funkcijas.

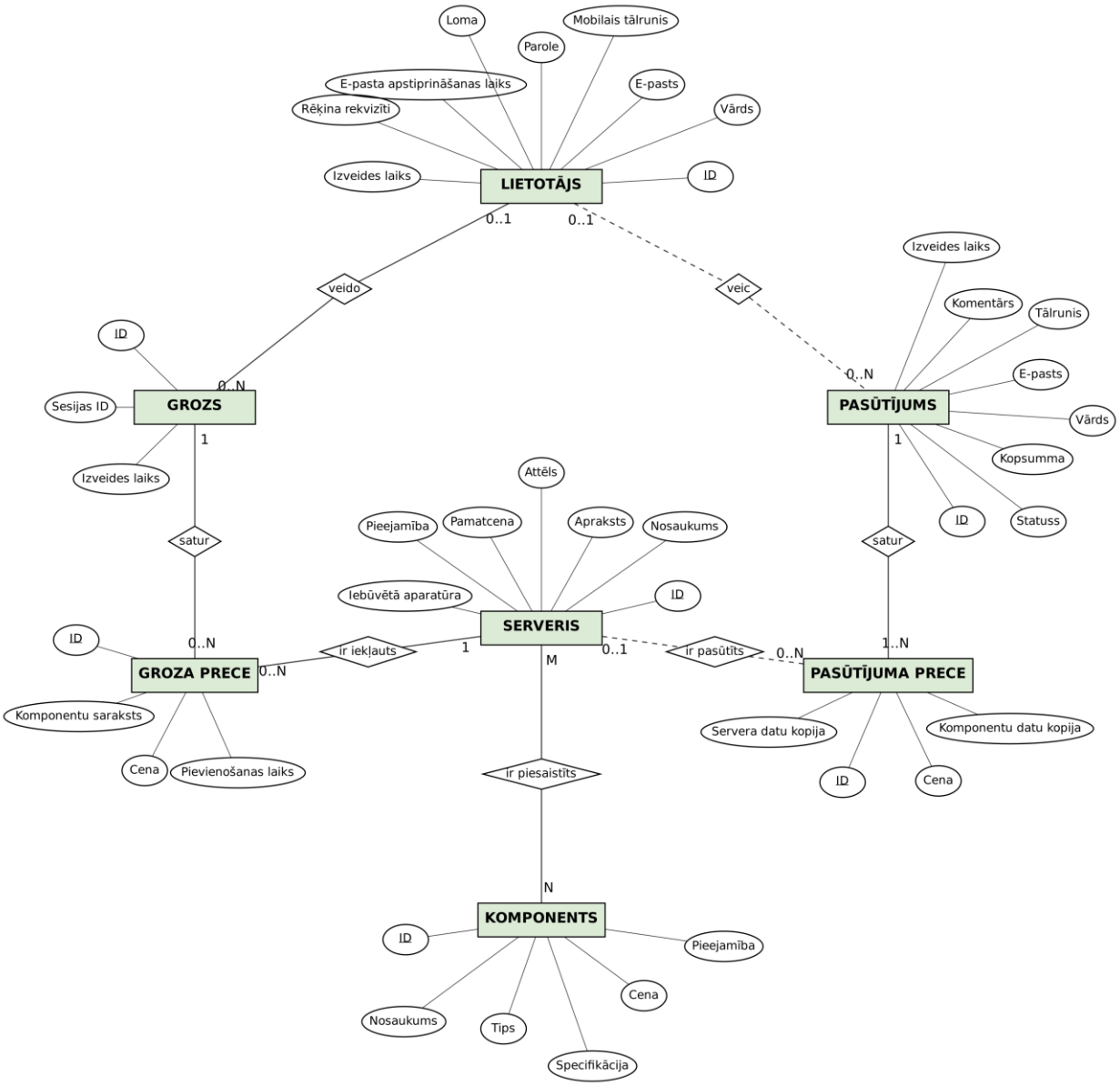


5.1. att. Sistēmas arhitektūras diagramma

Izstrādes vide sastāv no pieciem Docker konteineriem: “app” (Apache ar PHP 8.2), “db” (MySQL 8.4), “phpmyadmin”, “tailwind” (Node.js ar Tailwind CLI, kas ģenerē CSS failu) un pēc izvēles “mailpit” (lokāls SMTP serveris). Datubāze pirmajā palaišanas reizē tiek aizpildīta no faila `database/server_eshop.sql`.

Datubāzes ER diagramma

Sistēmas datubāze sastāv no astoņām tabulām. To savstarpējās saites attēlotas ER diagrammā (skat. 5.2. att.). Nepārtrauktas līnijas apzīmē saites ar ārējām atslēgām, raustītas – loģiskas saites bez ārējās atslēgas.



5.2. att. Datubāzes ER diagramma

Tabulas bez izejošajām atslēgām (FK):

- **users** – lietotāji un administratori;
- **servers** – serveru katalogs;
- **components** – komponentu bibliotēka;
- **orders** – pasūtījumi. Lauks user_id norāda uz lietotāju, kurš veica pasūtījumu, taču tam apzināti nav ārējās atslēgas, lai pasūtījumu vēsture saglabātos arī pēc lietotāja konta dzēšanas.

Savstarpējās attiecības:

- starp “servers” un “components” ir attiecība “daudzi pret daudziem”, kas realizēta ar starptabulu “server_components”: vienam serverim var būt piesaistīti daudzi komponenti, un viens komponents var būt pieejams daudziem serveriem;
- starp “users” un “carts” ir attiecība “viens pret daudziem”: lietotājam var būt vairāki grozi (dažādās sesijās), bet viesa grozam lietotāja nav;
- starp “carts” un “cart_items” ir attiecība “viens pret daudziem”: grozā var būt daudzas preces;
- starp “servers” un “cart_items” ir attiecība “viens pret daudziem”: viens serveris var būt daudzos grozos;
- starp “orders” un “order_items” ir attiecība “viens pret daudziem”: pasūtījumā ir viena vai vairākas preces;
- starp “users” un “orders” ir loģiska attiecība “viens pret daudziem” bez ārējās atslēgas;
- starp “servers” un “order_items” ir loģiska attiecība bez ārējās atslēgas – tā tika noņemta, lai, dzēšot serveri, netiktu dzēsta pasūtījumu vēsture. Servera un komponentu dati pasūtījumā tiek saglabāti kā kopijas (snapshot).

Visām ārējām atslēgām ir iestatīta darbība ON DELETE CASCADE: dzēšot serveri, tiek dzēstas tā komponentu piesaistes un groza preces, dzēšot grozu – tā preces, dzēšot pasūtījumu – tā preces.

Datu struktūru apraksts

Turpmākajās tabulās aprakstīta katras datubāzes tabulas struktūra.

Tabula “users” glabā informāciju par reģistrētajiem lietotājiem un administratoriem. Administratora ierakstā tiek glabāti arī rēķinos izmantojamie rekvizīti.

5.1. tabula

Tabulas “users” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Lietotāja identifikators, primārā atslēga, automātiski palielināms
2.	username	VARCHAR	255	Lietotāja vārds
3.	email	VARCHAR	255	E-pasta adrese, unikāla
4.	mobile_phone	VARCHAR	30	Mobilā tālruņa numurs, var būt tukšs
5.	password	VARCHAR	255	Paroles jaucējvērtība (password_hash)
6.	role	ENUM	–	Loma: “admin” vai “user”, noklusējumā “user”
7.	email_verified_at	TIMESTAMP	–	E-pasta apstiprināšanas laiks; NULL, ja nav apstiprināts

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
8.	email_verification_token	VARCHAR	64	Vienreizējs apstiprināšanas marķieris
9.	invoice_company_name	VARCHAR	255	Pārdevēja nosaukums rēķinā
10.	invoice_registration_number	VARCHAR	100	Reģistrācijas numurs
11.	invoice_vat_number	VARCHAR	100	PVN maksātāja numurs
12.	invoice_address	VARCHAR	255	Juridiskā adrese
13.	invoice_postal_code	VARCHAR	30	Pasta indekss
14.	invoice_city	VARCHAR	100	Pilsēta
15.	invoice_country	VARCHAR	100	Valsts
16.	invoice_phone	VARCHAR	50	Uzņēmuma tālrunis
17.	invoice_email	VARCHAR	255	Uzņēmuma e-pasts
18.	invoice_bank_account	VARCHAR	100	Bankas konts vai IBAN
19.	created_at	TIMESTAMP	–	Konta izveides laiks

Tabula "servers" glabā informāciju par katalogā piedāvātajiem serveriem.

5.2. tabula

Tabulas "servers" struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Servera identifikators, primārā atslēga
2.	name	VARCHAR	255	Servera nosaukums
3.	description	TEXT	–	Servera apraksts
4.	image	VARCHAR	255	Attēla faila nosaukums mapē public/assets/images
5.	base_price	DECIMAL	10,2	Pamatcena eiro bez papildu komponentiem
6.	available	TINYINT	1	Pieejamība: 1 – rādīt katalogā, 0 – paslēpts
7.	default_ram	DECIMAL	8,3	Iebūvētā operatīvā atmiņa GB (var būt daļskaitlis)
8.	default_storage	INT	11	Iebūvētās krātuves apjoms GB
9.	default_cpu_cores	INT	11	Procesora kodolu skaits
10.	default_gpu_vram	INT	11	Videokartes atmiņa GB
11.	created_at	TIMESTAMP	–	Pievienošanas laiks

Tabula "components" glabā komponentu bibliotēku, no kuras komponenti tiek piesaistīti serveriem.

5.3. tabula

Tabulas "components" struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Komponenta identifikators, primārā atslēga
2.	name	VARCHAR	255	Komponenta nosaukums, piem. "AMD Ryzen 9 5900X"
3.	type	ENUM	–	Tips: cpu, gpu, ram, ssd, hdd (vērtība "storage" saglabāta saderībai)
4.	value	VARCHAR	50	Specifikācija, piem. "12 cores, 3.7 GHz"
5.	price	DECIMAL	10,2	Piemaksa par komponentu eiro
6.	server_id	INT	11	Vēsturisks lauks, netiek izmantots (sasaiste caur server_components)
7.	available	TINYINT	1	Pieejamība konfiguratorā

Tabula “server_components” ir starptabula, kas realizē attiecību “daudzi pret daudziem” starp serveriem un komponentiem.

5.4. tabula

Tabulas “server_components” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Ieraksta identifikators, primārā atslēga
2.	server_id	INT	11	Ārējā atslēga uz servers.id
3.	component_id	INT	11	Ārējā atslēga uz components.id

Tabula “carts” glabā iepirkumu grozus. Grozs tiek piesaistīts sesijai, bet pēc autorizācijas – arī lietotājam.

5.5. tabula

Tabulas “carts” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Groza identifikators, primārā atslēga
2.	user_id	INT	11	Ārējā atslēga uz users.id; NULL viesim
3.	session_id	VARCHAR	255	PHP sesijas identifikators
4.	created_at	TIMESTAMP	–	Izveides laiks

Tabula “cart_items” glabā grozā ieliktās konfigurācijas.

5.6. tabula

Tabulas “cart_items” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Preces identifikators, primārā atslēga
2.	cart_id	INT	11	Ārējā atslēga uz carts.id
3.	server_id	INT	11	Ārējā atslēga uz servers.id
4.	total_price	DECIMAL	10,2	Serverī aprēķinātā konfigurācijas cena
5.	created_at	TIMESTAMP	–	Pievienošanas laiks
6.	component_ids	LONGTEXT	–	Izvēlēto komponentu identifikatori JSON masīvā

Tabula “orders” glabā pasūtījumus un pircēja kontaktinformāciju.

5.7. tabula

Tabulas “orders” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Pasūtījuma identifikators, primārā atslēga
2.	user_id	INT	11	Pasūtījumu veikušā lietotāja identifikators
3.	session_id	VARCHAR	255	Sesijas identifikators pasūtījuma izveides brīdī
4.	status	ENUM	–	pending, processing, paid, completed, cancelled
5.	total_price	DECIMAL	10,2	Pasūtījuma kopsumma eiro
6.	name	VARCHAR	100	Pircēja vārds un uzvārds
7.	email	VARCHAR	100	Pircēja e-pasta adrese
8.	phone	VARCHAR	20	Normalizēts tālruņa numurs, var būt tukšs
9.	comment	TEXT	–	Pircēja komentārs
10.	created_at	TIMESTAMP	–	Pasūtījuma izveides laiks

Tabula “order_items” glabā pasūtījuma preces kopā ar servera un komponentu datu kopijām pasūtījuma brīdī.

5.8. tabula

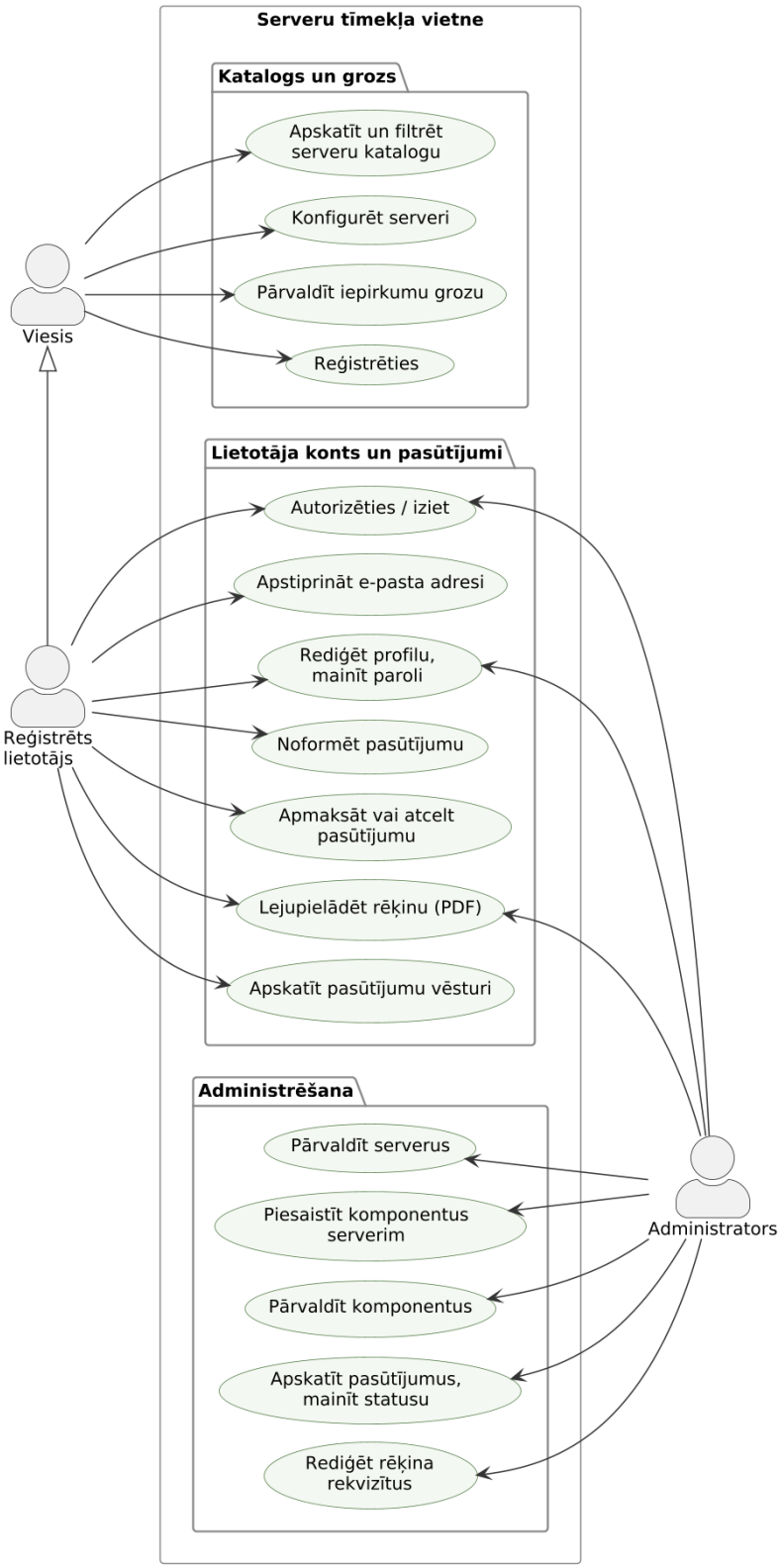
Tabulas “order_items” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	id	INT	11	Preces identifikators, primārā atslēga
2.	order_id	INT	11	Ārējā atslēga uz orders.id
3.	server_id	INT	11	Servera identifikators pasūtījuma brīdī (bez ārējās atslēgas)
4.	component_ids	LONGTEXT	–	Komponentu identifikatori JSON masīvā
5.	total_price	DECIMAL	10,2	Konfigurācijas cena pasūtījuma brīdī
6.	server_name_snapshot	VARCHAR	255	Servera nosaukuma kopija
7.	server_description_snapshot	TEXT	–	Servera apraksta kopija
8.	server_image_snapshot	VARCHAR	255	Servera attēla kopija
9.	server_base_price_snapshot	DECIMAL	10,2	Servera pamatcenas kopija
10.	components_snapshot	LONGTEXT	–	Komponentu datu kopija JSON formātā

6. FUNKCIONĀLAIS UN DINAMISKAIS MODELIS

Lietojumgadījumu diagramma

Lietojumgadījumu diagrammā (skat. 6.1. att.) attēloti trīs sistēmas lietotāju veidi un viņiem pieejamās funkcijas. Viesis var apskatīt katalogu, konfigurēt serveri, pārvaldīt grozu un reģistrēties. Reģistrēts lietotājs manto visas viesa iespējas un papildus var pieteikties sistēmā, apstiprināt e-pastu, noformēt, apmaksāt vai atcelt pasūtījumu, lejupielādēt rēķinu, apskatīt pasūtījumu vēsturi un rediģēt profilu. Administrators pārvalda serverus, komponentus, pasūtījumus un rēķinu rekvizītus, var lejupielādēt jebkura pasūtījuma rēķinu, taču pats nevar veikt pasūtījumus.

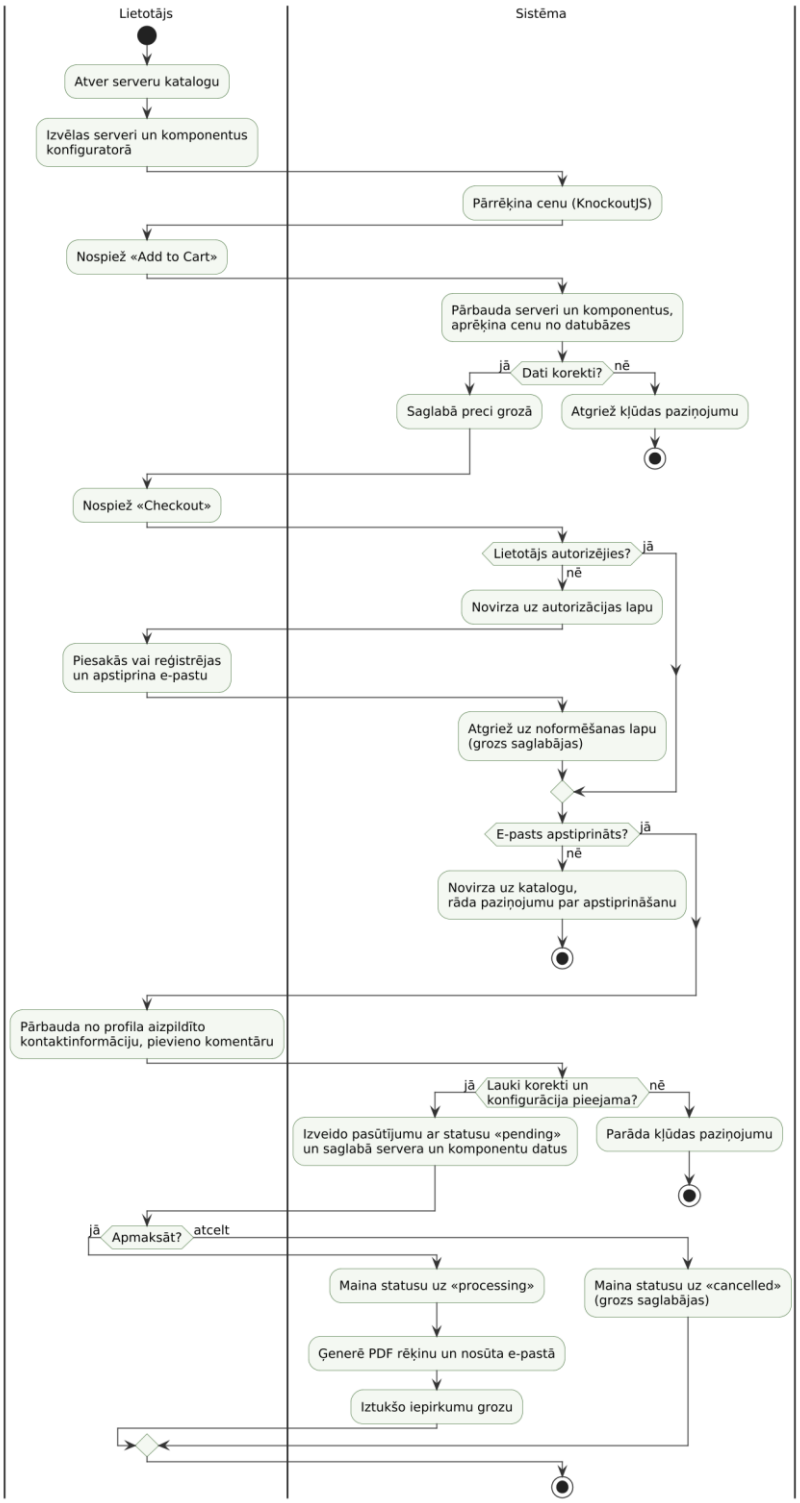


6.1. att. Sistēmas lietojumgadījumu diagramma

Pasūtījuma veikšanas aktivitāšu diagramma

Aktivitāšu diagrammā (skat. 6.2. att.) attēlots pasūtījuma process no servera izvēles līdz apmaksai vai atcelšanai. Diagramma parāda, kuras darbības veic lietotājs un kuras – sistēma, kā arī

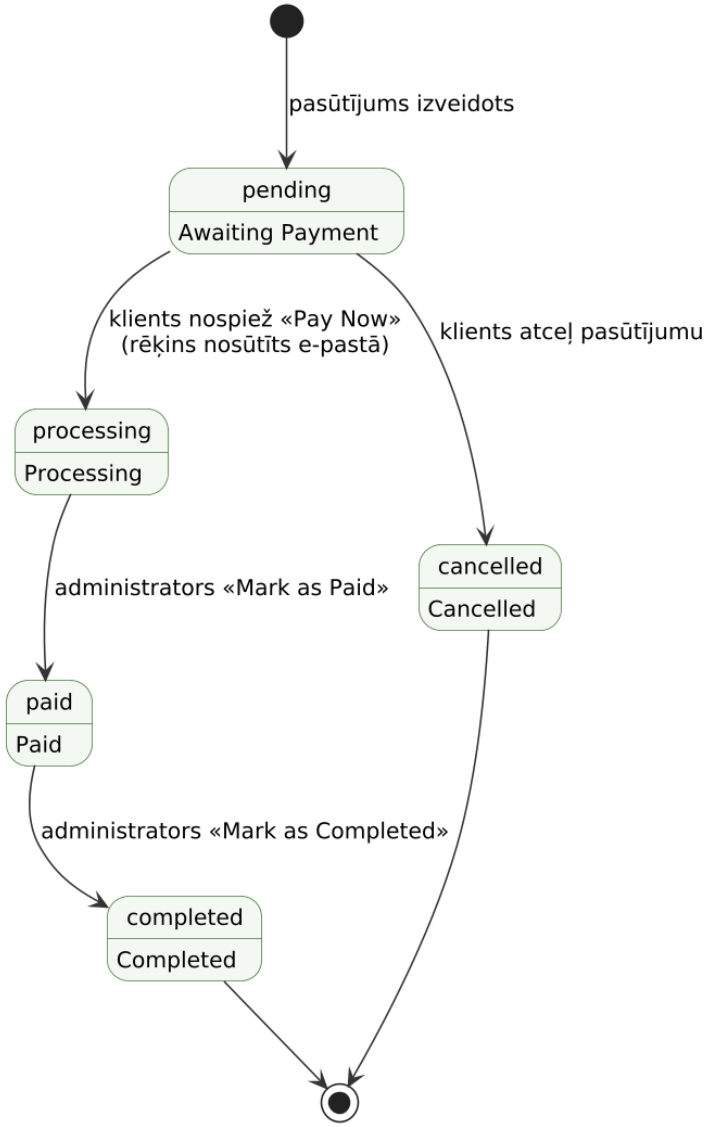
pārbaudes, kas tiek izpildītas, pirms dati tiek saglabāti. Grozu var aizpildīt arī viesis, bet, nospiežot “Checkout”, sistēma pieprasa pieteikties un pārbauda, vai e-pasta adrese ir apstiprināta.



6.2. att. Pasūtījuma veikšanas aktivitāšu diagramma

Pasūtījuma stāvokļu diagramma

Pasūtījuma statusi un pārejas starp tiem attēloti stāvokļu diagrammā (skat. 6.3. att.). Jauns pasūtījums saņem statusu “pending” (Awaiting Payment). Klients to var apmaksāt – tad statuss kļūst “processing” un klients saņem rēķinu – vai atcelt. Administrators pēc maksājuma saņemšanas atzīmē pasūtījumu kā apmaksātu (“paid”) un pēc izpildes – kā pabeigtu (“completed”). Citas pārejas sistēma neatļauj.



6.3. att. Pasūtījuma stāvokļu diagramma

7. LIETOTĀJA CEĻVEDIS

Sistēmas prasības aparatūrai un programmatūrai

Lai palaistu sistēmu lokāli ar Docker, datoram jāatbilst šādām prasībām:

- operētājsistēma: Windows 10 vai 11 (64 bitu) vai macOS;
- operatīvā atmiņa: vismaz 4 GB, ieteicams 8 GB;
- brīva vieta diskā: vismaz 3 GB (Docker attēli un datubāze);
- instalēts un palaists Docker Desktop;
- pirmajai palaišanai – interneta savienojums Docker attēlu lejupielādei.

Tīmekļa vietnes lietošanai pietiek ar mūsdienīgu pārlūkprogrammu (Google Chrome, Microsoft Edge, Mozilla Firefox vai Safari) ar ieslēgtu JavaScript. Saskarne pielāgojas dažādiem ekrāna izmēriem – no mobilajiem tālruņiem līdz platiem monitoriem.

Sistēmas instalācija un palaišana

1. Lejupielādēt projektu no GitHub ar komandu `git clone https://github.com/SergeyGrinn/serverShop-course-dev` vai pārkopēt projekta mapi uz datoru (bez .env faila).
2. Palaist Docker Desktop un pagaidīt, līdz tas ir gatavs darbam.
3. Projekta saknes mapē atvērt termināli (PowerShell) un izpildīt komandu `docker compose up -d --build`. Pirmajā reizē tiek lejupielādēti attēli, instalētas PHP un Node.js atkarības un no faila `database/server_eshop.sql` izveidota datubāze ar piemēra datiem.
4. Atvērt tīmekļa vietni pārlūkprogrammā adresē `http://127.0.0.1:8080`. Datubāzes pārvaldība ar phpMyAdmin pieejama adresē `http://127.0.0.1:8081`.
5. Lai pārbaudītu e-pasta vēstules lokāli, papildus palaist Mailpit ar komandu `docker compose --profile mail up -d mailpit` un atvērt iesūtni adresē `http://localhost:8026`.
6. Sistēmas apturēšanai izmantot komandu `docker compose down`. Datubāzes dati tiek saglabāti Docker sējumā un nākamajā palaišanas reizē netiek zaudēti.

Lai vēstules tiktu sūtītas īstiem adresātiem (piemēram, caur Gmail), lokālajā .env failā jānorāda mainīgie DOCKER_MAILER_HOST, DOCKER_MAILER_PORT, DOCKER_MAILER_USERNAME, DOCKER_MAILER_PASSWORD un

DOCKER_MAILER_ENCRYPTION, pēc tam jāpārstartē konteiners ar `docker compose up -d app`. Paraugs atrodas failā .env.example. Ja e-pasts netiek nosūtīts un konteinera žurnālā (`docker compose logs app`) redzams paziņojums "certificate verify failed", jāatslēdz antivīrusa e-pasta vai SSL/TLS skenēšana vai jāpievieno smtp.gmail.com izņēmumiem.

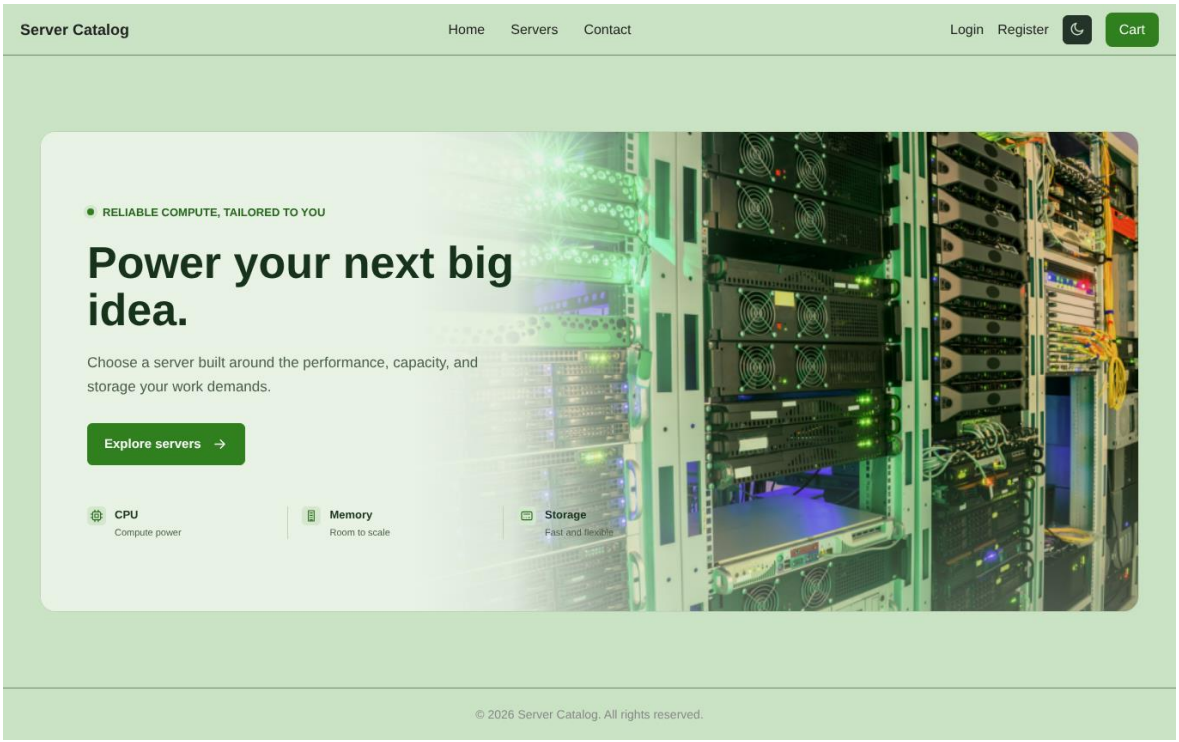
Sistēmu var palaist arī XAMPP vidē: projekta mapi ievieto mapē htdocs, ar phpMyAdmin importē failu `database/server_eshop.sql`, projekta mapē izpilda `composer install`, .env failā norāda datubāzes piekļuves datus (DB_HOST, DB_NAME, DB_USER, DB_PASS) un atver mapi public pārlūkprogrammā.

Piemēra datubāzē ir izveidoti divi lietotāji:

- administrators – e-pasts `admin@example.com`, parole `adminpass`;
- parasts lietotājs – e-pasts `user@example.com`, parole `userpass`.

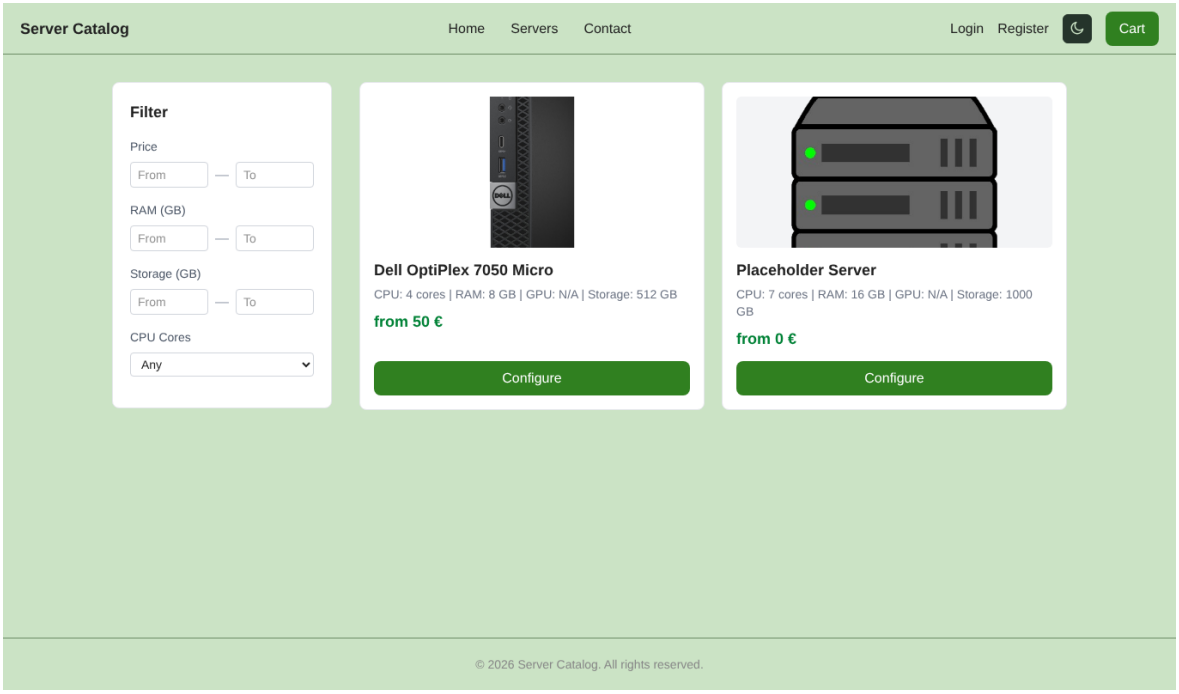
Programmas apraksts

Atverot tīmekļa vietni, lietotājs nonāk sākumlapā (skat. 7.1. att.). Lapas augšā atrodas galvene ar saitēm "Home", "Servers" un "Contact", autorizācijas saitēm, gaišā/tumšā režīma pārslēgu un pogu "Cart". Nosaukums "Server Catalog" galvenes kreisajā pusē arī ir saite uz sākumlapu. Pirmajā apmeklējumā lapas apakšā tiek parādīts paziņojums par sīkdatnēm ar pogām "Accept" un "Decline".



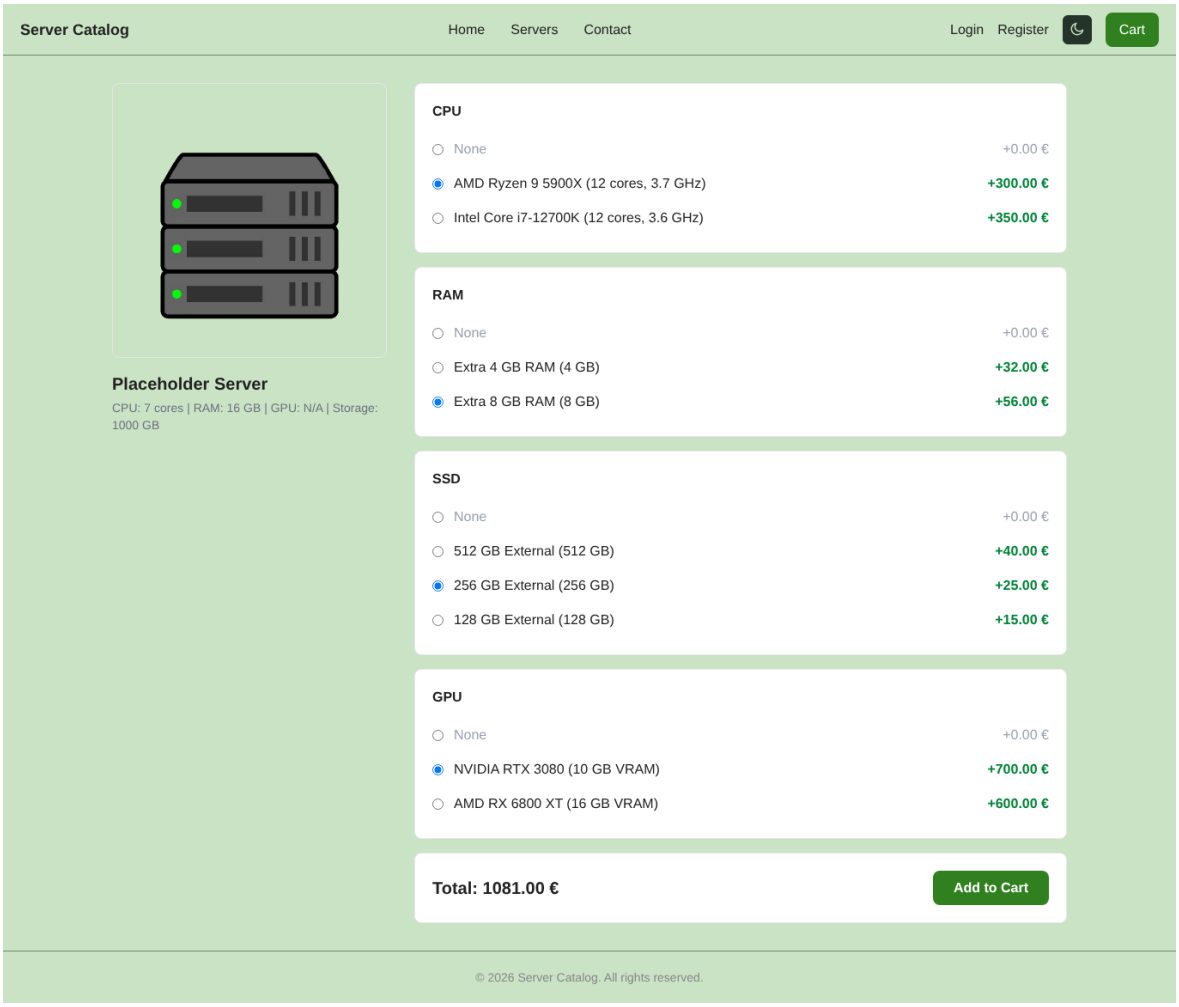
7.1. att. Sākumlapa

Saite “Servers” vai poga “Explore servers” atver serveru katalogu (skat. 7.2. att.). Kreisajā pusē atrodas filtri pēc cenas, RAM un krātuves apjoma un procesora kodolu skaita. Saraksts tiek atjaunots uzreiz pēc filtra maiņas. Ritinot lapu, filtru panelis paliek redzams ekrāna malā, tāpēc filtrus var mainīt, neritinot atpakaļ uz augšu. Ja minimālā vērtība ir lielāka par maksimālo, zem filtriem parādās kļūdas paziņojums.

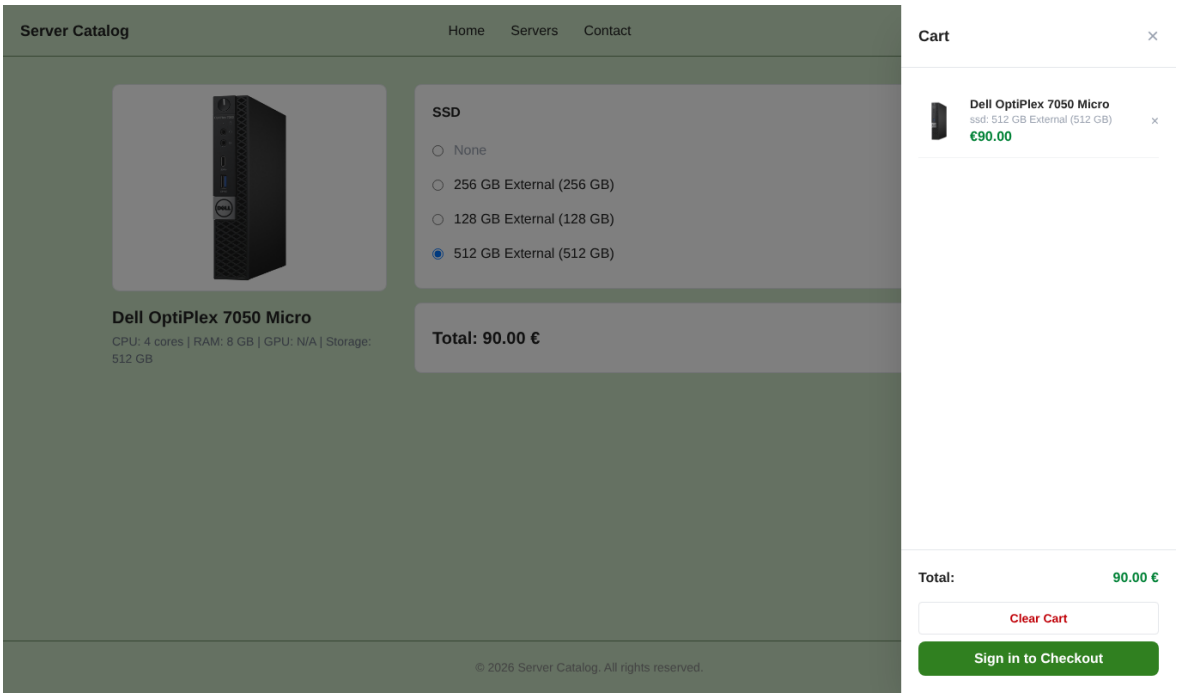


7.2. att. Serveru katalogs

Nospiežot pogu “Configure”, atveras servera konfigurators (skat. 7.3. att.). Konfiguratorā var izvēlēties komponentus no vairākām grupām (CPU, RAM, SSD, HDD, GPU) – katrā grupā vienu komponentu vai “None”. Kopējā cena lapas apakšā mainās uzreiz. Poga “Add to Cart” pievieno konfigurāciju grozam un atver groza sānu paneli (skat. 7.4. att.).

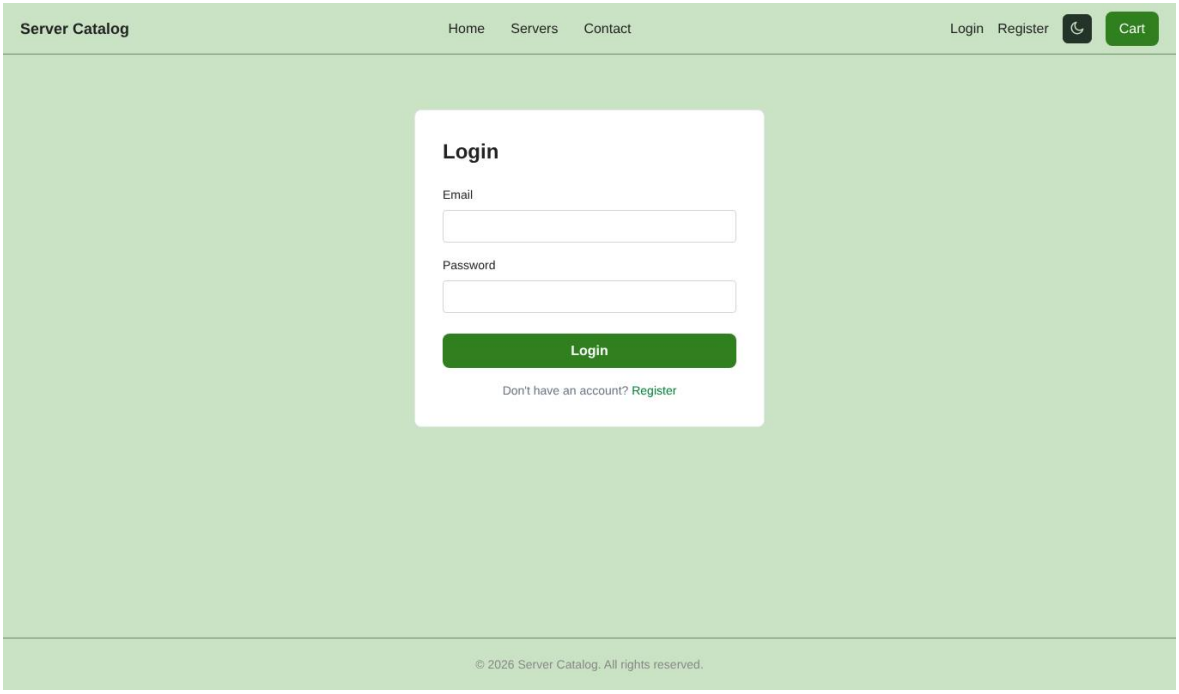


7.3. att. Servera konfigurators



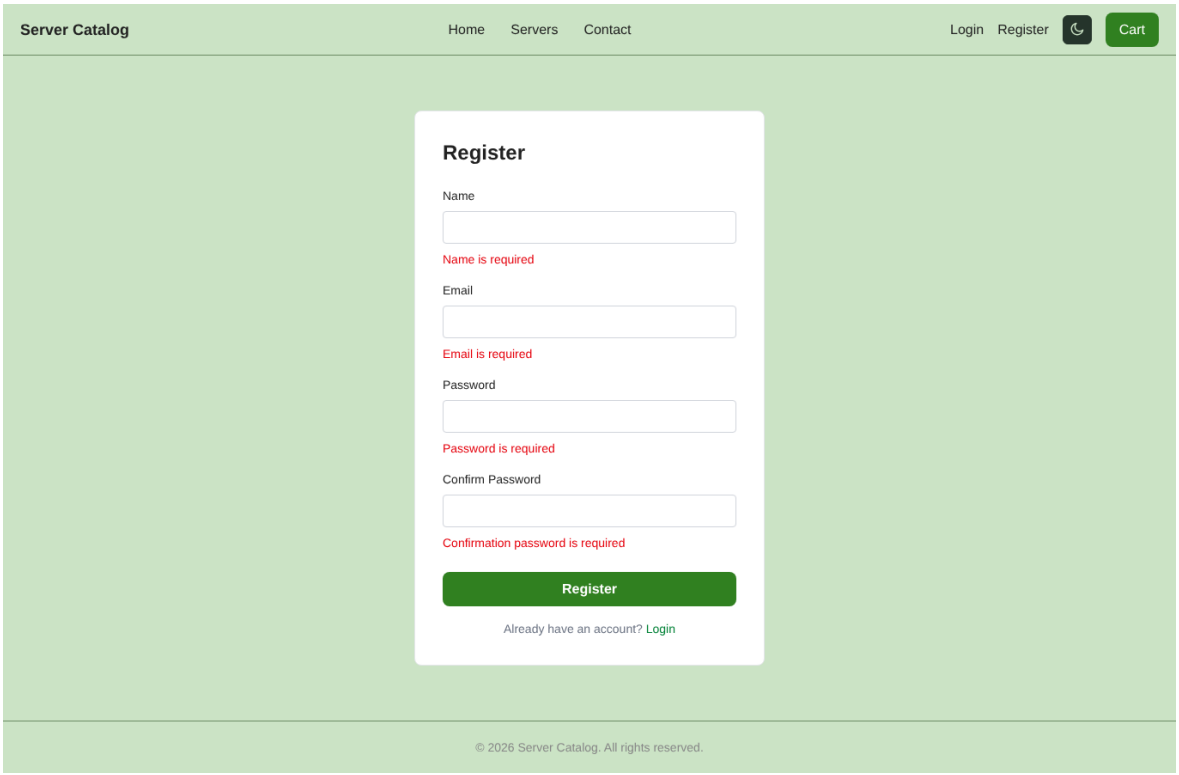
7.4. att. Iepirkumu groza sānu panelis neautorizētam lietotājam

Groza panelī redzamas pievienotās preces ar komponentiem un cenām. Preci var dzēst ar pogu "×", bet visu grozu – ar pogu "Clear Cart". Grozu var aizpildīt arī bez autorizācijas, taču pasūtījuma noformēšanai ir nepieciešams konts ar apstiprinātu e-pasta adresi. Tādēļ neautorizētam lietotājam groza apakšā ir poga "Sign in to Checkout", kas atver autorizācijas lapu (skat. 7.5. att.). Pēc pieteikšanās lietotājs tiek atgriezts uz noformēšanas lapu, un grozs saglabājas.

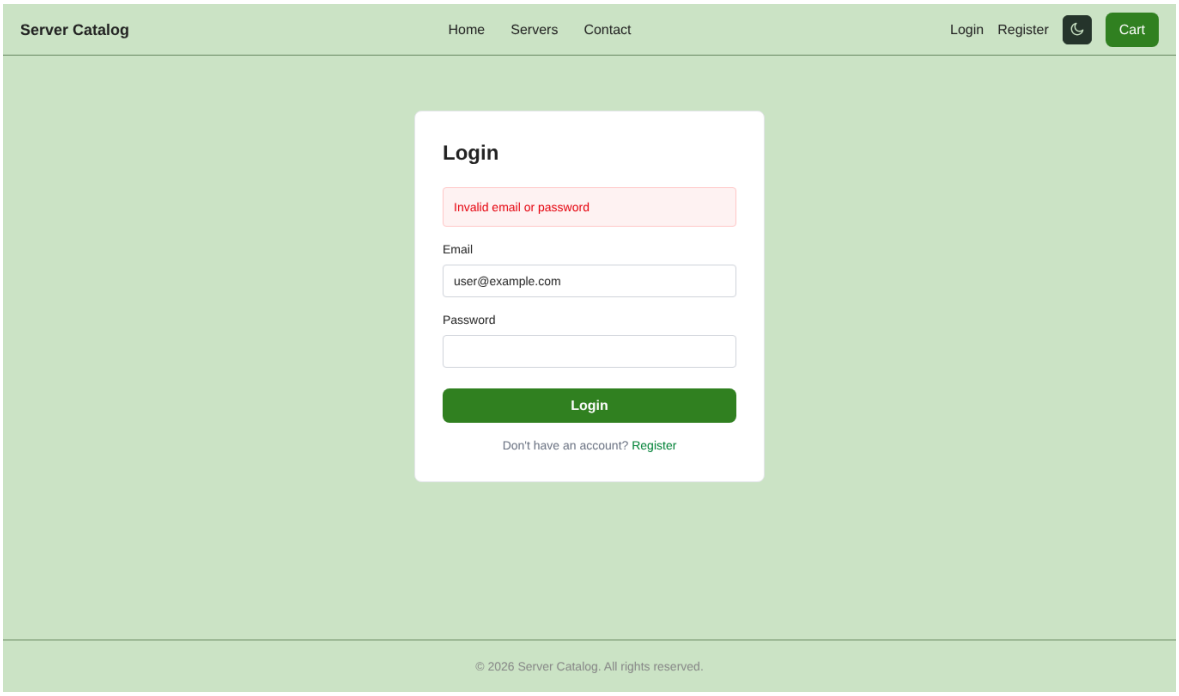


7.5. att. Autorizācijas lapa

Ja konta vēl nav, autorizācijas lapā jāizvēlas saite "Register". Reģistrācijas lapā (skat. 7.6. att.) jāievada vārds, e-pasta adrese, parole un paroles apstiprinājums. Ja kāds lauks nav aizpildīts vai aizpildīts nepareizi, zem tā parādās kļūdas paziņojums. Pēc reģistrācijas uz e-pastu tiek nosūtīta vēstule ar apstiprināšanas saiti, un lietotājs tiek novirzīts uz autorizācijas lapu. Nepareizas paroles gadījumā tiek rādīts paziņojums "Invalid email or password" (skat. 7.7. att.).

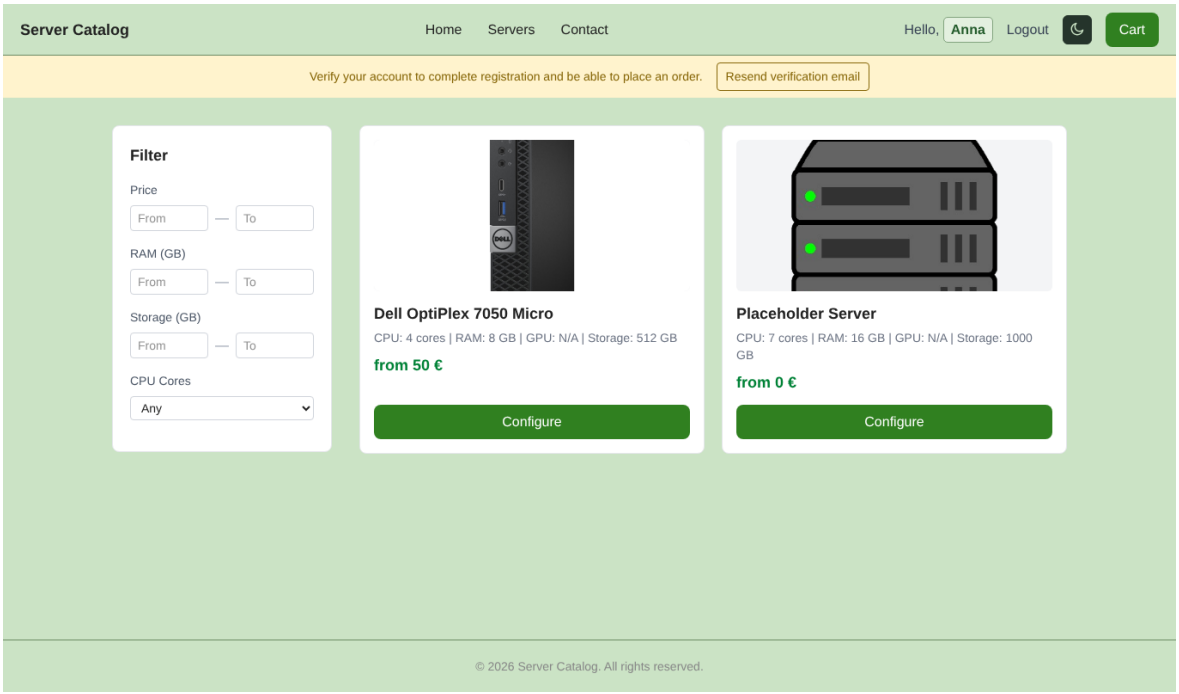


7.6. att. Reģistrācijas forma ar kļūdu paziņojumiem



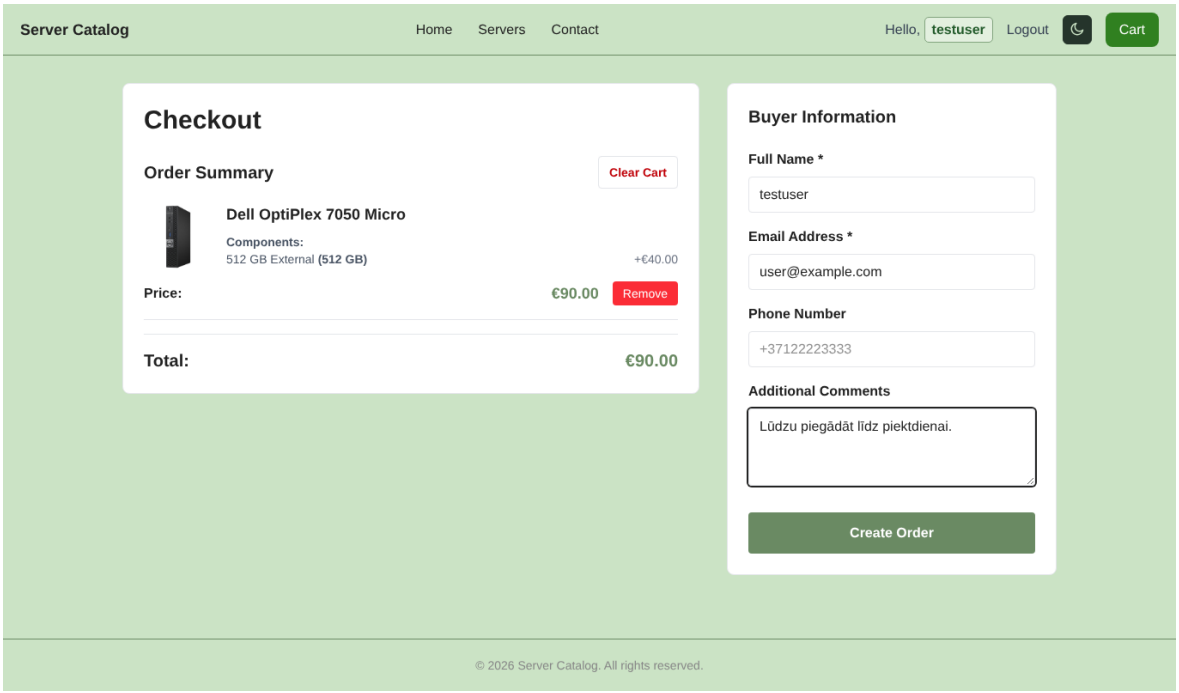
7.7. att. Autorizācijas kļūdas paziņojums

Kamēr lietotājs nav apstiprinājis e-pasta adresi, lapas augšā tiek rādīts paziņojums ar pogu “Resend verification email” (skat. 7.8. att.), un pasūtījumu noformēt nevar. Pēc saites atvēršanas e-pasta vēstulē adrese tiek apstiprināta un paziņojums pazūd.



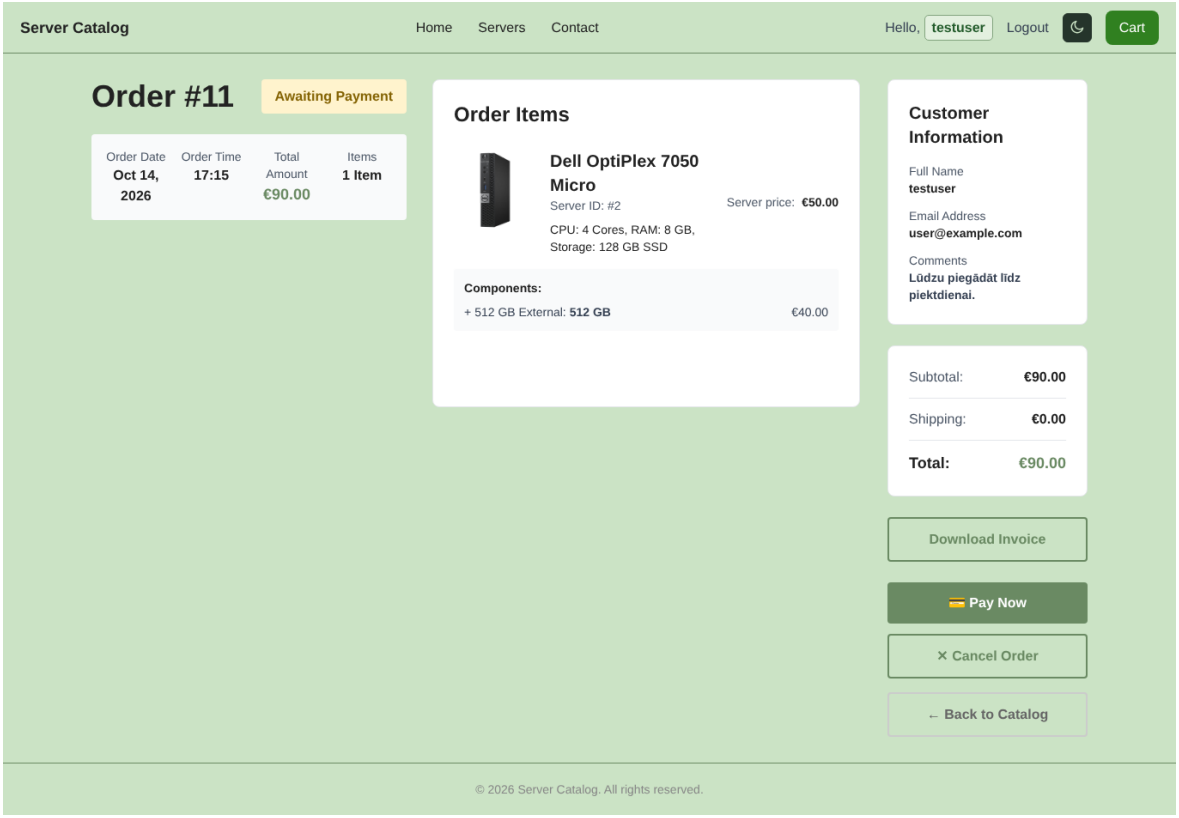
7.8. att. Paziņojums par neapstiprinātu e-pasta adresi

Lietotājam ar apstiprinātu e-pasta adresi poga “Checkout” atver pasūtījuma noformēšanas lapu (skat. 7.9. att.). Kreisajā pusē redzams pasūtījuma kopsavilkums, bet labajā – pircēja informācijas forma, kurā vārds, e-pasts un tālrunis jau ir aizpildīti no profila. Obligātie lauki ir atzīmēti ar zvaigznīti.



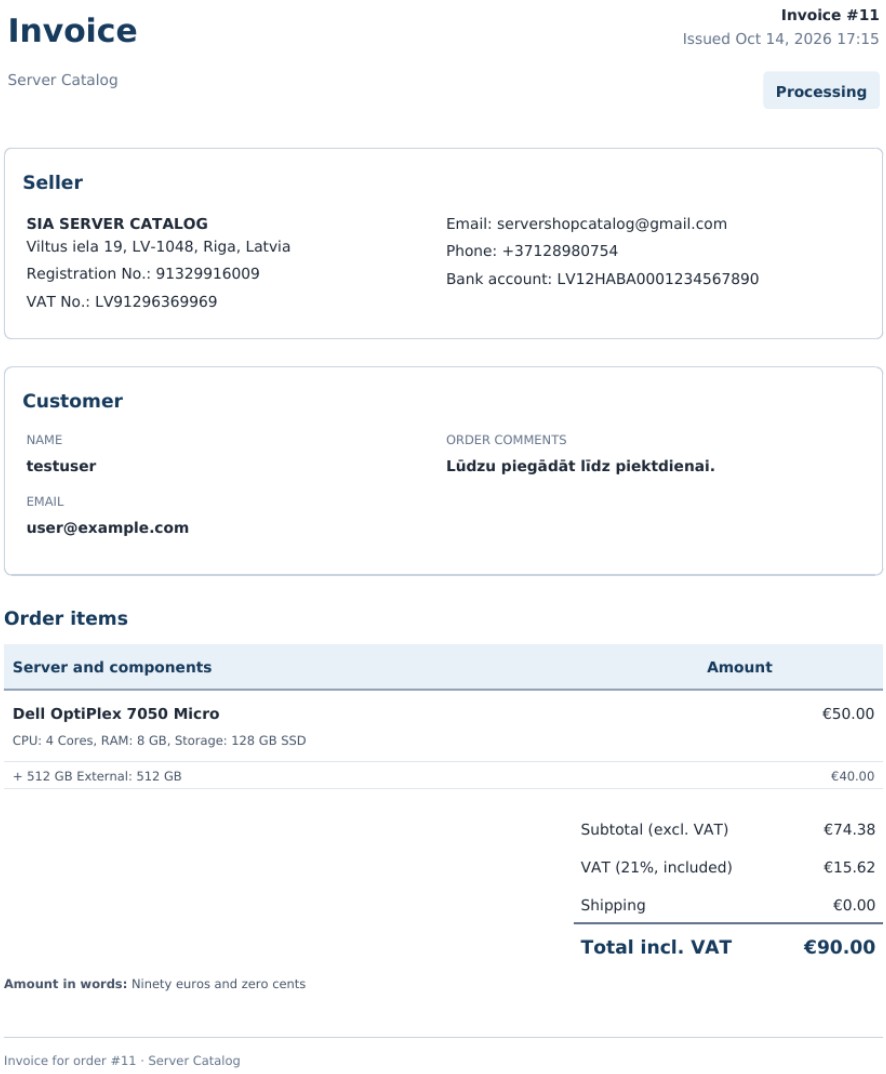
7.9. att. Pasūtījuma noformēšana

Pēc pogas “Create Order” nospiešanas tiek atvērta pasūtījuma lapa (skat. 7.10. att.) ar statusu “Awaiting Payment”. Šeit var lejupielādēt rēķinu, apmaksāt pasūtījumu ar pogu “Pay Now” vai atcelt to ar pogu “Cancel Order”. Pēc apmaksas statuss mainās uz “Processing” un rēķins tiek nosūtīts uz norādīto e-pasta adresi.



7.10. att. Pasūtījuma lapa pēc noformēšanas

Rēķins (skat. 7.11. att.) satur pārdevēja rekvizītus, pircēja datus, pasūtītos serverus ar komponentiem, summu bez PVN, PVN 21 % un kopsummu ciparіem un vārdiem.



Invoice

Server Catalog

Invoice #11
Issued Oct 14, 2026 17:15

Processing

Seller

SIA SERVER CATALOG
Viltus iela 19, LV-1048, Riga, Latvia
Registration No.: 91329916009
VAT No.: LV91296369969

Email: servershopcatalog@gmail.com
Phone: +37128980754
Bank account: LV12HABA0001234567890

Customer

NAME
testuser

EMAIL
user@example.com

ORDER COMMENTS
Lūdzu piegādāt līdz piektdienai.

Order items

Server and components	Amount
Dell OptiPlex 7050 Micro CPU: 4 Cores, RAM: 8 GB, Storage: 128 GB SSD	€50.00
+ 512 GB External: 512 GB	€40.00

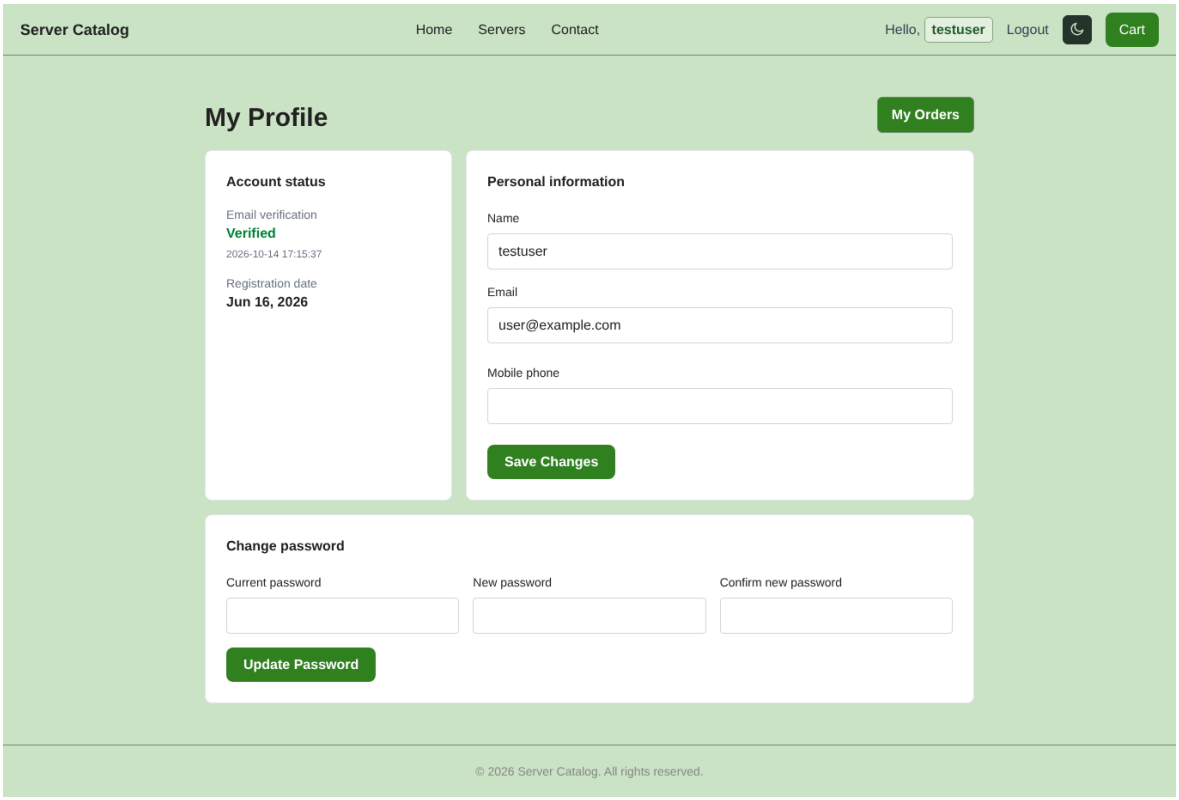
Subtotal (excl. VAT)	€74.38
VAT (21%, included)	€15.62
Shipping	€0.00
Total incl. VAT	**€90.00**

Amount in words: Ninety euros and zero cents

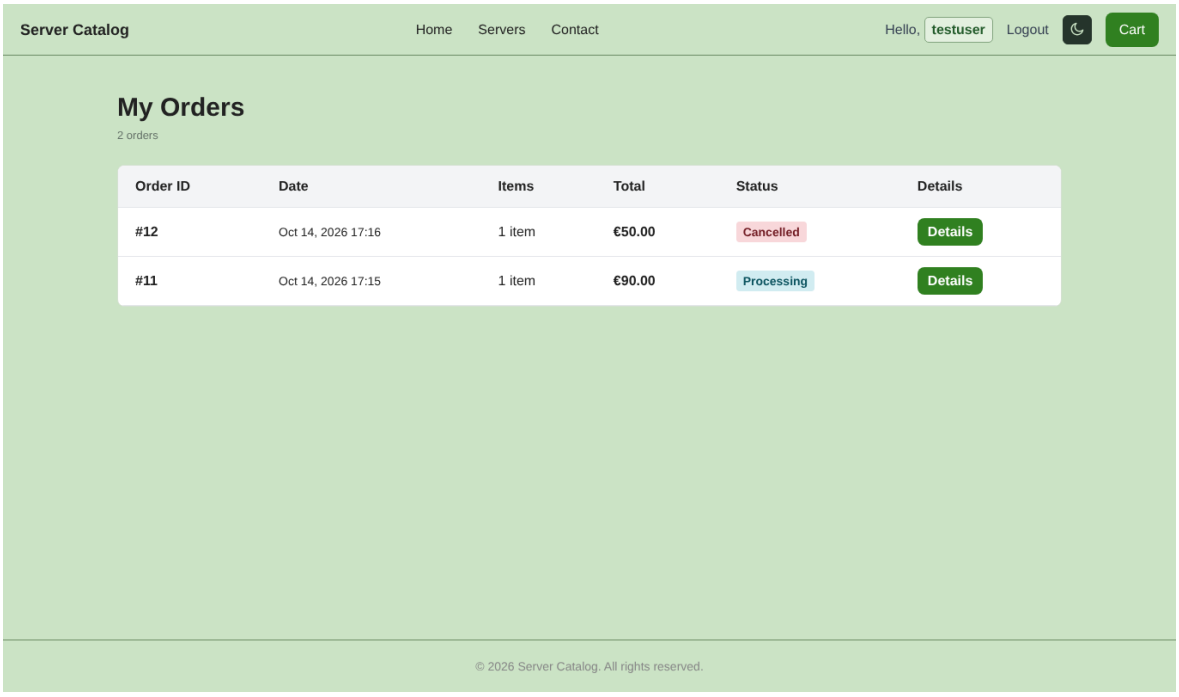
Invoice for order #11 · Server Catalog

7.11. att. PDF rēķins

Autorizēts lietotājs, nospiežot uz sava vārda galvenē, atver profilu (skat. 7.12. att.). Tajā redzams e-pasta apstiprināšanas statuss un reģistrācijas datums, var mainīt vārdu, e-pastu un mobilo tālruni, kā arī nomainīt paroli. Poga “My Orders” atver pasūtījumu vēsturi (skat. 7.13. att.).

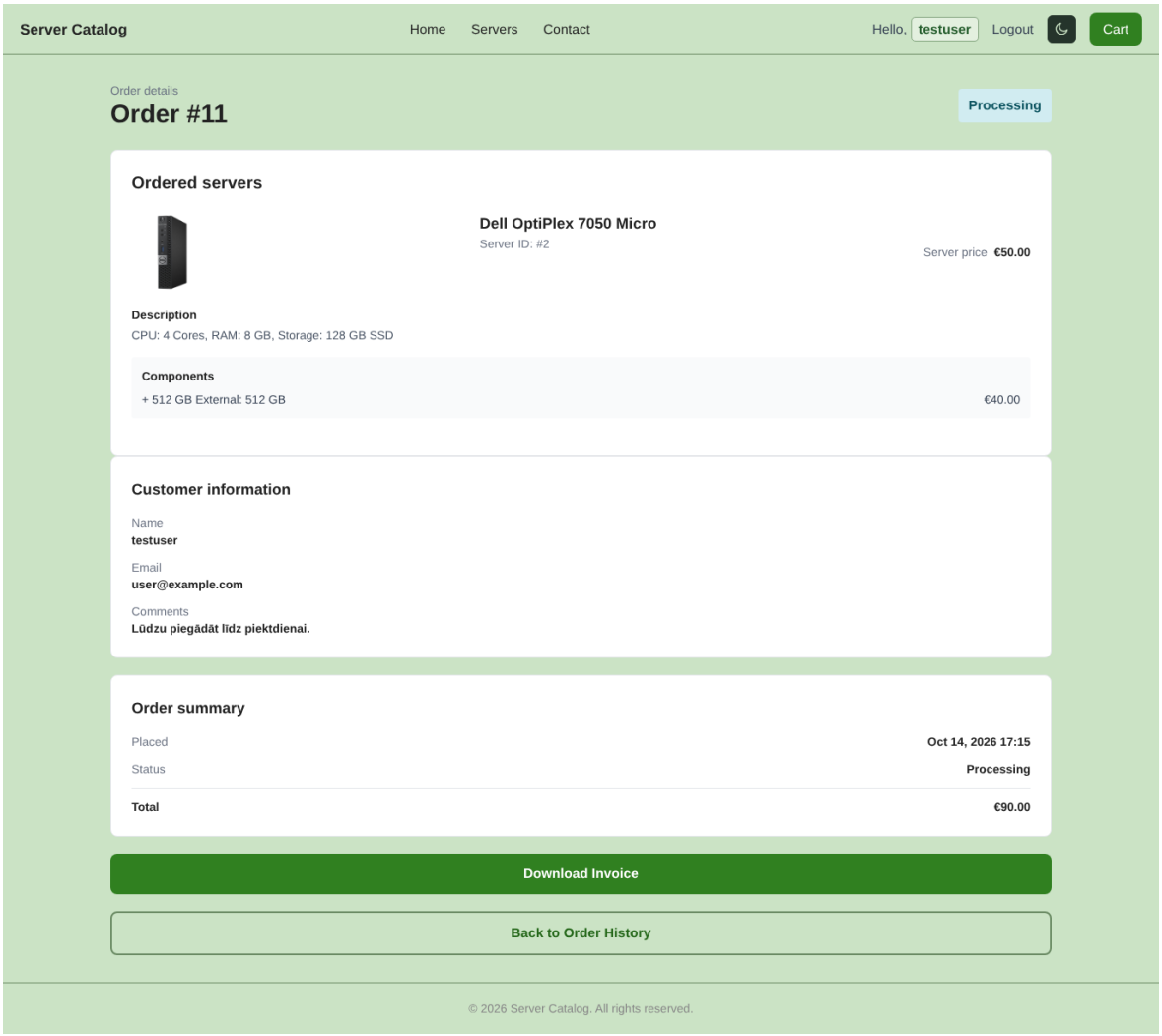


7.12. att. Lietotāja profils



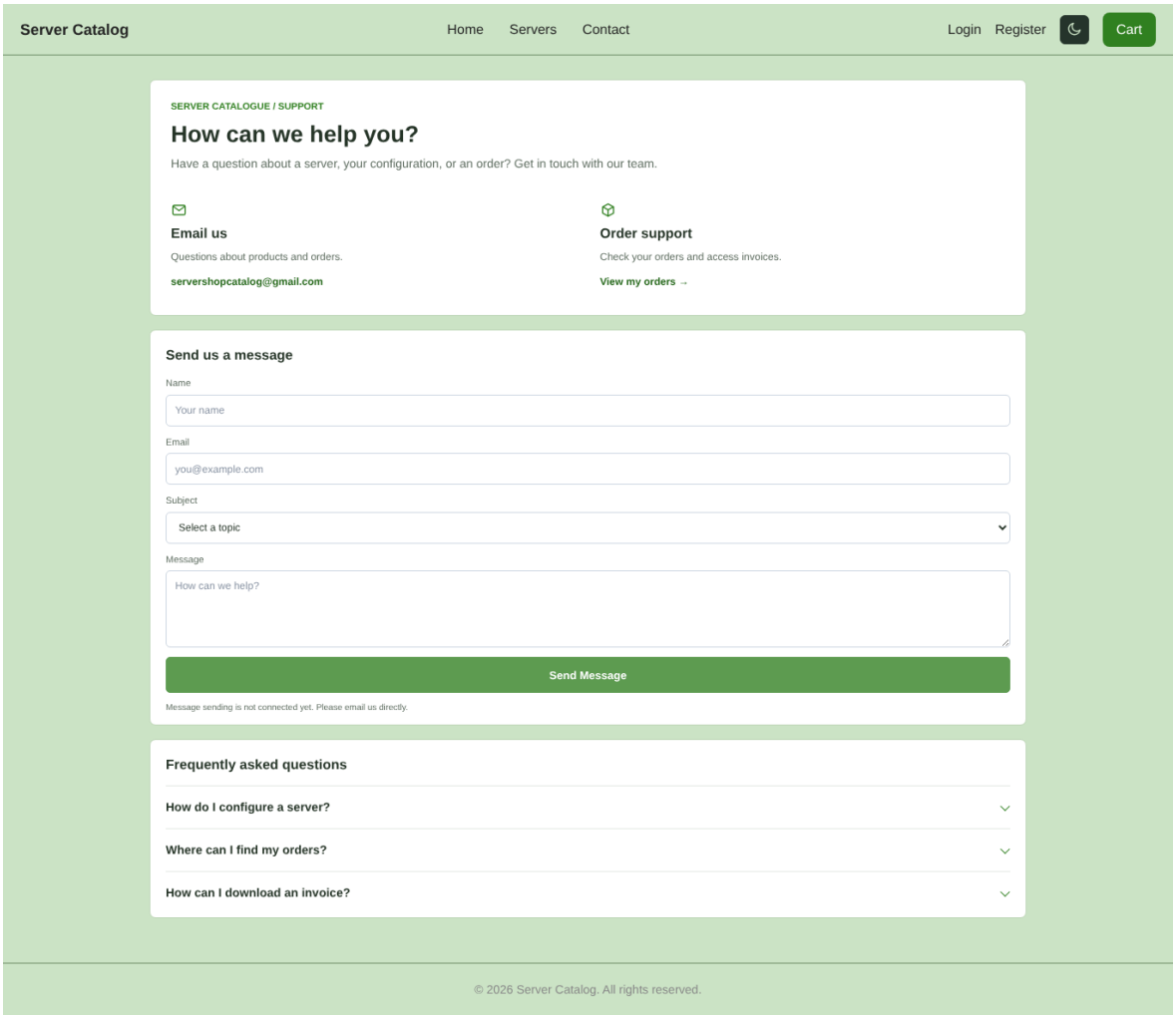
7.13. att. Pasūtījumu vēsture

Poga “Details” atver pasūtījuma detaļas (skat. 7.14. att.) ar pasūtītajiem serveriem, komponentiem, klienta informāciju, kopsavilkumu un rēķina lejupielādes pogu.



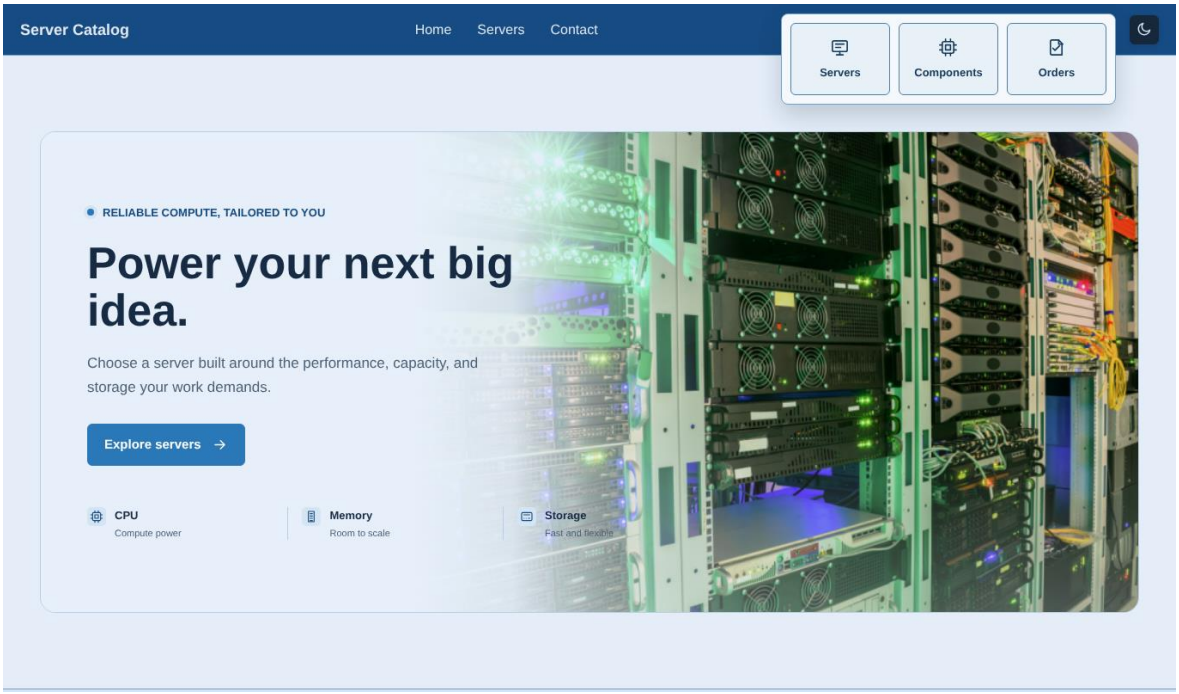
7.14. att. Pasūtījuma detaļas

Lapā “Contact” (skat. 7.15. att.) norādīta atbalsta e-pasta adrese, saite uz pasūtījumiem un biežāk uzdotie jautājumi.



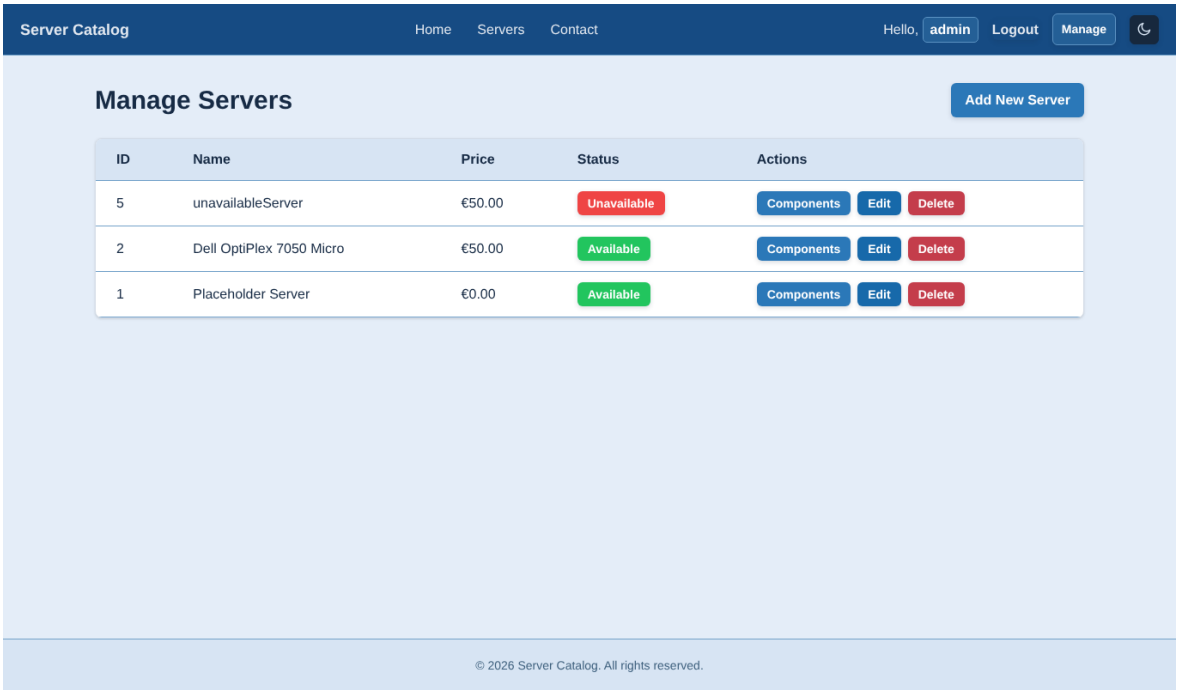
7.15. att. Kontaktu lapa

Administratora saskarne ir noformēta zilos toņos. Galvenē parādās izvēlne “Manage” ar saitēm uz serveru, komponentu un pasūtījumu pārvaldību (skat. 7.16. att.). Administratoram nav iepirkumu groza, jo administratori nevar veikt pasūtījumus.



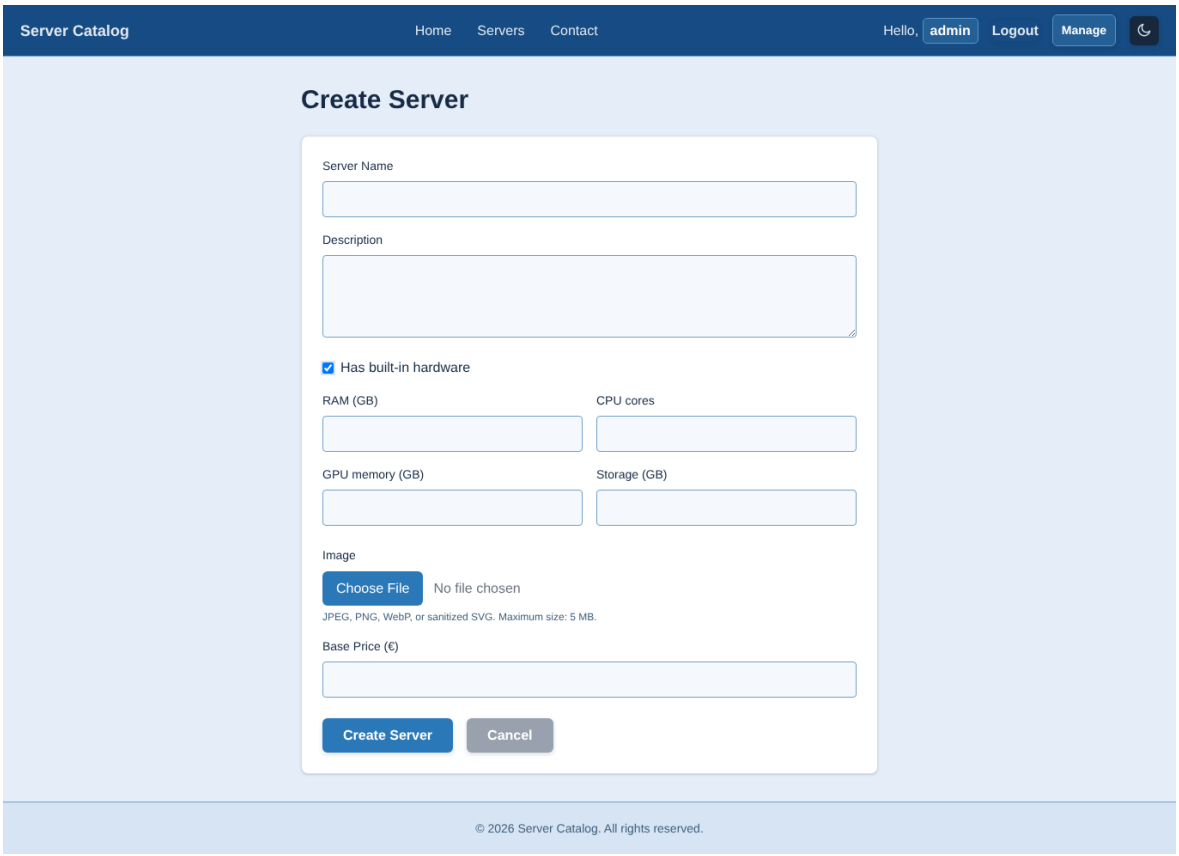
7.16. att. Administratora izvēlne “Manage”

Serveru pārvaldības lapā (skat. 7.17. att.) redzams visu serveru saraksts ar cenu un pieejamības statusu. Poga “Add New Server” atver servera pievienošanas formu, “Edit” – rediģēšanas formu, “Components” – komponentu piesaisti, “Delete” – dzēš serveri pēc apstiprinājuma. Statusa poga “Available”/“Unavailable” paslēpj serveri no kataloga vai atkal parāda to.



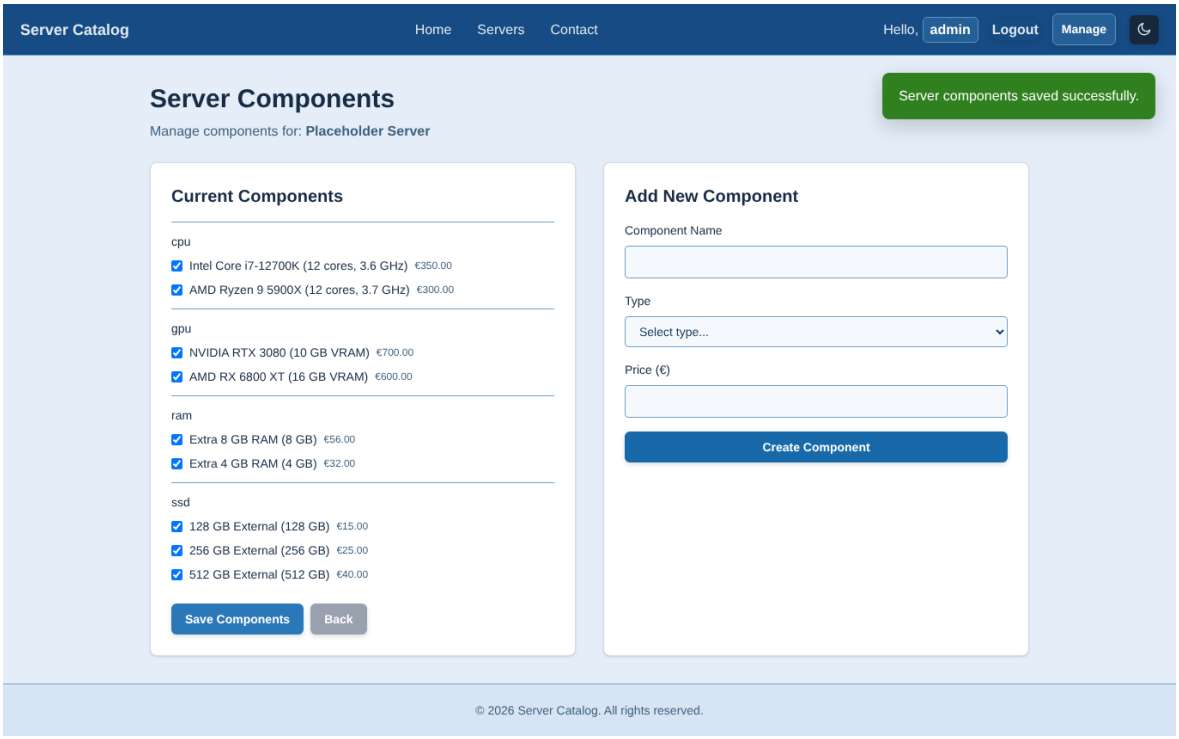
7.17. att. Serveru pārvaldība

Servera pievienošanas formā (skat. 7.18. att.) jāievada nosaukums, apraksts un pamatcena. Atzīmējot “Has built-in hardware”, parādās lauki iebūvētajai aparatūrai (RAM, CPU kodoli, GPU atmiņa, krātuve). Attēlu var augšupielādēt JPEG, PNG, WebP vai SVG formātā līdz 5 MB.



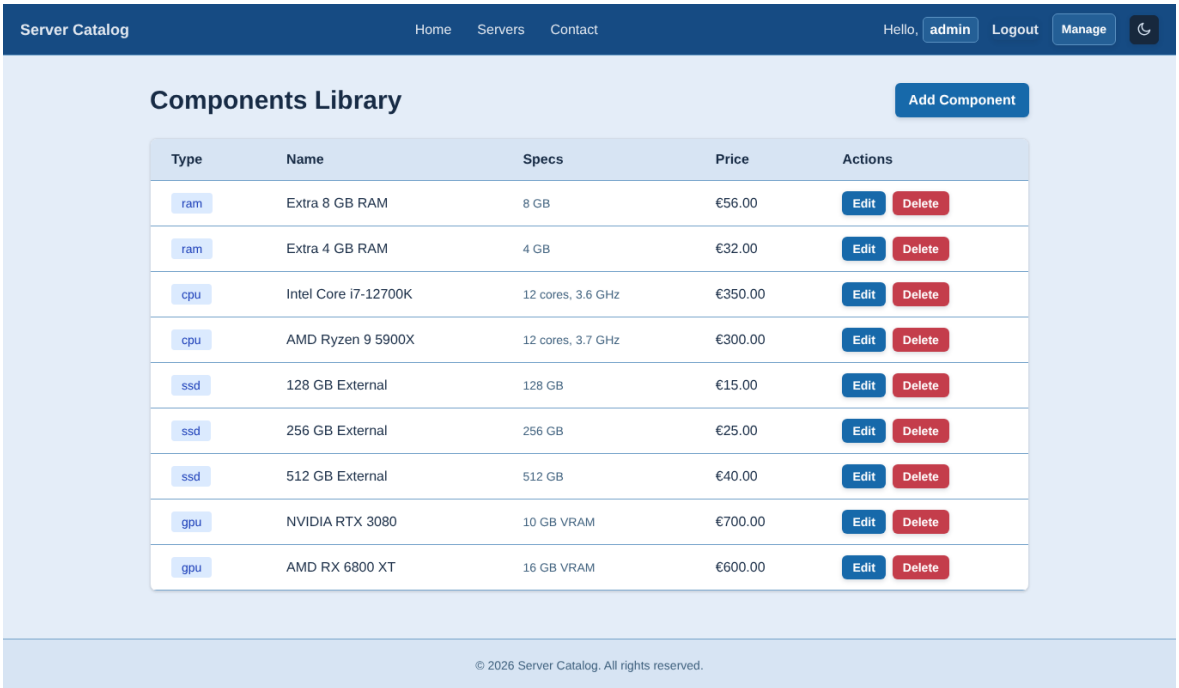
7.18. att. Servera pievienošanas forma

Lapā “Server Components” (skat. 7.19. att.) ar izvēles rūtiņām atzīmē, kuri komponenti būs pieejami servera konfiguratorā, un saglabā izvēli ar pogu “Save Components”. Pēc saglabāšanas tiek parādīts paziņojums “Server components saved successfully.” Labajā pusē var uzreiz izveidot jaunu komponentu.

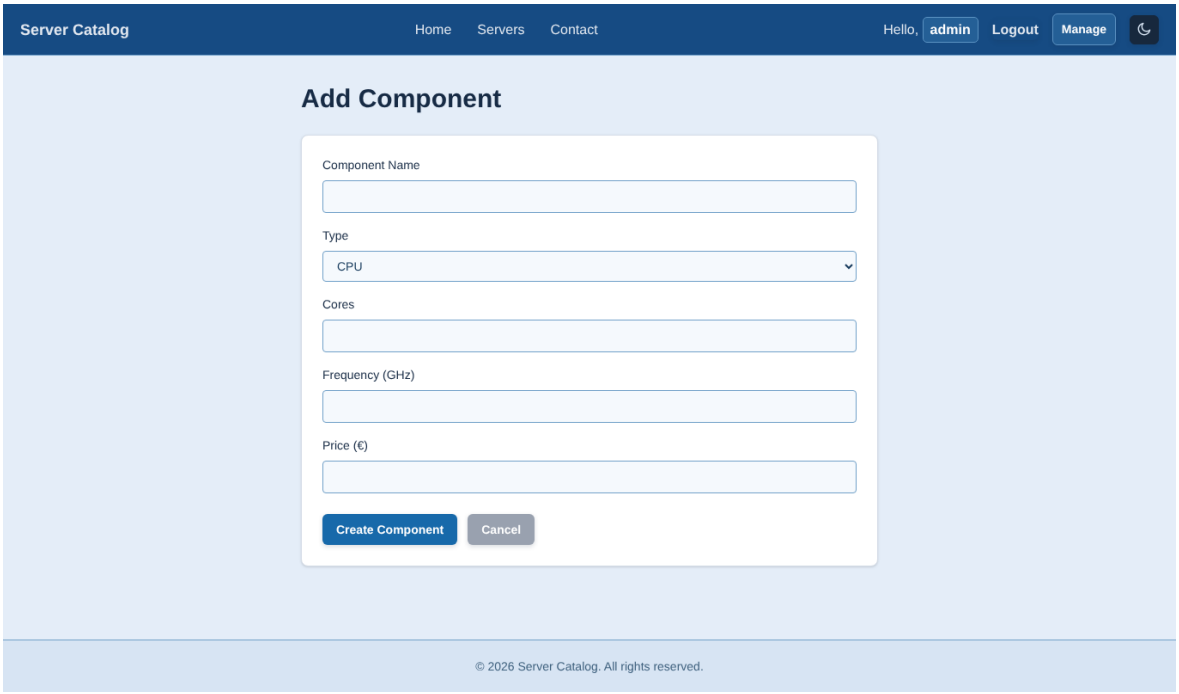


7.19. att. Komponentu piesaiste serverim

Komponentu bibliotēkā (skat. 7.20. att.) redzami visi komponenti ar tipu, specifikāciju un cenu. Pievienojot komponentu, pēc tipa izvēles parādās atbilstošie lauki: procesoram – kodolu skaits un frekvence, videokartei – atmiņa, RAM un diskiem – apjoms (skat. 7.21. att.).

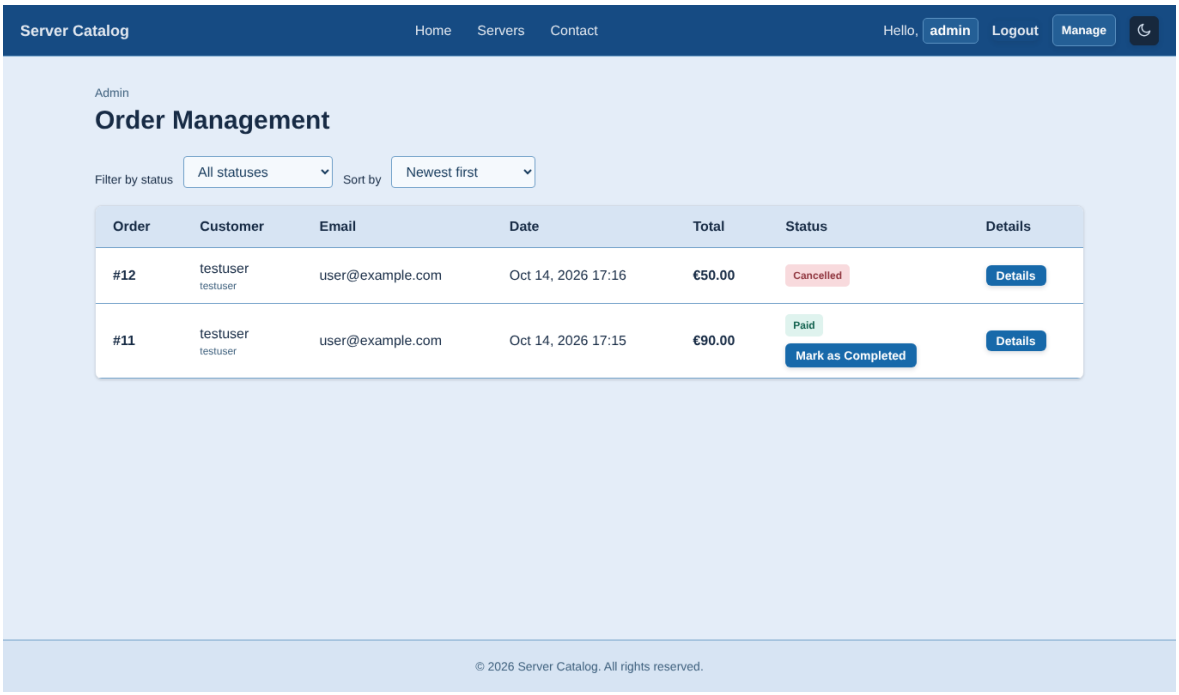


7.20. att. Komponentu bibliotēka



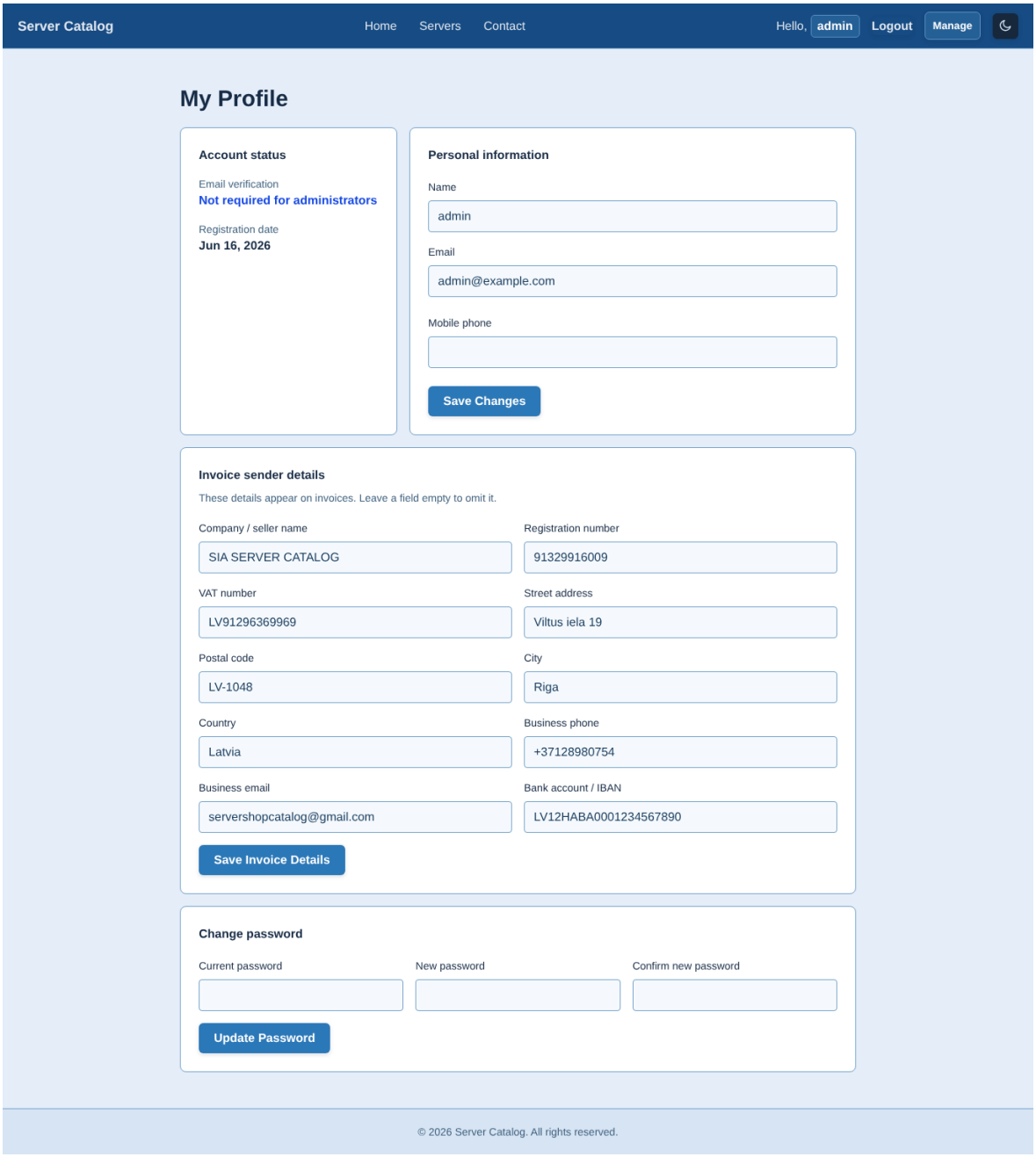
7.21. att. Komponenta pievienošanas forma

Pasūtījumu pārvaldības lapā (skat. 7.22. att.) redzami visi pasūtījumi. Tos var filtrēt pēc statusa un kārtot pēc datuma vai cenas. Pasūtījumam ar statusu "Processing" ir poga "Mark as Paid", bet ar statusu "Paid" – poga "Mark as Completed".



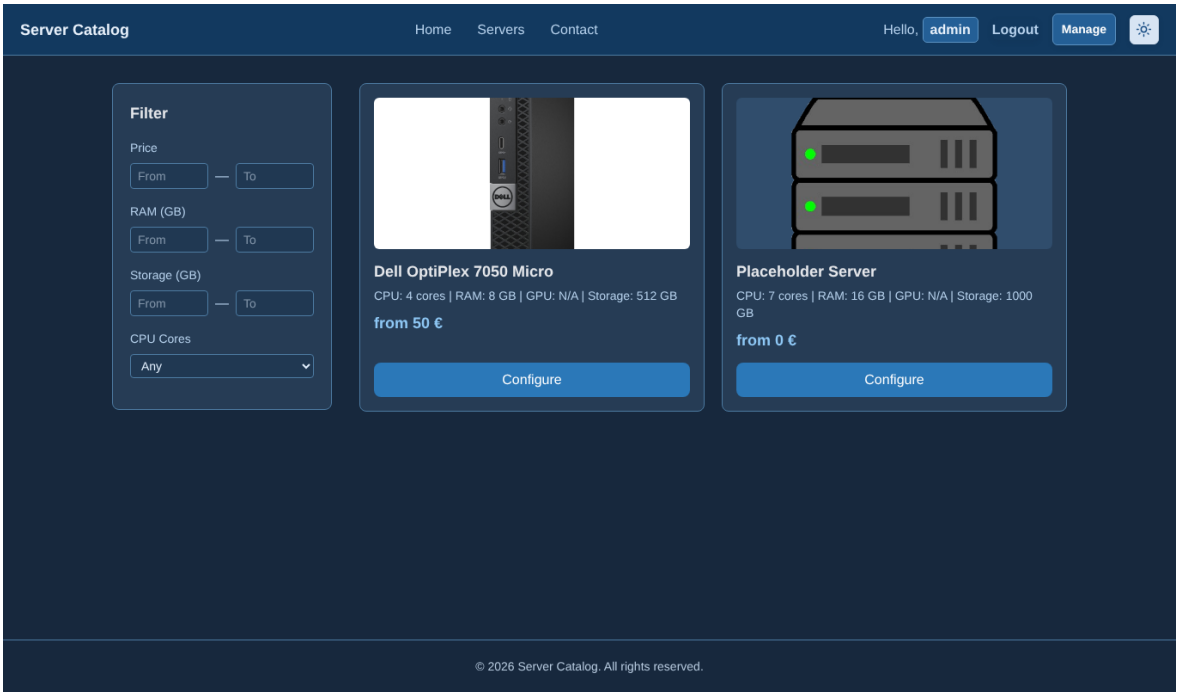
7.22. att. Pasūtījumu pārvaldība

Administratora profilā (skat. 7.23. att.) papildus ir sadaļa "Invoice sender details", kurā norāda rēķinos izmantojamos uzņēmuma rekvizītus.



7.23. att. Administratora profils ar rēķina rekvizītiem

Ar mēness ikonu galvenē var pārslēgt tumšo režīmu (skat. 7.24. att.). Izvēle tiek saglabāta pārlūkprogrammā.



7.24. att. Tumšais režīms

8. TESTA PIEMĒRS

Sistēmas funkcionalitāte tika pārbaudīta manuāli pārlūkprogrammā Google Chrome, kā arī ar tiešiem HTTP pieprasījumiem uz JSON saskarnes punktiem, lai pārbaudītu, ka servera puses validācija darbojas arī tad, ja pārlūkprogrammas pārbaudes tiek apietas. Testēšana tika veikta Docker vidē ar piemēra datubāzi. Turpmākajās tabulās apkopoti testa piemēri un to rezultāti.

8.1. tabula

Lietotāja reģistrācijas funkcionalitātes testēšana

Lietotāja loma:	Viesis	
Prasības identifikators:	PS 3.1.5	
Prasības apraksts		
Jāpārbauda, vai sistēma pareizi validē reģistrācijas formu un izveido jaunu kontu.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Iesniegt tukšu reģistrācijas formu.	Paziņojumi “Name is required”, “Email is required”, “Password is required”, “Confirmation password is required”.	Izpildīts
2. Ievadīt e-pastu “bad@x” un atšķirīgas paroles.	Paziņojumi “Invalid email format” un “Passwords do not match”.	Izpildīts
3. Ievadīt jau reģistrētu e-pastu “user@example.com”.	Paziņojums “This email is already taken”.	Izpildīts
4. Ievadīt korektus datus, reģistrāciju sākot no noformēšanas lapas.	Konts izveidots, lietotājs novirzīts uz `login.php?redirect=checkout.php`.	Izpildīts

8.2. tabula

Autorizācijas un piekļuves tiesību testēšana

Lietotāja loma:	Viesis, reģistrēts lietotājs	
Prasības identifikators:	PS 3.1.6, PS 3.1.14	
Prasības apraksts		
Jāpārbauda autorizācija, atgriešanās pēc pieteikšanās un administrācijas lapu aizsardzība.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Ievadīt nepareizu paroli.	Paziņojums “Invalid email or password”.	Izpildīts
2. Iesniegt formu ar nederīgu CSRF marķieri.	Paziņojums “Invalid CSRF token”.	Izpildīts
3. Pieteikties ar apstiprinātu kontu no lapas `login.php?redirect=checkout.php`.	Lietotājs novirzīts uz noformēšanas lapu, grozs saglabāts.	Izpildīts
4. Pieteikties ar parametru redirect, kas norāda uz svešu vietni.	Lietotājs novirzīts uz katalogu, nevis uz svešo vietni.	Izpildīts
5. Pieteikties ar neapstiprinātu kontu.	Lietotājs novirzīts uz katalogu, redzams paziņojums par e-pasta apstiprināšanu.	Izpildīts
6. Parastam lietotājam atvērt `admin/servers.php`.	Lietotājs novirzīts uz katalogu.	Izpildīts

8.3. tabula

Serveru kataloga filtrēšanas testēšana

Lietotāja loma:	Viesis	
Prasības identifikators:	PS 3.1.1	
Prasības apraksts		
Jāpārbauda, vai filtri atlasa pareizos serverus un vai nekorekts diapazons tiek noraidīts.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Katalogā izvēlēties "4 Cores".	Redzams tikai serveris "Dell OptiPlex 7050 Micro".	Izpildīts
2. Ievadīt cenu no 100 līdz 10.	Paziņojums "Maximum price must be greater than or equal to minimum."	Izpildīts
3. Atvērt katalogu.	Nepieejamais serveris "unavailableServer" netiek rādīts.	Izpildīts

8.4. tabula

Servera konfigurēšanas un groza testēšana

Lietotāja loma:	Viesis	
Prasības identifikators:	PS 3.1.2, PS 3.1.3, PS 3.1.4	
Prasības apraksts		
Jāpārbauda konfigurācijas pievienošana grozam un servera puses pārbaudes.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Konfiguratorā izvēlēties "512 GB External" un nospiest "Add to Cart".	Prece pievienota grozam ar cenu 90,00 € (50 € + 40 €), atvērts groza panelis ar pogu "Sign in to Checkout".	Izpildīts
2. Nosūtīt pieprasījumu ar komponentu, kas nav piesaistīts serverim.	Paziņojums "One or more components are unavailable for this server".	Izpildīts
3. Nosūtīt pieprasījumu ar diviem viena tipa komponentiem.	Paziņojums "Only one component per type can be selected".	Izpildīts
4. Nosūtīt pieprasījumu bez CSRF marķiera.	Paziņojums "Invalid CSRF token".	Izpildīts
5. Grozā nospiest "Clear Cart" un apstiprināt.	Grozs iztukšots.	Izpildīts

8.5. tabula

Pasūtījuma noformēšanas testēšana

Lietotāja loma:	Viesis, reģistrēts lietotājs	
Prasības identifikators:	PS 3.1.8	
Prasības apraksts		
Jāpārbauda, ka pasūtījumu var noformēt tikai autorizēts lietotājs ar apstiprinātu e-pastu, un jāpārbauda formas validācija.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Viesim atvērt noformēšanas lapu.	Novirzīšana uz `login.php?redirect=checkout.php`.	Izpildīts
2. Viesim nosūtīt pasūtījuma izveides pieprasījumu tieši uz API.	Atbilde “Sign in to place an order” (HTTP 401).	Izpildīts
3. Lietotājam ar neapstiprinātu e-pastu noformēt pasūtījumu.	Noformēšanas lapa novirza uz katalogu; API atbild “Verify your account to complete registration and be able to place an order.” (HTTP 403).	Izpildīts
4. Iesniegt formu bez vārda un e-pasta.	Paziņojums “Name and email are required”.	Izpildīts
5. Ievadīt e-pastu “abc”.	Paziņojums “Invalid email format”.	Izpildīts
6. Ievadīt tālruni “12ab”.	Paziņojums “Phone must contain up to 15 digits and may start with +”.	Izpildīts
7. Lietotājam ar apstiprinātu e-pastu iesniegt profilā aizpildīto formu.	Pasūtījums izveidots ar statusu “Awaiting Payment”, atvērta pasūtījuma lapa.	Izpildīts

8.6. tabula

Apmaksas, atcelšanas un rēķina testēšana

Lietotāja loma:	Viesis, reģistrēts lietotājs, administrators	
Prasības identifikators:	PS 3.1.9, PS 3.1.10, PS 3.1.11	
Prasības apraksts		
Jāpārbauda pasūtījuma statusa maiņa, piekļuves tiesības un rēķina ģenerēšana.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Pasūtījuma lapā nospiest “Pay Now”.	Statuss mainās uz “Processing”, grozs iztukšots.	Izpildīts
2. Mēģināt atcelt apmaksātu pasūtījumu.	Paziņojums “Cannot cancel order with status: processing”.	Izpildīts
3. Atcelt pasūtījumu ar statusu “Awaiting Payment”.	Statuss “Cancelled”, grozs netiek iztukšots.	Izpildīts
4. Lejupielādēt sava pasūtījuma rēķinu.	Tiek lejupielādēts PDF fails ar korektu PVN aprēķinu.	Izpildīts
5. Citam lietotājam vai viesim apmaksāt, atcelt vai lejupielādēt svešu pasūtījumu.	Paziņojumi “You can only pay for your own orders”, “You can only cancel your own orders”, rēķinam – “Access denied” (HTTP 403).	Izpildīts
6. Administratoram lejupielādēt jebkura pasūtījuma rēķinu.	Tiek lejupielādēts PDF fails.	Izpildīts

8.7. tabula

Profila rediģēšanas testēšana

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	PS 3.1.13	
Prasības apraksts		
Jāpārbauda profila datu un paroles maiņas validācija.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Paroles maiņā ievadīt nepareizu pašreizējo paroli, īsu un nesakrītošu jauno paroli.	Paziņojumi “Current password is incorrect”, “New password must be at least 8 characters”, “New passwords do not match”.	Izpildīts
2. Ievadīt tālruni “abc”.	Paziņojums “Phone must contain up to 15 digits and may start with +”.	Izpildīts
3. Ievadīt tālruni “+371 2000 0000”.	Paziņojums “Profile updated.”, saglabāts “+37120000000”.	Izpildīts

8.8. tabula

Serveru pārvaldības testēšana

Lietotāja loma:	Administrators	
Prasības identifikators:	PS 3.1.14	
Prasības apraksts		
Jāpārbauda servera formas validācija un attēlu augšupielādes drošība.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Iesniegt tukšu formu ar cenu “-1”.	Paziņojumi “Name is required”, “Description is required”, “Price must be a non-negative number”.	Izpildīts
2. Atzīmēt “Has built-in hardware”, neievadot vērtības.	Paziņojums “Enter at least one built-in hardware value”.	Izpildīts
3. Augšupielādēt teksta failu ar paplašinājumu .png.	Paziņojums “Only valid JPEG, PNG, WebP, and sanitized SVG images are allowed”.	Izpildīts
4. Augšupielādēt SVG attēlu ar elementu <script> un atribūtu onload.	Serveris izveidots, saglabātajā failā skripts un atribūts onload ir izdzēsti.	Izpildīts
5. Nosūtīt formu bez CSRF marķiera.	Atbilde “Invalid CSRF token” (HTTP 403).	Izpildīts
6. Pārslēgt servera pieejamību un dzēst serveri.	Serveris paslēpts no kataloga, pēc tam dzēsts.	Izpildīts

8.9. tabula

Komponentu pārvaldības testēšana

Lietotāja loma:	Administrators	
Prasības identifikators:	PS 3.1.16	
Prasības apraksts		
Jāpārbauda komponenta formas validācija.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Iesniegt formu bez nosaukuma, ar nederīgu tipu un cenu “-5”.	Paziņojumi “Component name is required”, “Component type is invalid”, “Price must be a non-negative number”.	Izpildīts
2. Izvēlēties tipu CPU, neievadot kodolus un frekvenci.	Paziņojums “CPU cores and frequency are required”.	Izpildīts

8.10. tabula

Pasūtījumu pārvaldības testēšana

Lietotāja loma:	Administrators	
Prasības identifikators:	PS 3.1.17	
Prasības apraksts		
Jāpārbauda pasūtījuma statusa maiņa un aizliegtās pārejas.		
Ievaddati / darbības apraksts	***Sagaidāmais rezultāts***	***Statuss***
1. Pasūtījumam ar statusu "Processing" nospiest "Mark as Paid".	Statuss mainās uz "Paid".	Izpildīts
2. Atkārtoti nosūtīt darbību "Mark as Paid" tam pašam pasūtījumam.	Paziņojums "This action is not available for the current order status".	Izpildīts
3. Nospiest "Mark as Completed".	Statuss mainās uz "Completed".	Izpildīts
4. Administratoram pievienot preci grozam.	Paziņojums "Administrator accounts cannot place orders".	Izpildīts

Papildus manuālajai testēšanai projektā ir automatizētie testi ar PHPUnit. Tie tiek palaisti ar komandu `docker compose exec app composer test`. Integrācijas tests pasūtījumam izmanto atsevišķu datubāzi server_eshop_test, lai neietekmētu darba datus.

8.11. tabula

Automatizēto testu rezultāti

Testa klase	Ko pārbauda	Testi	Rezultāts
ApiTest	API veselības punkts atbild ar 200 uz GET un ar 405 uz POST	2	Izpildīts
OrderTest	Pasūtījuma izveide reģistrētam lietotājam, statusa pāreja un vēstures saglabāšana pēc servera dzēšanas	1	Izpildīts
PhoneNumberTest	Tālruņa numura normalizēšana un nederīgu numuru noraidīšana	4	Izpildīts
RegistrationViewTest	Reģistrācijas formas kļūdu paziņojumu attēlošana	2	Izpildīts

Visi 9 automatizētie testi (30 pārbaudes) izpildījās veiksmīgi bez brīdinājumiem.

9. SECINĀJUMI

Kvalifikācijas darba rezultātā tika izstrādāta tīmekļa vietne lokālo serveru konfigurēšanai un pārdošanai. Sistēma ļauj apmeklētājiem apskatīt un filtrēt serveru katalogu, salikt servera konfigurāciju no pieejamajiem komponentiem un ielikt to grozā. Reģistrēti lietotāji pēc e-pasta adreses apstiprināšanas var noformēt pasūtījumu, apmaksāt vai atcelt to, saņemt PDF rēķinu e-pastā, pārvaldīt profilu un apskatīt pasūtījumu vēsturi, bet administrators – pārvaldīt serverus, komponentus, pasūtījumus un rēķinu rekvizītus. Visas uzdevuma nostādnē izvirzītās pamatfunkcijas ir realizētas.

Izstrādes gaitā tika nostiprinātas zināšanas PHP valodā, MVC arhitektūrā, darbā ar MySQL datubāzi caur PDO, kā arī KnockoutJS un Tailwind CSS izmantošanā. Īpaša uzmanība tika pievērsta drošībai: paroles tiek glabātas jauktā veidā, visi datus mainošie pieprasījumi ir aizsargāti ar CSRF marķieri, cena tiek aprēķināta serverī, augšupielādētie attēli tiek pārbaudīti, bet pasūtījumā tiek saglabātas preču datu kopijas, lai vēsture nemainītos. Projekta konteinerizācija ar Docker ļauj to palaist uz jebkura datora ar vienu komandu.

Salīdzinot ar sākotnējo plānu, daļa ieceru netika realizēta un var tikt pievienota nākotnē: iebūvētā tērzēšana ar administratoru, paziņojumi administratoram par jauniem pasūtījumiem, reģistrēto lietotāju pārvaldības lapa, operētājsistēmas izvēle konfiguratorā, izvēlēto komponentu savietojamības pārbaude ar pārējo servera aparatūru un integrācija ar reālu maksājumu sistēmu. Kontaktu lapas ziņojuma forma pagaidām nav savienota ar e-pasta sūtīšanu.

Izstrādātā sistēma ir gatava demonstrācijai un var kalpot par pamatu reālam serveru pārdošanas pakalpojumam, ja to papildina ar maksājumu apstrādi un publisku hostingu.

INFORMĀCIJAS AVOTI

1. [ANG] Nextcloud. – https://nextcloud.com/ – (Resurss apskatīts 15.11.2025.)

2. [ANG] Dropbox plāni un cenas. – https://www.dropbox.com/plans – (Resurss apskatīts 12.12.2025.)

3. [ANG] Google One plāni. – https://one.google.com/about/plans – (Resurss apskatīts 12.12.2025.)

4. [ANG] MySQL 8.4 Reference Manual. – https://dev.mysql.com/doc/refman/8.4/en/ – (Resurss apskatīts 28.04.2026.)

5. [ANG] MDN Web Docs. – https://developer.mozilla.org/ – (Resurss apskatīts 04.05.2026.)

6. [ANG] KnockoutJS dokumentācija. – https://knockoutjs.com/documentation/introduction.html – (Resurss apskatīts 11.05.2026.)

7. [ANG] PHP funkcija password_hash. – https://www.php.net/manual/en/function.password-hash.php – (Resurss apskatīts 22.05.2026.)

8. [ANG] Docker dokumentācija. – https://docs.docker.com/ – (Resurss apskatīts 18.08.2026.)

9. [ANG] Docker Compose dokumentācija. – https://docs.docker.com/compose/ – (Resurss apskatīts 18.08.2026.)

10. [ANG] Tailwind CSS dokumentācija. – https://tailwindcss.com/docs – (Resurss apskatīts 24.09.2026.)

11. [ANG] OWASP Cross-Site Request Forgery Prevention Cheat Sheet. – https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html – (Resurss apskatīts 29.09.2026.)

12. [ANG] PlantUML dokumentācija. – https://plantuml.com/ – (Resurss apskatīts 06.10.2026.)

13. [ANG] Dompdf (GitHub repozitorijs). – https://github.com/dompdf/dompdf – (Resurss apskatīts 08.10.2026.)

14. [ANG] PHPMailer (GitHub repozitorijs). – https://github.com/PHPMailer/PHPMailer – (Resurss apskatīts 08.10.2026.)

15. [ANG] phpdotenv (GitHub repozitorijs). – https://github.com/vlucas/phpdotenv – (Resurss apskatīts 08.10.2026.)

16. [ANG] Mailpit dokumentācija. – https://mailpit.axllent.org/docs/ – (Resurss apskatīts 08.10.2026.)

17. [ANG] PHPUnit 11.5 dokumentācija. – https://docs.phpunit.de/en/11.5/ – (Resurss apskatīts 10.10.2026.)

PIELIKUMI

10. PIELIKUMS

10.1. FAILU STRUKTŪRA

Projekta failu struktūra (bez mapēm vendor, node_modules un .git, kuras tiek ģenerētas automātiski):

```
serverShop-course-dev/
├── database/
│   ├── migrations/
│   │   ├── 001_create_users_table.sql
│   │   ├── 002_create_servers_table.sql
│   │   ├── 003_create_components_table.sql
│   │   └── … (9 migrāciju faili)
│   ├── schema.sql
│   └── server_eshop.sql
├── public/
│   ├── admin/
│   │   ├── components.php
│   │   ├── index.php
│   │   ├── orders.php
│   │   └── servers.php
│   ├── api/
│   │   ├── auth/
│   │   │   ├── login.php
│   │   │   ├── logout.php
│   │   │   ├── register.php
│   │   │   └── resend-verification.php
│   │   ├── cart/
│   │   │   ├── add.php
│   │   │   ├── clear.php
│   │   │   ├── get.php
│   │   │   └── remove.php
│   │   ├── cookies/
│   │   │   └── consent.php
│   │   ├── orders/
│   │   │   ├── cancel.php
│   │   │   ├── create.php
│   │   │   ├── get.php
│   │   │   ├── list.php
│   │   │   └── pay.php
│   │   ├── profile/
│   │   │   ├── get.php
│   │   │   └── update.php
│   │   ├── servers/
│   │   │   ├── components.php
│   │   │   ├── exactUnit.php
│   │   │   └── list.php
│   │   ├── .htaccess
│   │   └── test.php
│   ├── assets/
│   │   ├── images/
│   │   │   └── … (servera attēli)
│   │   └── uploads/
│   │       └── .gitkeep
│   ├── css/
│   │   ├── admin.css
│   │   ├── main.css
│   │   ├── tailwind-output.css
│   │   └── tailwind.css
│   ├── js/
│   │   ├── cart.js
│   │   ├── configurator.js
│   │   ├── knockout.js
│   │   └── notifications.js
│   ├── .htaccess
│   ├── checkout.php
│   ├── configurator.php
│   ├── contact.php
│   ├── home.php
```

```
│   ├── index.php
│   ├── invoice.php
│   ├── login.php
│   ├── logout.php
│   ├── my-orders.php
│   ├── order-details.php
│   ├── order.php
│   ├── profile.php
│   ├── register.php
│   └── verify.php
├── src/
│   ├── Core/
│   │   ├── bootstrap.php
│   │   └── functions.php
│   ├── config/
│   │   ├── app.php
│   │   └── db.php
│   ├── controllers/
│   │   ├── admin/
│   │   │   ├── ComponentController.php
│   │   │   ├── DashboardController.php
│   │   │   ├── OrderController.php
│   │   │   ├── ServerController.php
│   │   │   └── UserController.php
│   │   ├── AuthController.php
│   │   ├── ConfiguratorController.php
│   │   ├── OrderController.php
│   │   ├── ProfileController.php
│   │   └── ServerController.php
│   ├── helpers/
│   │   ├── Cookie.php
│   │   ├── Response.php
│   │   └── Validator.php
│   ├── middleware/
│   │   ├── Admin.php
│   │   ├── Auth.php
│   │   └── Guest.php
│   ├── models/
│   │   ├── Cart.php
│   │   ├── Component.php
│   │   ├── Order.php
│   │   ├── Server.php
│   │   ├── ServerComponent.php
│   │   └── User.php
│   └── services/
│       ├── CompatibilityChecker.php
│       ├── InvoiceGenerator.php
│       └── Mailer.php
├── templates/
│   ├── admin/
│   │   ├── component-create.php
│   │   ├── component-edit.php
│   │   ├── components-list.php
│   │   ├── orders-list.php
│   │   ├── server-components.php
│   │   ├── server-create.php
│   │   ├── server-edit.php
│   │   └── servers-list.php
│   ├── components/
│   │   ├── cartSidebar.php
│   │   └── cookieBanner.php
│   ├── catalog.php
│   ├── checkout.php
│   ├── configurator.php
│   ├── contact.php
│   ├── footer.php
│   ├── header.php
│   ├── home.php
│   ├── invoice.php
│   ├── login.php
│   ├── my-orders.php
│   ├── order-details.php
│   ├── order.php
│   ├── profile.php
│   ├── register.php
│   └── verify.php
```

```
├── tests/
│   ├── ApiTest.php
│   ├── OrderTest.php
│   ├── PhoneNumberTest.php
│   └── RegistrationViewTest.php
├── .dockerignore
├── .env
├── .env.example
├── .gitignore
├── Dockerfile
├── README.md
├── compose.yaml
├── composer.json
├── composer.lock
├── debug.php
├── package-lock.json
└── package.json
```

10.2. PIRMKODS

Šajā pielikumā iekļauts svarīgāko sistēmas daļu pirmkods ar komentāriem – 8 faili, kopā 1357 rindas. Pilns pirmkods ir pieejams GitHub repozitorijā: https://github.com/SergeyGrinn/serverShop-course-dev

Fails `src/Core/functions.php` – palīgfunkcijas, tostarp CSRF marķiera izveide un pārbaude:

```
<?php

if (!function_exists('base_path')) {
    /**
     * Builds an absolute filesystem path inside the project.
     *
     * @param string $path Relative path to append.
     * @return string Absolute path.
     */
    function base_path($path = '') {
        if (defined('BASE_PATH')) {
            return BASE_PATH . $path;
        }

        return ROOT_PATH . ltrim($path, DIRECTORY_SEPARATOR);
    }
}

if (!function_exists('csrf_token')) {
    /**
     * Returns the session CSRF token, generating it on first use.
     *
     * @return string Random hex token to embed in forms.
     */
    function csrf_token(): string
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        if (empty($_SESSION['csrf_token'])) {
            $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
        }

        return $_SESSION['csrf_token'];
```

```
    }
}

if (!function_exists('csrf_verify')) {
    /**
     * Checks a submitted CSRF token against the session token.
     *
     * @param mixed $token Token received from the request.
     * @return bool True if it matches.
     */
    function csrf_verify($token): bool
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        // hash_equals gives a constant-time comparison, avoiding timing attacks
        return is_string($token)
            && $token !== ''
            && !empty($_SESSION['csrf_token'])
            && hash_equals($_SESSION['csrf_token'], $token);
    }
}
```

Fails `src/models/Cart.php` – iepirkumu groza modelis – pārbauda izvēlēto serveri un komponentus un aprēķina cenu no datubāzes datiem:

```
<?php

/**
 * Data-access model for shopping carts and their items, keyed by PHP session.
 */
class Cart {
    private $pdo;

    /**
     * Stores the database connection used by all cart queries.
     *
     * @param PDO $pdo Active database connection.
     */
    public function __construct($pdo) {
        $this->pdo = $pdo;
    }


    /**
     * Returns the cart bound to the session, creating it so every visitor always has one.
     *
     * @param string $session_id Current PHP session id.
     * @return array Cart row (at least id and session_id).
     */
    public function getOrCreateCart($session_id){ // Get existing cart for session or create
new one
        $stmt = $this->pdo->prepare("SELECT * FROM carts WHERE session_id = :session_id"); //
Try to find cart for current session
        $stmt->execute([':session_id' => $session_id]); // If cart exists, return it.
Otherwise, create new cart and return it
        $cart = $stmt->fetch();
        if (!$cart) { // If no cart found, create new one
            $stmt = $this->pdo->prepare("INSERT INTO carts (session_id) VALUES
(:session_id)");
            $stmt->execute([':session_id' => $session_id]);
            $cart = ['id' => $this->pdo->lastInsertId(), 'session_id' => $session_id];
        }
        return $cart;
    }
```

```
    /**
     * Loads cart items with server info and the details of each chosen component.
     *
     * @param int $cart_id Cart identifier.
     * @return array List of items, each with a 'components' array.
     */
    public function getItems($cart_id) {
    $stmt = $this->pdo->prepare("
        SELECT ci.*, s.name, s.image
        FROM cart_items ci
        JOIN servers s ON ci.server_id = s.id
        WHERE ci.cart_id = :cart_id
    ");
    $stmt->execute([':cart_id' => $cart_id]);
    $items = $stmt->fetchAll();

    foreach ($items as &$item) {
        $ids = json_decode($item['component_ids'], true);
        if (!empty($ids)) {
            // One "?" per id keeps the IN() list parameterized (no SQL injection)
            $placeholders = implode(',', array_fill(0, count($ids), '?'));
            $stmt2 = $this->pdo->prepare("
                SELECT name, value, type, price
                FROM components
                WHERE id IN ($placeholders)
            ");
            $stmt2->execute($ids);
            $item['components'] = $stmt2->fetchAll();
        } else {
            $item['components'] = [];
        }
    }

    return $items;
}

    /**
     * Validates a configuration and recalculates its price on the server,
     * so prices sent by the browser are never trusted.
     *
     * @param mixed $serverId Server id from the request.
     * @param array $componentIds Selected component ids from the request.
     * @return array ['component_ids' => int[], 'total_price' => float].
     * @throws InvalidArgumentException If the server/components are invalid, unavailable or
duplicated by type.
     */
    public function calculateConfiguredItem($serverId, array $componentIds): array {
        $serverId = filter_var($serverId, FILTER_VALIDATE_INT);
        if ($serverId === false || $serverId < 1) {
            throw new InvalidArgumentException('Invalid server selected');
        }

        $serverStmt = $this->pdo->prepare('SELECT base_price FROM servers WHERE id = :id AND
available = TRUE');
        $serverStmt->execute([':id' => $serverId]);
        $server = $serverStmt->fetch();
        if (!$server) {
            throw new InvalidArgumentException('Server is unavailable');
        }

        // Cast every id to a positive int and reject duplicates
        $selectedIds = [];
        foreach ($componentIds as $componentId) {
            if ($componentId === null || $componentId === '') {
                continue;
            }

            $componentId = filter_var($componentId, FILTER_VALIDATE_INT);
```

```
            if ($componentId === false || $componentId < 1 || in_array($componentId,
$selectedIds, true)) {
                throw new InvalidArgumentException('Invalid component selection');
            }
            $selectedIds[] = $componentId;
        }

        $totalPrice = (float) $server['base_price'];
        if ($selectedIds) {
            // Named placeholders keep the query safe from SQL injection
            $placeholders = [];
            $params = [':server_id' => $serverId];
            foreach ($selectedIds as $index => $componentId) {
                $placeholder = ':component_' . $index;
                $placeholders[] = $placeholder;
                $params[$placeholder] = $componentId;
            }

            $componentStmt = $this->pdo->prepare('
                SELECT c.id, c.type, c.price
                FROM components c
                INNER JOIN server_components sc ON sc.component_id = c.id
                WHERE sc.server_id = :server_id
                  AND c.available = TRUE
                  AND c.id IN (' . implode(', ', $placeholders) . ')
            ');
            $componentStmt->execute($params);
            $components = $componentStmt->fetchAll();

            // A mismatch means some component is unavailable or not compatible with this
server
            if (count($components) !== count($selectedIds)) {
                throw new InvalidArgumentException('One or more components are unavailable for
this server');
            }

            $selectedTypes = [];
            foreach ($components as $component) {
                if (in_array($component['type'], $selectedTypes, true)) {
                    throw new InvalidArgumentException('Only one component per type can be
selected');
                }
                $selectedTypes[] = $component['type'];
                $totalPrice += (float) $component['price'];
            }
        }

        return [
            'component_ids' => $selectedIds,
            'total_price' => round($totalPrice, 2),
        ];
    }

    /**
     * Adds a configured server to the cart; components are stored as JSON.
     *
     * @param int $cart_id Target cart id.
     * @param int $server_id Chosen server id.
     * @param array $component_ids Validated component ids.
     * @param float $total_price Price calculated on the server.
     * @return void
     */
     public function addItem($cart_id, $server_id, $component_ids, $total_price) {
        $stmt = $this->pdo->prepare("
            INSERT INTO cart_items (cart_id, server_id, component_ids, total_price)
            VALUES (:cart_id, :server_id, :component_ids, :total_price)
        ");
        $stmt->execute([
            ':cart_id' => $cart_id,
```

```
            ':server_id' => $server_id,
            ':component_ids' => json_encode($component_ids),
            ':total_price' => $total_price
        ]);
    }

    /**
     * Removes an item only if its cart belongs to the given session.
     *
     * @param int $item_id Cart item id.
     * @param string $session_id Current session id.
     * @return bool True if an item was deleted.
     */
    public function removeItem($item_id, $session_id) {
        // Joining on session_id prevents deleting items from other users' carts
        $stmt = $this->pdo->prepare("
            DELETE ci
            FROM cart_items ci
            INNER JOIN carts c ON c.id = ci.cart_id
            WHERE ci.id = :item_id AND c.session_id = :session_id
        ");

        $stmt->execute([
            ':item_id' => $item_id,
            ':session_id' => $session_id
        ]);

        return $stmt->rowCount() > 0;
    }

    /**
     * Deletes all items from a cart, e.g. after an order is placed.
     *
     * @param int $cart_id Cart identifier.
     * @return bool True on success.
     */
    public function clearCart($cart_id) {
        $stmt = $this->pdo->prepare("DELETE FROM cart_items WHERE cart_id = :cart_id");
        return $stmt->execute([':cart_id' => $cart_id]);
    }
}
```

Fails `src/models/Order.php` – pasūtījumu modelis – vienā transakcijā izveido pasūtījumu un saglabā servera un komponentu datu kopijas:

```
<?php

/**
 * Data-access model for orders and their items.
 */
class Order {
    private $pdo;

    /**
     * Stores the database connection used by all order queries.
     *
     * @param PDO $pdo Active database connection.
     */
    public function __construct($pdo) {
        $this->pdo = $pdo;
    }

    /**
     * Creates an order and its items atomically, saving snapshots of server/component data.
     *
```

```
     * @param array $data Order fields: user_id, session_id, total_price, name, email, phone,
comment.
     * @param array $cartItems Cart rows with server_id, component_ids (JSON) and total_price.
     * @return string Id of the new order.
     * @throws Exception If any insert fails or a server no longer exists (transaction is
rolled back).
     */
    // Create new order with items
    public function create($data, $cartItems) {
        try {
            // Transaction: either the order and all items are saved, or nothing is
            $this->pdo->beginTransaction();

            $stmt = $this->pdo->prepare("
                INSERT INTO orders (user_id, session_id, status, total_price, name, email,
phone, comment)
                VALUES (:user_id, :session_id, 'pending', :total_price, :name, :email, :phone,
:comment)
            ");

            $stmt->execute([
                ':user_id' => $data['user_id'] ?? null,
                ':session_id' => $data['session_id'] ?? null,
                ':total_price' => $data['total_price'],
                ':name' => $data['name'],
                ':email' => $data['email'],
                ':phone' => $data['phone'] ?? null,
                ':comment' => $data['comment'] ?? null,
            ]);

            $orderId = $this->pdo->lastInsertId();

            foreach ($cartItems as $item) {
                $serverStmt = $this->pdo->prepare(
                    'SELECT name, description, image, base_price FROM servers WHERE id = :id'
                );
                $serverStmt->execute([':id' => $item['server_id']]);
                $server = $serverStmt->fetch();
                if (!$server) {
                    throw new RuntimeException('Cannot save an order item for a missing
server.');
                }

                $componentIds = json_decode((string) ($item['component_ids'] ?? '[]'), true);
                $componentsById = [];
                if (is_array($componentIds) && $componentIds !== []) {
                    $componentIds = array_values(array_unique(array_map('intval',
$componentIds)));
                    $placeholders = implode(',', array_fill(0, count($componentIds), '?'));
                    $componentsStmt = $this->pdo->prepare(
                        "SELECT id, name, value, type, price FROM components WHERE id IN
($placeholders)"
                    );
                    $componentsStmt->execute($componentIds);
                    foreach ($componentsStmt->fetchAll() as $component) {
                        $componentsById[(int) $component['id']] = $component;
                    }
                }

                $componentSnapshot = [];
                // Snapshot keeps the order accurate even if components are later changed or
deleted
                foreach ($componentIds ?? [] as $componentId) {
                    if (isset($componentsById[$componentId])) {
                        $componentSnapshot[] = $componentsById[$componentId];
                    }
                }

                $stmt = $this->pdo->prepare("
```

```
                    INSERT INTO order_items (
                        order_id, server_id, component_ids, total_price,
                        server_name_snapshot, server_description_snapshot,
server_image_snapshot,
                        server_base_price_snapshot, components_snapshot
                    )
                    VALUES (
                        :order_id, :server_id, :component_ids, :total_price,
                        :server_name_snapshot, :server_description_snapshot,
:server_image_snapshot,
                        :server_base_price_snapshot, :components_snapshot
                    )
                ");

                $stmt->execute([
                    ':order_id' => $orderId,
                    ':server_id' => $item['server_id'],
                    ':component_ids' => $item['component_ids'], // JSON строка
                    ':total_price' => $item['total_price'],
                    ':server_name_snapshot' => $server['name'],
                    ':server_description_snapshot' => $server['description'] ?? '',
                    ':server_image_snapshot' => $server['image'] ?? '',
                    ':server_base_price_snapshot' => $server['base_price'],
                    ':components_snapshot' => json_encode($componentSnapshot,
JSON_THROW_ON_ERROR | JSON_UNESCAPED_UNICODE),
                ]);
            }

            $this->pdo->commit();

            return $orderId;

        } catch (Exception $e) {
            // Undo partial inserts, then let the caller handle the error
            $this->pdo->rollBack();
            throw $e;
        }
    }

    /**
     * Loads an order with its items; falls back to live data when no snapshot exists.
     *
     * @param int $orderId Order id.
     * @return array|false Order with 'items', or false if not found.
     */
    // Get order by ID
    public function get($orderId) {
        $stmt = $this->pdo->prepare("SELECT * FROM orders WHERE id = :id");
        $stmt->execute([':id' => $orderId]);
        $order = $stmt->fetch();

        if (!$order) {
            return false;
        }

        $stmt = $this->pdo->prepare("
                 SELECT oi.*,
                     COALESCE(oi.server_name_snapshot, s.name, CONCAT('Deleted server #',
oi.server_id)) AS server_name,
                     COALESCE(oi.server_description_snapshot, s.description) AS
server_description,
                     COALESCE(oi.server_image_snapshot, s.image) AS image,
                     COALESCE(oi.server_base_price_snapshot, s.base_price) AS
server_base_price
            FROM order_items oi
                 LEFT JOIN servers s ON oi.server_id = s.id
            WHERE oi.order_id = :order_id
        ");
        $stmt->execute([':order_id' => $orderId]);
```

```
        $order['items'] = $stmt->fetchAll();

        //For each order item, decode component_ids and fetch component details
        foreach ($order['items'] as &$item) {
            $componentSnapshot = json_decode((string) ($item['components_snapshot'] ?? ''),
true);
            if (is_array($componentSnapshot)) {
                $item['components'] = $componentSnapshot;
                continue;
            }

            $componentIds = json_decode($item['component_ids'], true);
            if (!empty($componentIds)) {
                $placeholders = implode(',', array_fill(0, count($componentIds), '?'));
                $stmt = $this->pdo->prepare("
                    SELECT id, name, value, type, price FROM components WHERE id IN
($placeholders)
                ");
                $stmt->execute($componentIds);
                $item['components'] = $stmt->fetchAll();
            } else {
                $item['components'] = [];
            }
        }

        return $order;
    }
    /**
     * Sets an order status unconditionally (admin use).
     *
     * @param int $orderId Order id.
     * @param string $status New status.
     * @return bool True on success.
     */
    // Update order status
    public function updateStatus($orderId, $status) {
        $stmt = $this->pdo->prepare("UPDATE orders SET status = :status WHERE id = :id");
        return $stmt->execute([
            ':status' => $status,
            ':id' => $orderId,
        ]);
    }

    /**
     * Changes status only if the order is still in the expected status.
     *
     * @param int $orderId Order id.
     * @param string $currentStatus Status the order must currently have.
     * @param string $newStatus Status to set.
     * @return bool True if exactly one row changed.
     */
    public function transitionStatus($orderId, $currentStatus, $newStatus) {
        // Checking the current status in WHERE avoids race conditions and invalid transitions
        $stmt = $this->pdo->prepare("UPDATE orders SET status = :new_status WHERE id = :id AND
status = :current_status");
        $stmt->execute([
            ':new_status' => $newStatus,
            ':id' => $orderId,
            ':current_status' => $currentStatus,
        ]);
        return $stmt->rowCount() === 1;
    }

    /**
     * Lists all orders of one user, newest first.
     *
     * @param int $userId User id.
     * @return array Order rows.
     */
```

```
    // Get all orders for a user
    public function getByUser($userId) {
        $stmt = $this->pdo->prepare("
            SELECT * FROM orders
            WHERE user_id = :user_id
            ORDER BY created_at DESC
        ");
        $stmt->execute([':user_id' => $userId]);
        return $stmt->fetchAll();
    }

    /**
     * Lists a user's orders with the number of items in each, for the profile page.
     *
     * @param int $userId User id.
     * @return array Order rows with items_count.
     */
    public function getByUserWithItemCount($userId) {
        $stmt = $this->pdo->prepare("
            SELECT o.*,
                   (SELECT COUNT(*) FROM order_items oi WHERE oi.order_id = o.id) AS
items_count
            FROM orders o
            WHERE o.user_id = :user_id
            ORDER BY o.created_at DESC
        ");
        $stmt->execute([':user_id' => $userId]);
        return $stmt->fetchAll();
    }

    /**
     * Lists all orders for the admin panel, optionally filtered by status.
     *
     * @param string|null $status Status to filter by, or null for all.
     * @return array Order rows with account_username.
     */
    public function getAllAdmin($status = null) {
        $sql = "
            SELECT o.*, u.username AS account_username
            FROM orders o
            LEFT JOIN users u ON u.id = o.user_id
        ";

        if ($status !== null) {
            $sql .= " WHERE o.status = :status";
        }

        $sql .= "
            ORDER BY o.created_at DESC, o.id DESC
        ";

        $stmt = $this->pdo->prepare($sql);
        if ($status !== null) {
            $stmt->bindValue(':status', $status);
        }
        $stmt->execute();
        return $stmt->fetchAll();
    }
}
```

Fails `src/controllers/AuthController.php` – reģistrācija, autorizācija, iziešana un e-pasta adreses apstiprināšana:

```
<?php

require_once base_path('src/Models/User.php');
```

```
require_once base_path('src/Models/Cart.php');
require_once base_path('src/Services/Mailer.php');
require_once base_path('src/Helpers/Response.php');

/**
 * Handles registration, login and logout for both HTML forms and the JSON API.
 */
class AuthController {
    private $userModel;
    private $pdo;

    /**
     * Sets up the controller with the database and the user model.
     *
     * @param PDO $pdo Active database connection.
     */
    public function __construct($pdo) {
        $this->pdo = $pdo;
        $this->userModel = new User($pdo);
    }

    /**
     * Shows the registration form and processes its submission.
     *
     * @return void
     */
    public function register() {
        $registerRedirect = $this->safeLoginRedirect($_POST['redirect'] ?? $_GET['redirect']
?? null);

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $name = $_POST['name'] ?? '';
            $email = $_POST['email'] ?? '';
            $password = $_POST['password'] ?? '';
            $confPassword = $_POST['confirm_password'] ?? '';

            $errors = [];

            // CSRF check: the form must carry the token stored in the session
            if (!csrf_verify($_POST['csrf_token'] ?? null)) {
                $errors[] = 'Invalid CSRF token';
            }

            // Server-side validation; client-side checks can be bypassed
            if (empty($name)) $errors[] = 'Name is required';
            if (empty($email)) {
                $errors[] = 'Email is required';
            } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL) ||
!preg_match('/^[^@]+@[^@]+\.[a-z]{2,6}$/i', $email)) {
                $errors[] = 'Invalid email format';
            }
            if (empty($password)) $errors[] = 'Password is required';
            if (empty($confPassword)) $errors[] = 'Confirmation password is required';
            if (!empty($password) && !empty($confPassword) && $password !== $confPassword)
$errors[] = 'Passwords do not match';
            if (!empty($email) && $this->userModel->findByEmail($email)) $errors[] = 'Email
already taken';

            if (empty($errors)) {
                // Random token proves ownership of the email address; the user model hashes
the password
                $verificationToken = bin2hex(random_bytes(32));
                $this->userModel->create($name, $email, $password, $verificationToken);
                Mailer::sendVerificationEmail($email, $name, $verificationToken);
                header('Location: ' . BASE_URL . '/login.php?redirect=' .
rawurlencode($registerRedirect));
                exit;
            }
```

```
        }

        require base_path('templates/header.php');
        require base_path('templates/register.php');
        require base_path('templates/footer.php');
    }

    /**
     * Shows the login form and authenticates the submitted credentials.
     *
     * @return void
     */
    public function login() {
        $loginRedirect = $this->safeLoginRedirect($_POST['redirect'] ?? $_GET['redirect'] ??
null);

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $email = $_POST['email'] ?? '';
            $password = $_POST['password'] ?? '';

            // CSRF check before touching credentials
            if (!csrf_verify($_POST['csrf_token'] ?? null)) {
                $error = 'Invalid CSRF token';
            } else {
                $user = $this->userModel->findByEmail($email);

            // password_verify compares against the stored hash; no plain-text passwords
            if ($user && password_verify($password, $user['password'])) {
                if (session_status() === PHP_SESSION_NONE) {
                    session_start();
                }
                $_SESSION['user_id'] = $user['id'];
                $_SESSION['user_name'] = $user['username'];
                $_SESSION['user_role'] = $user['role'];
                $_SESSION['email_verified_at'] = $user['email_verified_at'];

            // Attach the guest cart to the logged-in user so it is not lost
            $cartModel = new Cart($this->pdo);
            $cart = $cartModel->getOrCreateCart(session_id());
            $stmt = $this->pdo->prepare("UPDATE carts SET user_id = :user_id WHERE id =
:cart_id");
            $stmt->execute([
                ':user_id' => $user['id'],
                ':cart_id' => $cart['id']
            ]);

                // Unverified customers are kept off protected pages until they confirm their
email
                $destination = $user['role'] !== 'admin' && $user['email_verified_at'] ===
null
                    ? 'index.php'
                    : $loginRedirect;
                header('Location: ' . BASE_URL . '/' . $destination);
                exit;
            }

                // Same message for unknown email and wrong password to avoid user enumeration
                $error = 'Invalid email or password';
            }
        }

        require base_path('templates/header.php');
        require base_path('templates/login.php');
        require base_path('templates/footer.php');
    }

    /**
```

```
     * Restricts post-login redirects to a whitelist to prevent open redirects.
     *
     * @param mixed $redirect Requested redirect target.
     * @return string Safe relative page name.
     */
    private function safeLoginRedirect($redirect): string {
        if (!is_string($redirect)) {
            return 'index.php';
        }

        $redirect = ltrim($redirect, '/');
        return in_array($redirect, ['checkout.php', 'my-orders.php'], true)
            ? $redirect
            : 'index.php';
    }

    /**
     * JSON login endpoint; starts a fresh session and moves the guest cart to the user.
     *
     * @return void
     */
    public function loginApi(): void {
        $data = $this->readApiPost();
        $email = $this->stringValue($data['email'] ?? null);
        $password = $this->stringValue($data['password'] ?? null);

        if ($email === '' || $password === '') {
            http_response_code(400);
            Response::json(['success' => false, 'message' => 'Email and password are
required']);
        }

        // Generic 401 message so attackers cannot tell which part was wrong
        $user = $this->userModel->findByEmail($email);
        if (!$user || !password_verify($password, $user['password'])) {
            http_response_code(401);
            Response::json(['success' => false, 'message' => 'Invalid email or password']);
        }

        // Regenerate the session id after login to prevent session fixation
        $oldSessionId = session_id();
        if (!session_regenerate_id(true)) {
            http_response_code(500);
            Response::json(['success' => false, 'message' => 'Could not start a secure
session']);
        }

        $_SESSION['user_id'] = $user['id'];
        $_SESSION['user_name'] = $user['username'];
        $_SESSION['user_role'] = $user['role'];
        $_SESSION['email_verified_at'] = $user['email_verified_at'];
        $_SESSION['session_id'] = session_id();

        // The cart follows the regenerated session id and is linked to the user
        $cartStmt = $this->pdo->prepare(
            'UPDATE carts SET user_id = :user_id, session_id = :new_session_id WHERE
session_id = :old_session_id'
        );
        $cartStmt->execute([
            ':user_id' => $user['id'],
            ':new_session_id' => session_id(),
            ':old_session_id' => $oldSessionId,
        ]);

        Response::json([
            'success' => true,
            'message' => 'Login successful',
            'user' => [
```

```
                'id' => (int) $user['id'],
                'name' => $user['username'],
                'email' => $user['email'],
                'role' => $user['role'],
                'email_verified' => $user['email_verified_at'] !== null || $user['role'] ===
'admin',
            ],
        ]);
    }

    /**
     * JSON registration endpoint with server-side validation and email verification.
     *
     * @return void
     */
    public function registerApi(): void {
        $data = $this->readApiPost();
        $name = trim($this->stringValue($data['name'] ?? null));
        $email = trim($this->stringValue($data['email'] ?? null));
        $password = $this->stringValue($data['password'] ?? null);
        $confirmation = $this->stringValue($data['confirm_password'] ?? null);

        // Validate all input on the server before creating the account
        if ($name === '' || $email === '' || $password === '' || $confirmation === '') {
            http_response_code(400);
            Response::json(['success' => false, 'message' => 'Name, email, password, and
confirmation are required']);
        }
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            http_response_code(400);
            Response::json(['success' => false, 'message' => 'Invalid email format']);
        }
        if (strlen($password) < 8) {
            http_response_code(400);
            Response::json(['success' => false, 'message' => 'Password must be at least 8
characters']);
        }
        if ($password !== $confirmation) {
            http_response_code(400);
            Response::json(['success' => false, 'message' => 'Passwords do not match']);
        }
        // 409 Conflict: the email is already registered
        if ($this->userModel->findByEmail($email)) {
            http_response_code(409);
            Response::json(['success' => false, 'message' => 'Email already taken']);
        }

        // Account is created with a random verification token that is sent by email
        $verificationToken = bin2hex(random_bytes(32));
        $this->userModel->create($name, $email, $password, $verificationToken);
        $emailSent = Mailer::sendVerificationEmail($email, $name, $verificationToken);

        http_response_code(201);
        Response::json([
            'success' => true,
            'message' => $emailSent
                ? 'Registration successful. Check your email to verify your account.'
                : 'Registration successful, but the verification email could not be sent.',
            'verificationEmailSent' => $emailSent,
        ]);
    }

    /**
     * JSON logout endpoint: clears session data and expires the session cookie.
     *
     * @return void
     */
    public function logoutApi(): void {
        $this->readApiPost();
```

```
        $_SESSION = [];
        // Expire the cookie in the browser as well, not only the server-side data
        if (ini_get('session.use_cookies')) {
            $cookie = session_get_cookie_params();
            setcookie(session_name(), '', [
                'expires' => time() - 42000,
                'path' => $cookie['path'],
                'domain' => $cookie['domain'],
                'secure' => $cookie['secure'],
                'httponly' => $cookie['httponly'],
                'samesite' => $cookie['samesite'] ?? 'Lax',
            ]);
        }
        if (session_status() === PHP_SESSION_ACTIVE) {
            session_destroy();
        }

        Response::json(['success' => true, 'message' => 'Logout successful']);
    }

    /**
     * Starts the session, enforces POST + CSRF and returns the parsed request body.
     *
     * @return array Request data from form fields or a JSON body.
     */
    private function readApiPost(): array {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
            http_response_code(405);
            Response::json(['success' => false, 'message' => 'Request must be POST']);
        }

        $data = $_POST;
        if (stripos($_SERVER['CONTENT_TYPE'] ?? '', 'application/json') !== false) {
            $decoded = json_decode(file_get_contents('php://input'), true);
            if (!is_array($decoded)) {
                http_response_code(400);
                Response::json(['success' => false, 'message' => 'Invalid JSON request']);
            }
            $data = $decoded;
        }

        // CSRF token may come from a header (fetch/AJAX) or from the body
        $csrfToken = $_SERVER['HTTP_X_CSRF_TOKEN'] ?? ($data['csrf_token'] ?? null);
        if (!is_string($csrfToken) || !csrf_verify($csrfToken)) {
            http_response_code(403);
            Response::json(['success' => false, 'message' => 'Invalid CSRF token']);
        }

        return $data;
    }

    /**
     * Coerces a request value to a string so arrays or null cannot break validation.
     *
     * @param mixed $value Raw request value.
     * @return string The value if it is a string, otherwise an empty string.
     */
    private function stringValue($value): string {
        return is_string($value) ? $value : '';
    }
}
```

Fails `src/controllers/OrderController.php` – pasūtījuma noformēšana, apmaksa, atcelšana un pasūtījumu apskate:

```
<?php

namespace App\Controllers;

require_once \base_path('src/Models/Order.php');
require_once \base_path('src/Models/Cart.php');
require_once \base_path('src/Models/User.php');
require_once \base_path('src/Helpers/Response.php');

/**
 * Renders checkout and order pages and serves order data, enforcing ownership/admin access.
 */
class OrderController {
    private const STATUS_COLORS = [
        'pending' => ['bg' => '#fff3cd', 'text' => '#856404', 'label' => 'Awaiting Payment'],
        'paid' => ['bg' => '#dff3ee', 'text' => '#12624f', 'label' => 'Paid'],
        'processing' => ['bg' => '#d1ecf1', 'text' => '#0c5460', 'label' => 'Processing'],
        'completed' => ['bg' => '#d4edda', 'text' => '#155724', 'label' => 'Completed'],
        'cancelled' => ['bg' => '#f8d7da', 'text' => '#721c24', 'label' => 'Cancelled'],
    ];

    private $orderModel;
    private $pdo;

    /**
     * Sets up the controller with the database and the order model.
     *
     * @param \PDO $pdo Active database connection.
     */
    public function __construct($pdo) {
        $this->pdo = $pdo;
        $this->orderModel = new \Order($pdo);
    }

    /**
     * Shows the checkout page for a verified customer with a non-empty cart.
     *
     * @return void
     */
    public function checkout(): void {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        // Prevent caching so a stale cart/price is never shown after navigation
        header('Cache-Control: no-store, private, max-age=0');
        header('Pragma: no-cache');
        header('Expires: 0');
        // Admins manage orders and do not buy
        if (($_SESSION['user_role'] ?? '') === 'admin') {
            header('Location: ' . \BASE_URL . '/admin/orders.php');
            exit;
        }
        // Guests must log in first
        if (empty($_SESSION['user_id'])) {
            header('Location: ' . \BASE_URL . '/login.php?redirect=checkout.php');
            exit;
        }

        // Re-read the user from the DB: the session may refer to a deleted account
        $checkoutUser = (new \User($this->pdo))->findById($_SESSION['user_id']);
        if (!$checkoutUser) {
            $_SESSION = [];
            session_destroy();
```

```
            header('Location: ' . \BASE_URL . '/login.php?redirect=checkout.php');
            exit;
        }
        $_SESSION['email_verified_at'] = $checkoutUser['email_verified_at'];
        // Only users with a verified email may place orders
        if ($checkoutUser['email_verified_at'] === null) {
            header('Location: ' . \BASE_URL . '/index.php');
            exit;
        }

        if (!isset($_SESSION['session_id'])) {
            $_SESSION['session_id'] = session_id();
        }

        $cartModel = new \Cart($this->pdo);
        $cart = $cartModel->getOrCreateCart($_SESSION['session_id']);
        $cartItems = $cartModel->getItems($cart['id']);

        // Nothing to buy: send the user back to the catalog
        if (empty($cartItems)) {
            header('Location: ' . \BASE_URL . '/index.php');
            exit;
        }

        $checkoutContact = [
            'name' => '',
            'email' => '',
            'phone' => '',
        ];
        $checkoutContact = [
            'name' => $checkoutUser['username'] ?? '',
            'email' => $checkoutUser['email'] ?? '',
            'phone' => $checkoutUser['mobile_phone'] ?? '',
        ];

        $totalPrice = array_sum(array_column($cartItems, 'total_price'));
        require_once \base_path('templates/header.php');
        require_once \base_path('templates/checkout.php');
        require_once \base_path('templates/footer.php');
    }

    /**
     * Shows the current customer's order history.
     *
     * @return void
     */
    public function myOrders(): void {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        if (!isset($_SESSION['user_id'])) {
            header('Location: ' . \BASE_URL . '/login.php?redirect=my-orders.php');
            exit;
        }
        if (($_SESSION['user_role'] ?? '') === 'admin') {
            header('Location: ' . \BASE_URL . '/admin/orders.php');
            exit;
        }

        // Only the user's own orders are loaded, using the id from the session
        $orders = $this->orderModel->getByUserWithItemCount($_SESSION['user_id']);
        foreach ($orders as &$order) {
            $order['current_status'] = self::STATUS_COLORS[$order['status']] ??
self::STATUS_COLORS['pending'];
        }
        unset($order);
```

```
        require_once \base_path('templates/header.php');
        require_once \base_path('templates/my-orders.php');
        require_once \base_path('templates/footer.php');
    }

    /**
     * Renders a full order page if the current user may view it.
     *
     * @param mixed $requestedOrderId Untrusted order id from the request.
     * @return void
     */
    public function showOrder($requestedOrderId): void {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        // Validate the id as a positive integer before querying
        $orderId = filter_var($requestedOrderId, FILTER_VALIDATE_INT);
        if ($orderId === false || $orderId < 1) {
            header('Location: ' . \BASE_URL . '/index.php?message=Order%20not%20found');
            exit;
        }

        $order = $this->orderModel->get($orderId);
        if (!$order) {
            header('Location: ' . \BASE_URL . '/index.php?message=Order%20not%20found');
            exit;
        }

        // Access check: only the owner or an admin
        if (!$this->canViewOrder($order)) {
            http_response_code(403);
            header('Location: ' . \BASE_URL . '/index.php?message=Access%20denied');
            exit;
        }

        $currentStatus = self::STATUS_COLORS[$order['status']] ??
self::STATUS_COLORS['pending'];
        require_once \base_path('templates/header.php');
        require_once \base_path('templates/order.php');
        require_once \base_path('templates/footer.php');
    }

    /**
     * Renders the order details fragment (used by the order lists) with plain-text errors.
     *
     * @param mixed $requestedOrderId Untrusted order id from the request.
     * @return void
     */
    public function details($requestedOrderId): void {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        // Validate the id as a positive integer before querying
        $orderId = filter_var($requestedOrderId, FILTER_VALIDATE_INT);
        if ($orderId === false || $orderId < 1) {
            http_response_code(400);
            exit('Invalid order ID');
        }

        $order = $this->orderModel->get($orderId);
        if (!$order) {
            http_response_code(404);
            exit('Order not found');
        }

        // Access check: only the owner or an admin
```

```
        if (!$this->canViewOrder($order)) {
            http_response_code(403);
            exit('Access denied');
        }

        $isAdmin = ($_SESSION['user_role'] ?? '') === 'admin';
        $currentStatus = self::STATUS_COLORS[$order['status']] ??
self::STATUS_COLORS['pending'];
        $backUrl = $isAdmin ? \BASE_URL . '/admin/orders.php' : \BASE_URL . '/my-orders.php';
        $backLabel = $isAdmin ? 'Back to Order Management' : 'Back to Order History';

        require_once \base_path('templates/order-details.php');
    }

    /**
     * Returns an order as JSON if the current user may view it.
     *
     * @param mixed $requestedOrderId Untrusted order id from the request.
     * @return void
     */
    public function getApi($requestedOrderId): void {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }

        // Validate the id as a positive integer before querying
        $orderId = filter_var($requestedOrderId, FILTER_VALIDATE_INT);
        if ($orderId === false || $orderId < 1) {
            http_response_code(400);
            \Response::json(['success' => false, 'message' => 'Invalid order ID']);
        }

        $order = $this->orderModel->get($orderId);
        if (!$order) {
            http_response_code(404);
            \Response::json(['success' => false, 'message' => 'Order not found']);
        }

        // Access check: only the owner or an admin
        if (!$this->canViewOrder($order)) {
            http_response_code(403);
            \Response::json(['success' => false, 'message' => 'Access denied']);
        }

        \Response::json(['success' => true, 'order' => $order]);
    }

    /**
     * Decides whether the current session user may see the order (owner or admin).
     *
     * @param array $order Order row containing user_id.
     * @return bool True if access is allowed.
     */
    private function canViewOrder(array $order): bool {
        if (($_SESSION['user_role'] ?? '') === 'admin') {
            return true;
        }

        return isset($_SESSION['user_id'])
            && $order['user_id'] !== null
            && (int) $order['user_id'] === (int) $_SESSION['user_id'];
    }
}
```

Fails `src/services/Mailer.php` – e-pasta vēstuļu sūtīšana ar PHPMailer:

```
<?php

/**
 * Sends transactional emails (invoice, account verification) through PHPMailer.
 */
class Mailer {
    /**
     * Emails the order invoice as a PDF attachment.
     *
     * @param string $email Recipient address.
     * @param string $name Recipient name.
     * @param int $orderId Order id, used in subject and file name.
     * @param string $pdf Binary PDF content.
     * @return bool True if the email was sent.
     */
    public static function sendInvoiceEmail($email, $name, $orderId, $pdf): bool {
        // Do not send to malformed addresses or without an invoice
        if (!filter_var($email, FILTER_VALIDATE_EMAIL) || $pdf === '') {
            return false;
        }

        $subject = 'Invoice for order #' . (int) $orderId;
        $message = "Hello {$name},\n\n"
            . "Thank you for your order. Your invoice is attached to this email.\r\n\r\n"
            . "Server Catalog";
        $filename = 'invoice-order-' . (int) $orderId . '.pdf';

        return self::send($email, $name, $subject, $message, $filename, $pdf);
    }

    /**
     * Emails the account verification link.
     *
     * @param string $email Recipient address.
     * @param string $name Recipient name.
     * @param string $token Random verification token.
     * @return bool True if the email was sent.
     */
    public static function sendVerificationEmail($email, $name, $token): bool {
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            return false;
        }

        // urlencode keeps the token safe inside the query string
        $verificationUrl = BASE_URL . '/verify.php?token=' . urlencode($token);
        $subject = 'Verify your Server Catalog account';
        $message = "Hello {$name},\n\n"
            . "Please verify your account by opening this link:\n"
            . $verificationUrl . "\n\n"
            . "If you did not register, you can ignore this email.";

        return self::send($email, $name, $subject, $message);
    }

    /**
     * Configures PHPMailer from environment settings and delivers one message.
     *
     * @param string $email Recipient address.
     * @param string $name Recipient name.
     * @param string $subject Message subject.
     * @param string $message Plain-text body.
     * @param string|null $attachmentName Attachment file name, if any.
     * @param string|null $attachment Binary PDF attachment, if any.
     * @return bool True on success, false if delivery failed (error is logged).
     */
```

```
    private static function send(
        string $email,
        string $name,
        string $subject,
        string $message,
        ?string $attachmentName = null,
        ?string $attachment = null
    ): bool {
        $mailer = new PHPMailer\PHPMailer\PHPMailer(true);

        try {
            // SMTP settings come from the environment so credentials are not stored in code
            $host = trim(self::env('MAILER_HOST'));
            if ($host !== '') {
                $mailer->isSMTP();
                $mailer->Host = $host;
                $mailer->Port = (int) self::env('MAILER_PORT', '1025');
                $mailer->SMTPAuth = self::env('MAILER_USERNAME') !== '';
                $mailer->Username = self::env('MAILER_USERNAME');
                $mailer->Password = self::env('MAILER_PASSWORD');

                $encryption = strtolower(self::env('MAILER_ENCRYPTION'));
                if ($encryption === 'tls') {
                    $mailer->SMTPSecure = PHPMailer\PHPMailer\PHPMailer::ENCRYPTION_STARTTLS;
                } elseif ($encryption === 'ssl') {
                    $mailer->SMTPSecure = PHPMailer\PHPMailer\PHPMailer::ENCRYPTION_SMTPS;
                }
            } else {
                $mailer->isMail();
            }

            $mailer->CharSet = PHPMailer\PHPMailer\PHPMailer::CHARSET_UTF8;
            $mailer->setFrom(
                self::env('MAILER_FROM_ADDRESS', 'servershopcatalog@gmail.com'),
                self::env('MAILER_FROM_NAME', 'Server Catalog')
            );
            $mailer->addAddress($email, $name);
            $mailer->Subject = $subject;
            $mailer->Body = $message;
            // Plain text avoids HTML/XSS injection through the user's name
            $mailer->isHTML(false);

            if ($attachment !== null && $attachmentName !== null) {
                $mailer->addStringAttachment($attachment, $attachmentName, 'base64',
'application/pdf');
            }

            return $mailer->send();
        } catch (Throwable $error) {
            // Mail failure must not break the user flow; log it and report false
            error_log('Email delivery failed: ' . $error->getMessage());
            return false;
        }
    }

    /**
     * Reads a configuration value from the environment.
     *
     * @param string $name Variable name.
     * @param string $default Value used when the variable is not set.
     * @return string The value or the default.
     */
    private static function env(string $name, string $default = ''): string {
        $value = $_ENV[$name] ?? $_SERVER[$name] ?? getenv($name);
        return is_string($value) ? $value : $default;
    }
}
```

Fails `src/services/InvoiceGenerator.php` – PDF rēķina ģenerēšana ar Dompdf:

```
<?php

/**
 * Renders order invoices to PDF with Dompdf.
 */
class InvoiceGenerator {
    /**
     * Renders the invoice template for an order into PDF bytes.
     *
     * @param array $order Order data with items (used by the template).
     * @param array $invoiceSeller Seller details shown on the invoice.
     * @return string Binary PDF content.
     */
    public static function renderPdf(array $order, array $invoiceSeller): string {
        require_once base_path('vendor/autoload.php');

        // Map internal status codes to readable labels, with a fallback for unknown ones
        $statusLabels = [
            'pending' => 'Awaiting Payment',
            'paid' => 'Paid',
            'processing' => 'Processing',
            'completed' => 'Completed',
            'cancelled' => 'Cancelled',
        ];
        $orderStatus = $statusLabels[$order['status']] ?? ucfirst($order['status']);

        // Capture the template output as HTML instead of sending it to the browser
        ob_start();
        require base_path('templates/invoice.php');
        $invoiceHtml = ob_get_clean();

        $dompdf = new Dompdf\Dompdf();
        $dompdf->loadHtml($invoiceHtml, 'UTF-8');
        $dompdf->setPaper('A4', 'portrait');
        $dompdf->render();

        return $dompdf->output();
    }
}
```

Fails `public/js/configurator.js` – servera konfiguratora KnockoutJS skatu modelis:

```
/**
 * Knockout view model that keeps a live price estimate for the server configurator.
 *
 * @param {number} basePrice - Base price of the selected server.
 */
function ConfiguratorViewModel(basePrice) {
    var self = this;

    self.basePrice = basePrice;
    self.selectedComponents = ko.observableArray([]);
    self.selectedByType = {};

    /**
     * Records a component choice, replacing the previous one of the same type.
     *
     * @param {string} type - Component type (e.g. RAM, CPU).
     * @param {string|number} price - Price of the chosen component.
     * @returns {void}
     */
    self.selectComponent = function(type, price) {
        // Only one component per type may be counted in the total
        if (self.selectedByType[type]) {
```

```
            self.selectedComponents.remove(self.selectedByType[type]);
        }
        self.selectedByType[type] = price;
        self.selectedComponents.push(price);
    };

    /**
     * Base price plus selected components; display-only, the server recalculates the real
price.
     *
     * @returns {string} Total formatted with two decimals.
     */
    self.totalPrice = ko.computed(function() {
        var total = self.basePrice;
        self.selectedComponents().forEach(function(price) {
            total += parseFloat(price);
        });
        return total.toFixed(2);
    });
}

var vm = new ConfiguratorViewModel(parseFloat(document.getElementById('base-price').value));
ko.applyBindings(vm, document.getElementById('configurator-app'));

document.getElementById('add-to-cart')?.addEventListener('click', function() {
    const serverId = new URLSearchParams(window.location.search).get('id');
    const selectedInputs = document.querySelectorAll('input[type="radio"]:checked');

    const componentIds = Array.from(selectedInputs).map(input => parseInt(input.value));

    // Send only ids; the price is recalculated on the server. CSRF token protects the POST
    fetch(BASE_URL + '/api/cart/add.php', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-CSRF-Token': CSRF_TOKEN
        },
        body: JSON.stringify({
            server_id: serverId,
            component_ids: componentIds
        })
    })
    .then(res => res.json())
    .then(data => {
        if (data.success) {
            loadCart(); // Refresh cart contents even if it's already open
            openCart();
        }
    });
});
```